

La Agente Óptima: Towards Agentic Self-Driving Laboratories

Marcel Müller^{1,7,†}, Jiaru Bai^{1,7,†}, Willi Gottstein^{6,†}, Abhijoy Mandal^{2,‡}, Mohammad Nazeri^{10,‡},
Elia Savino^{11,‡}, Yanlin Fang², Sujoy Das², Sergio Pablo García Carrillo^{6,12}, Yeonghun Kang^{1,7,13},
Juan B. Pérez-Sánchez¹, Simone Pilon¹¹, Martin Fitzner¹⁴, Timothy Noël¹¹, Frank Gu^{4,6,10},
Varinia Bernales^{1,2,6,*}, Alán Aspuru-Guzik^{1,2,3,4,5,6,7,8,9,*}

¹Department of Chemistry, University of Toronto, 80 St. George St., Toronto, ON M5S 3H6, Canada

²Department of Computer Science, University of Toronto, 40 St George St., Toronto, ON M5S 2E4, Canada

³Department of Materials Science & Engineering, University of Toronto, 184 College St., Toronto, ON M5S 3E4, Canada

⁴Department of Chemical Engineering & Applied Chemistry, University of Toronto, 200 College St., Toronto, ON M5S 3E5, Canada

⁵Institute of Medical Science, 1 King's College Circle, Medical Sciences Building, Room 2374, Toronto, ON M5S 1A8, Canada

⁶Acceleration Consortium, 700 University Ave., Toronto, ON M7A 2S4, Canada

⁷Vector Institute for Artificial Intelligence, W1140-108 College St., Schwartz Reisman Innovation Campus, Toronto, ON M5G 0C6, Canada

⁸Canadian Institute for Advanced Research (CIFAR), 661 University Ave., Toronto, ON M5G 1M1, Canada

⁹NVIDIA, 431 King St W #6th, Toronto, ON M5V 1K4, Canada

¹⁰Institute of Biomedical Engineering, University of Toronto, 164 College St, Toronto, Canada

¹¹Flow Chemistry Group, van 't Hoff Institute for Molecular Sciences (HIMS), University of Amsterdam, Science Park 904, 1098 XH Amsterdam, Netherlands.

¹²Instituto de Micro y Nanotecnología, IMN-CNM, CSIC (CEI UAM+CSIC), Isaac Newton, 8, Tres Cantos, Madrid, Spain, 28760

¹³Department of Chemistry, Sungkyunkwan University, 2066 Seobu-ro, Suwon-si, Gyeonggi, Republic of Korea, 16419

¹⁴Merck KGaA, Frankfurter Str. 250, 64293 Darmstadt, Germany

[†]These authors contributed equally, [‡]these authors also contributed equally.

Self-driving laboratories (SDLs) combine automated experimentation with adaptive decision-making to accelerate scientific discovery. Their operation nevertheless often depends on human specialists who translate scientific objectives into executable closed-loop campaigns. Specialists adjust them as data and operating conditions change. Here, we present LA AGENTE ÓPTIMA, an agentic framework that constructs and supervises Bayesian optimization campaigns across computational and experimental systems while maintaining a persistent optimization state. By separating large language model (LLM) reasoning from executed campaigns, ÓPTIMA runs repetitive optimization loops consistently, returns control to the agent only when progress requires interpretation or campaign revision, and keeps every decision auditable. We evaluate ÓPTIMA across ablation studies, five digital discovery tasks, and two physical platforms. Throughout, ÓPTIMA maintained executable campaigns as both the scientific problem and execution environment evolved. In a closed-loop contact angle optimization campaign, ÓPTIMA identified and corrected a mid-run measurement failure, bringing the contact angle from 71.4° to 67.8° , just outside the $65^\circ \pm 1^\circ$ target. From this result, ÓPTIMA correctly inferred that the target was likely unattainable with the available reagents and recommended changing the formulation. In a five-day multi-objective flow-chemistry campaign, ÓPTIMA increased the yield from 30% to 59% over 23 experiments. Despite substantial inference costs, it cost less and used substantially less starting material than a human-directed campaign, while selecting a more mass-efficient operating point. These results show that LLM-based agents can make rigorous, long-running optimization campaigns accessible to domain scientists without specialist setup, expanding the scope of SDLs.

Date: September 7, 2026

Correspondence: varinia@bernales.org and alan@aspuru.com



1 Introduction

Discovering a better catalyst, ligand, or emitter means searching a design space that is usually far larger than anyone could test exhaustively (1; 2; 3). Navigating such a space effectively means deciding what to test next based on everything learned so far. Self-driving laboratories (SDLs) (4) address this by closing the loop: they combine automated laboratory infrastructure, such as robotic arms or automated liquid-dispensing platforms, with automated experiment planning, so that each round of testing informs the decision of the next. Among the methods available for this planning step, Bayesian optimization (BO) has become the algorithm of choice for many parameter- and reaction-optimization tasks in chemistry (5; 6; 7; 8; 9; 10; 11), enabling sequential, model-guided, adaptive decision-making that uses prior observations to select the next most informative or promising experiment. This makes it particularly well suited to low-data, expensive-to-evaluate black-box optimization problems (12).

In practice, however, deploying BO remains difficult for many scientists. Users typically rely either on bespoke user interfaces (UIs) developed for a particular organization or experimental setup (13), or on direct access to a BO package through its internal application programming interfaces (APIs) (9), embedded in a custom setup for a specific problem. Dedicated optimization libraries can reduce the implementation burden. General-purpose frameworks such as BoTorch and Ax support mixed parameter spaces, constraints, and parallel or asynchronous optimization, while Atlas provides capabilities specifically oriented toward experimental science and SDLs (6; 14; 10). Nevertheless, connecting an optimizer to the system that evaluates each candidate, configuring and maintaining the resulting campaign, and interpreting its decision trajectory still typically require programming and BO expertise that domain scientists may not have. Although automated experimental discovery has already been demonstrated across a range of applications (15; 16; 17; 18), making such workflows part of everyday scientific practice still depends heavily on specialist support because of these practical challenges.

At the same time, increasingly autonomous scientific workflows (19) create a complementary need for scalable decision-making systems in which human intervention does not become the rate-limiting step as the scale of such campaigns increases. A recent closed-loop catalysis campaign provides an example in which human reasoning accounted for 34.7 person-hours across 160 experiments, compared with about 1 hour of agent reasoning at twice the experimental throughput (20). Ideally, such methods should therefore both empower experimentalists through accessible, general-purpose interfaces and support reliable semi-autonomous operation at scales that would be impractical to coordinate manually.

Large language models (LLMs) have shown substantial promise across scientific discovery (21). When equipped with access to scientific tools (22), LLM-based agents offer one way to reduce the aforementioned integration burden by operating those tools through their software interfaces. Many reported applications address *in-silico* research tasks, including quantum chemistry calculations (23; 24; 25; 26; 27; 28; 29), molecular structure investigations (30; 31; 32)¹, hypothesis generation (19; 33), as well as biochemistry tasks and molecular inverse design (34; 35; 36). Wet-lab studies have extended this approach to physical experimentation: Coscientist, a GPT-4-driven autonomous agent combining web search, documentation retrieval, code execution, and laboratory automation, designed, planned, and executed chemical experiments (37), while Smith et al. (38) coupled GPT-5 to a cloud laboratory for six rounds of cell-free protein synthesis optimization. Although the latter demonstrates that an agent can sustain an optimization campaign over multiple experimental rounds, such demonstrations remain tied to execution environments tailored to particular tasks and platforms (4).

How agent reasoning should interact with dedicated optimization methods is a separate question, and one on which the current literature explores different options: Several approaches build directly on suggestions generated by LLMs (20; 39). Such approaches can benefit from the substantial chemical knowledge encoded during pretraining and have shown strong performance for reaction optimization, particularly in categorical search spaces that resemble well-represented chemistry (39). However, this advantage depends on the relevance of that prior knowledge: when the chemistry is under-studied, the search space is weakly represented in the literature, or optimization requires learning primarily from newly acquired experimental feedback, dedicated BO methods provide a complementary and often preferable strategy (39). The same holds for the reliability of

¹As another example, Anthropic reported that Claude could perform [NMR prediction and structure elucidation](#) without external scientific tools.

the search itself: in a 25-dimensional closed-loop catalysis campaign, an LLM concentrated on a narrow subset of formulations and made silent reasoning errors that were caught only by human inspection, whereas, within the same 160-experiment budget, a domain-expert team explored more diverse formulations and identified a catalyst that was more active on average than those found in two independent LLM runs (20). Taken together, these studies suggest that LLMs and BO are best viewed as complementary components: agent reasoning can contribute chemical knowledge, formulate objectives, and interact flexibly with experimental systems, whereas dedicated BO provides a principled mechanism for data-driven exploration and optimization when prior knowledge is sparse, unreliable, or insufficient. This complementarity is also supported by a recent study by Gupta et al. (40), who found that replacing measured outcomes with randomly permuted ones left agent performance essentially unchanged, indicating that candidate selection was driven by pretrained priors rather than by an updated posterior, and that combining these priors with an explicit acquisition step recovered the lost performance.

Other approaches have sought to enhance BO’s performance with LLMs. For example, Mottafegh and Ahn (41) used LLM-based agents to incorporate human knowledge-driven biases into structured priors for Gaussian processes (GPs). Similarly, Cissé et al. (42) showed that LLM/BO hybrids outperform BO alone in closed-loop optimization, in particular when early exploration is warm-started by LLM-generated hypotheses, although purely LLM-driven optimization matched or surpassed the hybrid in some settings. In CICERO, Ritchhart et al. (43) demonstrated an agentic workflow in which LLMs formulate selective-precipitation campaigns and can optionally invoke BO for batched experimental refinement. However, in this case, BO remains embedded in a domain-specific separations workflow with a tightly coupled experimental interface, target definition, and objective formulation. Brunzema et al. (44) placed an LLM agent at the centre of the optimization loop, delegating surrogate modelling and acquisition optimization to a Bayesian backend that the agent can reconfigure at run time, and reported that on reaction-optimization benchmarks neither an LLM-only optimizer nor standard BO alone matched this combination. Their evaluation, however, is carried out entirely on synthetic and analytical benchmark functions. Across these examples, the optimizer remains private to a single agent or workflow and is exercised either on benchmark objectives or within a single experimental domain. This motivates a general architecture in which the optimizer is a shared and persistent component that supports both BO-driven search and direct experimental proposals from the agent across heterogeneous computational and experimental platforms.

Here, we present LA AGENTE ÓPTIMA, an agentic framework for autonomously constructing and managing BO campaigns across computational and experimental systems. It separates agent reasoning from programmatic execution and a persistent, typed optimizer state. A user states a scientific objective in natural language; the agent configures the campaign, runs it, diagnoses and repairs problems as they arise, and keeps every decision it makes auditable afterward. After outlining the architecture of ÓPTIMA and its building blocks, we compare it against alternative variants and benchmark its performance. We then demonstrate how ÓPTIMA can be coupled to electronic structure theory experiments using EL AGENTE GRÁFICO and experimental platforms more generally by agentizing the liquid formulation platform RAISE (45) and flow chemistry platform RoboChem-Flex (46).

2 Implementation

2.1 Agentic framework

Fig. 1 illustrates the overall agentic architecture. LA AGENTE ÓPTIMA extends EL AGENTE GRÁFICO (28) by enabling it to construct and supervise BO campaigns. Each chat room provides a shared, filesystem-backed workspace that is accessible to the coordinating agent and its BO specialist and exposed to the user through the frontend. The coordinating agent interprets the scientific objective and campaign results, while the specialist implements and validates the campaign in a separate model context, keeping implementation details out of the coordinating agent’s context. The specialist maintains a persistent memory of implementation lessons acquired during campaign authoring and validation, allowing subsequent tasks to build on established solutions and design decisions.

For each task, the specialist writes a Python campaign program and its operating instructions to the shared workspace. The program executes the repeated BO loop with consistent settings, leaving the coordinating

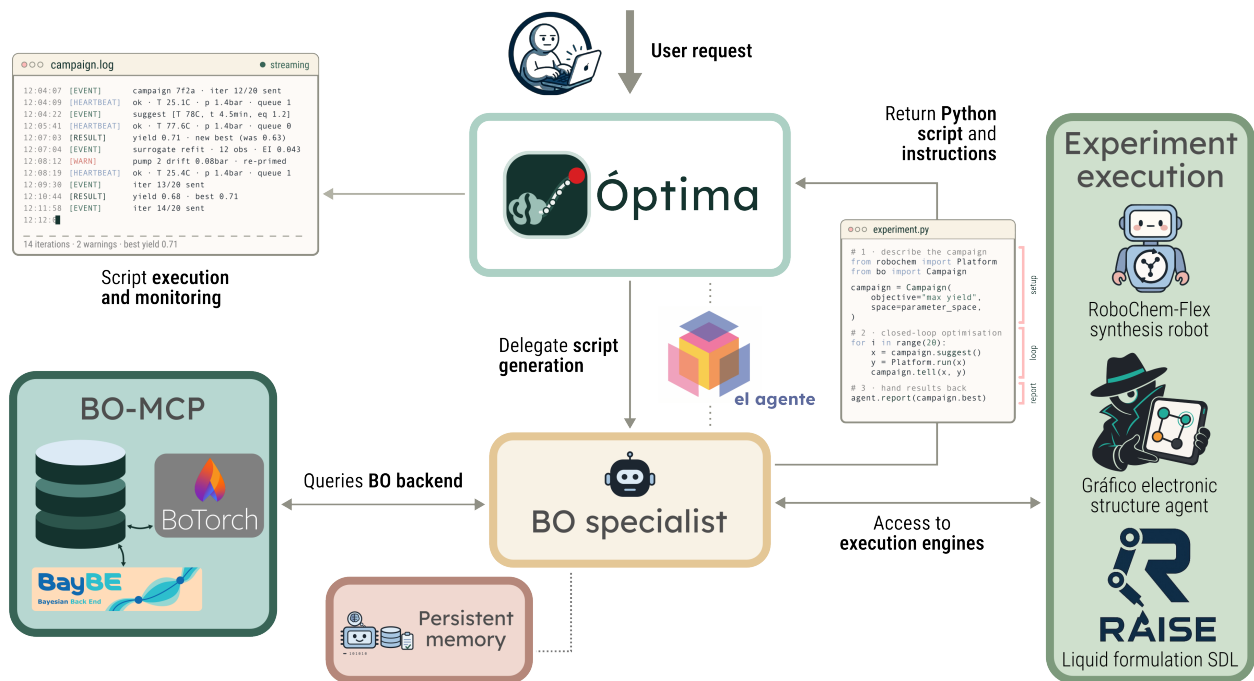


Figure 1 Agentic architecture of ÓPTIMA. A user request is interpreted by ÓPTIMA, which delegates campaign authoring to the BO specialist. The specialist interacts with BO-MCP and the appropriate execution system to produce a Python campaign program and operating instructions. The specialist also maintains a persistent memory of reusable implementation notes acquired during campaign authoring and validation. The generated program is executed and monitored by ÓPTIMA. BO-MCP stores optimizer state (see Sec. 2.2), and computational or experimental platforms perform the evaluations. Campaign artifacts remain available in the shared room workspace for inspection by the agent and the user.

agent to interpret progress, revise the campaign, or propose experiments directly when scientific judgment is required. The generated code and execution outputs remain inspectable by both agents and the user in the workspace, while staying outside the coordinating agent’s context window until selectively inspected.

In the deployment used here, the coordinating agent and specialist run in the same application container, while BO-MCP runs as an independent service on the same Docker network (Sec. 2.2). This also separates the compute used for BO campaign management, model fitting and acquisition optimization, from that used for agent reasoning, so each can be provisioned and scaled independently. Evaluation systems retain their existing interfaces: electronic structure workflows are accessed through native LLM tool calls, RAISE through an MQTT bridge, and RoboChem-Flex through an authenticated HTTP API. The specialist adapts the campaign code to each evaluator, allowing existing systems to be integrated into BO campaigns without requiring a dedicated agent protocol.

During long-running campaigns, ÓPTIMA launches the campaign program as a monitored background task. The monitoring system reads incremental process output and filters progress and heartbeat events. The resulting steering messages enter an active ÓPTIMA run at the next model invocation or wake the agent when it is idle, and the same events are mirrored to the frontend. This event-driven path prompts ÓPTIMA to notify the user and intervene when required without loading the complete polling log into the LLM context. The same mechanism supervises computational workflows and experimental platforms.

2.2 BO-MCP

BO-MCP exposes BO as a service through model context protocol (MCP) and through a REST API available to HTTP clients on an internal Docker network. A thin tool layer forwards each request to a protocol-neutral operations layer that handles campaign lookup, state transitions, caching, and provenance. The optimization

engine sits behind a backend interface and is therefore replaceable: BayBE (9) serves as the default backend, a BoTorch-based engine is also available (6), and additional backends that implement the same interface can be registered as plugins and discovered through Python entry points. For categorical chemical parameters, the BayBE backend supports both standard chemistry-aware representations, including Mordred and RDKit descriptors and Morgan fingerprints, and user-supplied numerical descriptors, as used several times throughout this work. BO-MCP also maintains a queryable, per-campaign record of actions, providing a transparent account of what was done and when. It also helps interpret the optimizer rather than treating it as a black box: the BoTorch backend reports kernel lengthscales for the model’s computational input dimensions, whereas BayBE provides SHAP-based importance for either experimental parameters or their computational representations. For each suggestion, the system also records how it was generated, including the acquisition function and value together with the model’s predictions and uncertainty. Together, these design choices make BO-MCP easy to use while still providing full transparency: an agent can run a full optimization campaign through simple tool calls or API requests, while campaign actions and suggestions remain inspectable and explainable afterwards.

2.3 PySCF execution graph

LA AGENTE ÓPTIMA uses the PySCF execution graph from EL AGENTE GRÁFICO as an evaluator for quantum chemistry tasks that can be solved by PySCF. The generated Python campaign code invokes the graph tool for each candidate with the same computational settings, ensuring that all evaluations follow a common protocol. Typed nodes and admissible transitions coordinate the required calculations and validation steps, while scientific state passes directly between nodes via Python memory without entering the LLM context. The graph returns a compact, structured result from which the campaign program derives objective values and submits valid observations to BO-MCP. Independent candidates can be evaluated concurrently in isolated GPU processes, providing consistent execution and failure containment across the campaign.

2.4 Agent-to-agent integration with EL AGENTE ESTRUCTURAL

Molecular-structure operations are delegated over an agent-to-agent (A2A) interface to EL AGENTE ESTRUCTURAL, an independently deployed agent that owns the generation and editing of molecular structures (32). Communication follows the A2A protocol via HTTP: a request is submitted as a task, and the client polls that task until it reaches a terminal state. As with BO-MCP, EL AGENTE ESTRUCTURAL is reachable in two ways that share one endpoint: LA AGENTE ÓPTIMA calls it as a tool for single interactive requests, whereas the BO specialist additionally constructs the same client directly inside generated campaign scripts, so that long optimization loops do not depend on LLM tool calls. Either path passes the current chat room as the context identifier, which preserves conversational context across follow-up requests and directs EL AGENTE ESTRUCTURAL to write into that room’s workspace. The protocol itself therefore carries only task state and compact textual results, while the structure files remain available to campaign scripts through the shared workspace, without ever serializing atomic coordinates into an LLM context.

2.5 RAISE platform access

The RAISE platform (45) is an SDL built to link liquid formulation to interfacial property measurement in a closed loop; the present integration uses its first demonstrated capability, static contact angle measurement (45). RAISE was benchmarked against conventional goniometry and showed similar accuracy, while reducing measurement variability by 20–62%, depending on the substrate. It also achieved a throughput of about one contact angle measurement per minute. It is exposed to LA AGENTE ÓPTIMA through a typed `run_raise_experiment` interface, which can be invoked as an agent tool or imported by generated Python campaign programs. Each call sends a formulation to the physical platform through an MQTT bridge and waits for the returned static contact angle (45). The interface verifies that the returned formulation matches the submitted candidate before making the measurement available to the campaign program. Because the current bridge uses shared request and response channels without identifiers, experiments are serialized to prevent responses from being assigned to the wrong candidate. Validated measurements can then be submitted to BO-MCP as observations, allowing RAISE to serve as a physical evaluator.

2.6 RoboChem-Flex platform access

RoboChem-Flex is a low-cost, modular SDL for synthetic organic chemistry that combines Cartesian-robot samplers, custom syringe pumps, and reconfigurable flow reactors with Python-based device control and inline analytical instruments (46). In LA AGENTE ÓPTIMA, the platform is connected through *RoBridge*, an authenticated and stateful HTTP service deployed on the robot computer alongside the platform control software and securely exposed online through Cloudflare. The integration exposes tools that retrieve the platform manual and live OpenAPI description, inspect individual operations and schemas, and query read-only resources such as status and capabilities. Before generating an executable request, the agent compares the proposed experimental conditions with the reactor configuration, analytical method, and parameter constraints reported by the live platform API. A dedicated *bo-robotflex-specialist* is equipped with both RoboChem-Flex and BO-MCP API-inspection toolsets that ÓPTIMA can delegate campaign construction to. *RoBridge* provides the complementary deterministic safeguards: it admits only one active campaign, rejects requests inconsistent with its explicit robot state, validates conditions against live capabilities, and requires a locally authenticated technician to certify the physical vial layout before chemistry can begin. Thus, ÓPTIMA retains conceptual control over the complete closed loop while querying both services for their authoritative current state; optimization state remains in BO-MCP, platform and safety state remains in *RoBridge*, and invalid actions are rejected on either side. The technical implementation is detailed in Supporting Information (SI), Sec. C.6.2.

3 Results

We evaluate LA AGENTE ÓPTIMA in three settings of increasing interaction with the physical world: architectural benchmarks against controlled ablations, purely computational discovery campaigns, and closed-loop optimization on two live experimental platforms. The benchmarks quantify optimization performance and resource use across repeated runs, whereas the case studies assess whether ÓPTIMA turns a stated objective into a valid, reproducible campaign with a complete experimental record.

Overall, the benchmarks against controlled ablations support combining established BO implementations with programmatic execution and provide qualified support for specialist-subagent delegation, particularly when campaign design materially affects the search. They further show that optimization quality and operational robustness are distinct, task-dependent properties. The remainder of the main text focuses on the applied showcases; complete benchmark results are reported in the SI (Sec. B).

3.1 Digital discovery campaigns

3.1.1 Combinatorial molecular discovery for organic solid-state lasers

Strieth-Kalthoff et al. (15) recently demonstrated the delocalized, asynchronous, closed-loop discovery of organic laser gain materials, in which BO steered robotic synthesis and spectroscopic characterization of fragment-assembled emitter candidates across five laboratories. Mirroring this discovery problem, ÓPTIMA was tasked with running a closed-loop digital campaign over the same published fragment catalogues, exploring the fragment chemistry through an inexpensive digital proxy workflow and assuming the orchestration, diagnosis, and analysis roles that otherwise require bespoke infrastructure and human planners. Rather than treating this as a static screening problem, ÓPTIMA formulated it as a multi-objective BO campaign coupled to molecular assembly, conformer generation, and quantum-chemical evaluation.

The molecular design space followed an A–B–C–B–A composition rule, where the decision variables corresponded to cap, bridge, and core fragments. Before starting the optimization, ÓPTIMA inferred the assembly rule from the fragment catalogues, including the reactive halogen sites used to join fragments. The reconstructed products were validated against a user-provided reference subset of 1129 molecules, all reproducing the reference connectivity; this file served only as a correctness check and contributed neither candidates nor objective values. Previously, constructing this design space required a domain expert to identify the assembly logic, write a dedicated Python script, and manually inspect the resulting structures. ÓPTIMA represented the resulting search space using three categorical parameters, `cap_id`, `bridge_id`, and `core_id`, supplemented by a concise set of RDKit-derived molecular descriptors (47) characterizing each fragment’s size,

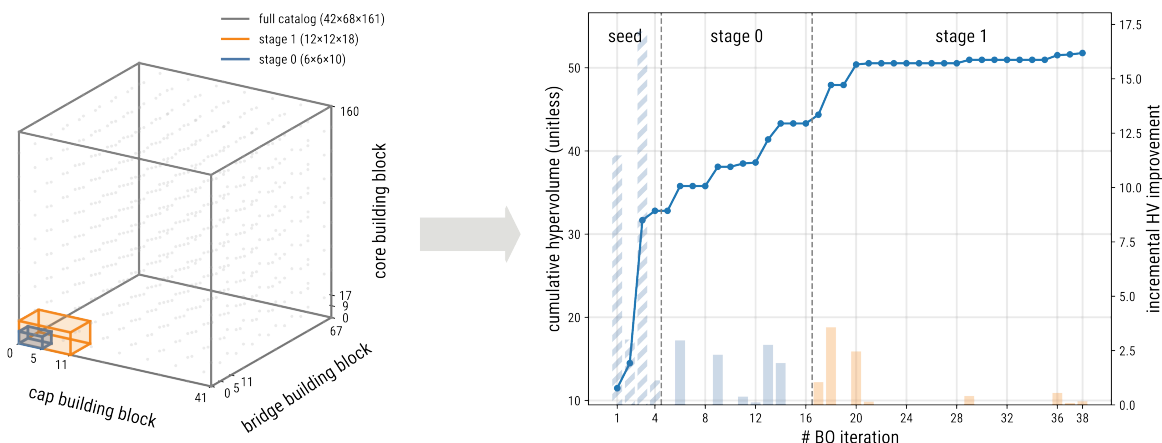


Figure 2 Autonomous digital discovery of organic solid-state laser (OSL) candidates with LA AGENTE ÓPTIMA. **(a)** Schematic representation of the explored search spaces relative to the full combinatorial design space spanned by the cap, bridge, and core building blocks. **(b)** Closed-loop BO progress across the seed set, stage 0, and stage 1, shown as the cumulative best-so-far hypervolume together with the incremental hypervolume improvement at each iteration. Both panels were generated by ÓPTIMA and required only minimal visual modification.

composition, structure, and physicochemical properties. These descriptors informed both the surrogate model, enabling it to exploit fragment similarity (9), and the distance metric used for frontier-aware search-space expansion.

Of the 462,672 theoretically accessible A–B–C–B–A combinations (42 caps, 68 bridges, and 162 cores in the published catalogues), the campaign began in a deliberately restricted and inexpensive search space of 360 candidates (Stage 0, Fig. 2a), selected by a small-fragment-first strategy. The four initial observations were selected using fragment descriptors to provide balanced and representative coverage across all three fragment types. The campaign optimized three objectives jointly: maximizing oscillator strength, minimizing the colour error relative to a target visible excitation energy, and minimizing a structural ambiguity penalty. In this initial stage, the twelve BO-selected candidates contributed more cumulative hypervolume (3.76) than the four seeds (2.78), indicating that the optimizer added value beyond the initial design (Fig. 2b).

After 16 successful observations, ÓPTIMA analyzed the Pareto front and expanded the search space through a deterministic, frontier-aware rule: the fragments occurring in Pareto-optimal candidates served as per-slot anchors, and additional caps, bridges, and cores were ranked by their z-scored descriptor distance to these anchors. Stage 1 expanded the search space from 360 to 2592 candidates while retaining the original space, yet still covered only about 0.6% of all theoretically accessible combinations. The campaign then continued in the expanded Stage 1 space, with BO building on the initial 16 observations. The expanded campaign ultimately produced 38 successful observations in total and revealed recurring fragment motifs associated with favourable brightness–colour–robustness trade-offs. Asked to interpret the most promising candidates, ÓPTIMA identified several recurring chemical motifs. These included strong dimethylamino–aryl donor caps, compact vinylene or thiophene π -bridges, and cores that tuned acceptor strength. It also distinguished chemically promising Pareto points from formally non-dominated but impractical extremes, such as a rigid but essentially dark candidate with near-zero oscillator strength. It also cautioned that these interpretations rest on an inexpensive digital proxy rather than converged photophysics.

The campaign also tested the resilience of the agentic workflow: When a configuration mismatch disrupted Stage 1, ÓPTIMA autonomously diagnosed and patched its evaluator, validated the fix, and resumed production without human intervention. Throughout, ÓPTIMA preserved campaign integrity by excluding failed evaluations and reconciling the optimizer state after timeouts. Automatically generated diagnostics tracked optimization progress, search-space expansion, and Pareto trade-offs, providing an auditable record of the search.

Overall, the campaign demonstrates that ÓPTIMA can autonomously run and recover an iterative molecular discovery workflow, while leaving strategic control over evaluation budgets and search-space expansion to the

operator. Over 47 top-level turns spanning about 3 d 22 h calendar time (16.0 h of agent-run wall-clock), this conversation made 397 LLM calls — 137 in the main agent plus 260 subagent calls (240 across 4 gpt-5.4 subagents + 20 gpt-4.1 one-shots) — consuming 35.4M input / 229k output tokens at a total cost of \$24.40.

3.1.2 Finite-space discovery across chemistry and materials

Four additional campaigns test ÓPTIMA across finite candidate spaces spanning ligand electronics, excited-state screening, transition-metal complexes, and porous materials. Their common quantitative record is collected in [Tab. 1](#); full campaign details are provided in the corresponding SI sections. The improvement column compares the seed baseline with the final campaign state.

Table 1 Comparison of the four computational discovery campaigns. Activity reports main-orchestrator tool calls followed by all recorded LLM calls, including specialist and evaluator-routing calls. Tokens are input/output totals, and cost is the total recorded LLM cost. Evaluation counts distinguish attempted or historical observations from successful observations where relevant.

Showcase	Search space	Evaluation budget	Recorded BO improvement	Tool / LLM calls	Tokens in / out	Cost (USD)
Phosphine ligands	364 ligands	8 + 40 = 48; 48 successful	Hypervolume 0.791 → 1.038; Pareto set 3 → 17	33 / 128	6.18M / 50.4k	\$7.08
Inverted-gap emitters	1512 molecules	44 attempted; 39 successful	Best gap 0.227 → 0.223 eV	34 / 279	13.1M / 76.5k	\$13.01
Co bisphosphines	144 ligands	4 + 10 = 14; 6 feasible	Feasible-only hypervolume 0.572 → 1.000	94 / 557	56.1M / 210k	\$53.29
Xe/Kr MOFs	2800 nominal (420 valid); refined to 109 valid	30 + 50 = 80; 65 successful	Desirability 0.487 → 0.502; Pareto set 7 → 12	25 / 162	11.6M / 61.3k	\$11.44

Laplaza et al. (48) optimized phosphine and carbene ligands for Ni-catalyzed aryl-ether cleavage with a genetic algorithm and a molecular-volcano fitness. We instead exposed the underlying electronic and structural trade-offs by asking ÓPTIMA to tune monodentate phosphines $P(R^1)(R^2)(R^3)$ as a ligand-level proxy for Ni catalysis. The campaign did not yield a single best ligand because its electronic objectives conflicted. ÓPTIMA recognized that BO was instead finding smaller ligands with similar electronic properties, identified an uninformative steric objective, and recommended stopping as further evaluations produced diminishing returns (SI, [Sec. C.1](#)).

Pollice et al. (49) screened heptazine- and cyclazine-derived chromophores for inverted singlet–triplet gaps and reported excitation energies for the resulting library. We used a size-filtered subset of 1512 molecules as a BO benchmark with known published results. The published values were hidden from the agent, allowing us to assess both its candidate selection and the accuracy of its inexpensive evaluator. Before launching the campaign, ÓPTIMA timed a trial evaluation and switched to a less expensive method and smaller budget when server data showed that its runtime estimate was too optimistic. The small improvement in the best gap (0.227 → 0.223 eV) reflects a fortunate random initialization: the first of five randomly sampled seed molecules was already near-optimal. A post-campaign comparison with the published references showed that the inexpensive time-dependent density functional theory (TD-DFT) evaluator ranked the evaluated molecules well but did not reproduce their inverted gaps.

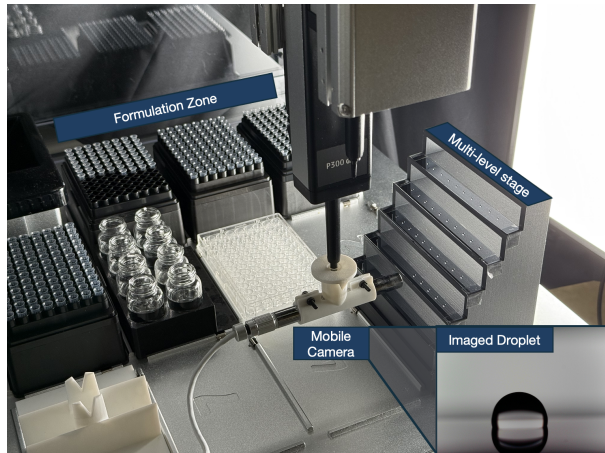
Hood et al. (50) reported cationic Co(II) bisphosphine hydroformylation catalysts whose activity depends sharply on the linker and phosphorus substituents. We recast the ligand space as a finite multi-objective campaign in which ÓPTIMA requested every three-dimensional $[Co(acac)(P_2)]^+$ structure from EL AGENTE ESTRUCTURAL over the A2A interface, left optimizer state to BO-MCP, and evaluated converged geometries through GRÁFICO. ÓPTIMA therefore coordinated candidate selection, structure generation, and electronic-structure calculations across separate services. Faced with eight failed evaluations, ÓPTIMA distinguished computational failures from evidence about ligand performance. It traced most failures to unconverged geometry optimizations and concluded that improving the generated starting structures and relaxation protocol, rather than extending the BO campaign, was the appropriate next step (SI, [Sec. C.3](#)).

Screening metal-organic frameworks (MOFs) for Xe/Kr separation is conventionally approached by enumerating

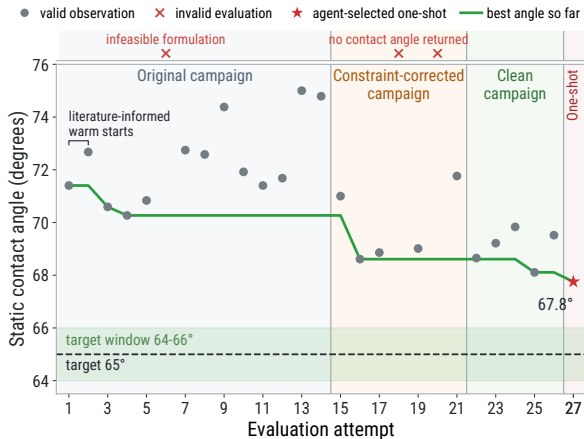
and scoring large hypothetical framework libraries one candidate at a time, commonly using pore-geometric filters such as the pore-limiting diameter as a first pass (51), or by inverse design against a user-specified selectivity target (52). Following the design goal of Lim et al. (52), but with their selectivity objective replaced by a geometric proxy that requires no adsorption simulation, we posed the same design question as a small-budget BO campaign to test the workflow. ÓPTIMA assembled candidates with PORMAKE (53) and analyzed their pore geometry with Zeo++ (54). Here, the salient capability emerged only after the first campaign: the agent recognized that independent topology, node, and edge variables described mostly unconstructible combinations, declined to spend more budget on the same representation, and rebuilt the problem as a finite set of connectivity-compatible triples while carrying prior successes forward as evidence. The fully feasible follow-up improved only modestly, but ÓPTIMA identified the search-space representation, rather than the optimizer, as the bottleneck and replaced it while retaining all prior observations (SI, Sec. C.4).

3.2 Closed-loop formulation optimization with RAISE

To evaluate LA AGENTE ÓPTIMA in a physical closed loop with RAISE (45), we asked ÓPTIMA to identify an ethanol and sodium dodecyl sulfate (SDS) mixture with a target contact angle of 65° , stopping when a measurement fell within $\pm 1^\circ$ and proceeding in small, explicitly approved increments. Before campaign construction, ÓPTIMA selected two literature-informed warm-start formulations. ÓPTIMA then executed a specialist-authored program that connected the RAISE evaluator to a BayBE-backed BO-MCP campaign.



(A) RAISE SDL platform.



(B) Contact-angle optimization trajectory.

Figure 3 Closed-loop formulation optimization with RAISE. (A) The RAISE SDL platform was used to prepare formulations and measure their static contact angles (reproduced from Ref. (45) under CC BY-NC 4.0 licence). (B) Contact-angle measurements and the best angle obtained so far across the original, constraint-corrected (ethanol upper bound reduced from 60 to 50 v/v%), and clean campaigns, followed by the agent-selected one-shot experiment. Grey circles denote valid observations, red crosses mark one infeasible formulation and two evaluations for which RAISE returned no contact-angle value, and the red star marks the final one-shot result. The upper event strip places the invalid evaluations in sequence without assigning them contact-angle values; background shading identifies the campaign stages, and the green band denotes the $65 \pm 1^\circ$ target window. The panel shows the 27 evaluations in the reported campaigns and the final ÓPTIMA one-shot.

The campaign exposed a consequential distinction when handling infeasible conditions and failed measurements. The initial campaign script recorded every failed evaluation as a fallback observation of 180° . This convention conflated a permanently infeasible formulation with two feasible formulations with missing measurements as equivalent outcomes. The infeasible suggestion was 60 v/v% ethanol, which exceeded the maximum ethanol concentration achievable under this campaign’s well-volume and stock-concentration constraints, approximately 50 v/v%. Following operator approval, ÓPTIMA reduced the upper bound to 50 v/v% and transferred the 13 valid observations from the original campaign into a new campaign with corrected constraints.

The 180° penalty assigned to the infeasible formulation led all eight subsequent suggestions to remain below 20 v/v% ethanol. The latter two measurement failures occurred at feasible formulations containing 30–36 v/v%

ethanol, near the best observations obtained at that stage. Recording each failure as 180° falsely associated those candidates with highly unfavourable contact angles and biased the surrogate against this promising region. The experimentalist traced the measurement errors to a slight shift in the backlight position relative to the droplet, which disrupted contour detection in the image-processing pipeline. This cause was not reported to ÓPTIMA during the campaign; therefore, the same fallback penalty was applied to the measurement.

On reviewing the results, ÓPTIMA identified the fallback encoding as the source of the problem, where a failed measurement should be treated as missing data instead of an unfavourable outcome. It proposed a new campaign seeded with all 18 unique valid observations from the two preceding campaigns, so no physical experiment was repeated. The revised procedure retried the same formulation up to twice and, if neither retry produced a contact-angle value, recorded the failure locally without submitting an objective value to BO-MCP. Following operator approval, all five BO iterations were completed without any measurement failures or retries. This highlights a broader requirement for agentic SDLs, in which sensor data and platform state are readily accessible to the agent, enabling it to determine whether an unexpected measurement arose from the formulation or from changes in its physical environment.

Across the 27 plotted evaluations, the best result moved from 71.4° for the warm start to 68.1° in the clean campaign. Five additional physical measurements were performed for implementation validation and preliminary checking and were not included in the optimization trajectory. When the operator allocated one final experiment because of the remaining time constraint, ÓPTIMA selected the maximum feasible ethanol concentration with SDS fixed at its best observed level, yielding 67.8°. This result demonstrates how one-shot agent suggestions can complement BO by translating the evidence accumulated during a campaign into an effective final experiment under changing operational constraints. The campaign did not reach the $\pm 1^\circ$ target window. From this boundary result, ÓPTIMA inferred that 65° was unlikely to be attainable with the two available reagents and recommended replicate measurements and a change of formulation system before further optimization. A complete report of the session, including the optimization configuration and per-campaign data, is provided in SI, [Sec. C.5](#).

Over 11 top-level turns spanning 2 h 38 min of calendar time, the campaign used 148 LLM calls: 34 in the main agent (gpt-5.5) and 114 across five subagent (gpt-5.4) sessions. These calls consumed 7.75M input and 97k output tokens at a total cost of \$15.45.

3.3 Multi-objective flow photochemistry with RoboChem-Flex

We next evaluated LA AGENTE ÓPTIMA in continuous-flow synthetic chemistry using RoboChem-Flex. As a test case, we revisited the platform’s published photocatalytic radical trifluoromethylation benchmark, with trifluoroacetic anhydride as the CF₃ source and a pyridine *N*-oxide as the activator. Whereas the original workflow delegated experimental design to a dedicated BoTorch-based optimization engine ([46](#)), here ÓPTIMA assumed that role. ÓPTIMA executed the live campaign, transferred the experimental history between objectives, and increased the best measured yield from 30.0% among the informed seed experiments to 58.8%.

3.3.1 Campaign design under platform constraints

The operator supplied CSV files defining the available chemistry, experimental search space, stock concentrations, and analytical constants. From these inputs, ÓPTIMA constructed a seven-parameter search space spanning photocatalyst and oxidant identities and loadings, trifluoroacetic anhydride (TFAA) loading, residence time, and light intensity (SI, [Sec. C.6.3](#)).

Interestingly, both the operator’s process table and the platform’s live capability descriptor reported a generic light-intensity range of 0–100%. However, *RoBridge*’s capability descriptors of the mounted UFlow ([55](#)) reactor light array supported only {0, 25, 50, 75, 100}%. ÓPTIMA therefore adopted these discrete settings, preserving the hardware’s capabilities. More generally, this demonstrates the ability of the agent to bridge the gap between user intent and laboratory reality, automatically mapping abstract experimental requests to the nearest physically realizable application.

ÓPTIMA initialized the optimization by deliberately selecting six diverse experiments covering all categorical variables, including five photocatalysts, both oxidants, and both bounds of each stoichiometric variable,

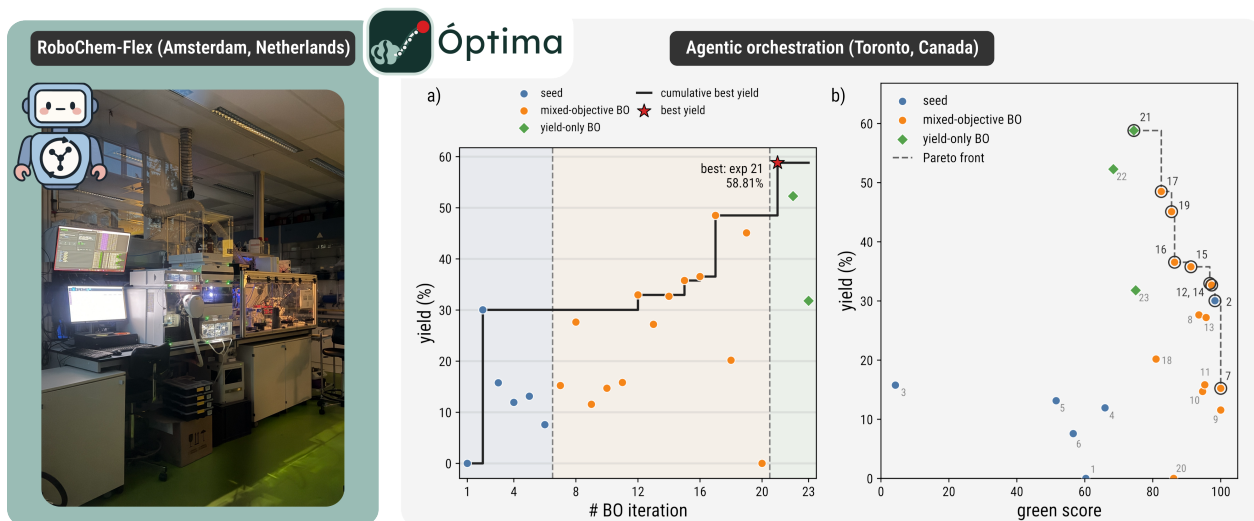


Figure 4 Agent-controlled photocatalytic trifluoromethylation campaign on RoboChem-Flex. **(a)** Measured yield across the 23 valid experiments, together with the cumulative best-so-far yield. Background shading indicates the six informed seed experiments, the 14 multi-objective BO experiments, and the three yield-only BO experiments initialized from the migrated campaign history. The highest measured yield, 58.8% in experiment 21, is marked by a star. **(b)** The same observations in the two-dimensional objective space defined by yield and the agent-defined green score. The dashed staircase denotes the empirical Pareto front obtained by maximizing both objectives, and labels indicate experiment numbers. Open outlined markers identify non-dominated points on the Pareto front. Three additional runs affected by analytical failures are excluded from both panels. Both panels were generated by ÓPTIMA from its exported campaign data.

while sampling multiple light intensities and residence-time regimes. ÓPTIMA autonomously translated each candidate’s named search-space parameters into a RoboChem-Flex experimental request, mapping reagent identities to platform identifiers, converting residence time to the required units, and appending the fixed reaction and analytical settings. Before any hardware was addressed, ÓPTIMA emitted the experimental request as a hypothetical JSON document for operator inspection, making its interpretation of the proposed conditions auditable in advance.

3.3.2 Multi-objective optimization and objective revision

The operator requested optimizing two objectives in the first campaign: rewarding greener conditions alongside yield. ÓPTIMA defined an analytic green score from the same parameter vector. The score combined normalized penalties for catalyst loading, TFAA and oxidant equivalents, and photonic dose based on light intensity and residence time; its full definition and normalization are provided in the SI (Sec. C.6.3). Notably, although the agent was not prompted with any explicit formulation of the “green score”, the resulting metric closely matches the normalization strategy and weighting that operators have manually designed for unrelated previous experimental campaigns on RoboChem, indicating that the agent independently converged on a comparable and chemically intuitive objective (56).

With over 20 valid observations, comprising six informed seeds and 14 BO-selected experiments, the campaign explicitly mapped the trade-off between yield and this agent-defined resource-efficiency metric. The observed conditions ranged from maximally frugal but poorly productive settings, yielding 15.2% at a green score of 100, to the most productive point of this phase, yielding 48.5% at a green score of 82.5 (Fig. 4b). When the operator subsequently narrowed the goal to yield alone, ÓPTIMA transferred all valid observations from the multi-objective campaign into a new single-objective campaign. Continuing from that history, the best measured yield rose to 58.8% in experiment 21 (R0067), roughly double the best seed observation (30.0%) and above the best point of the mixed-objective phase (Fig. 4a; SI, Tab. 19). When evaluated against both objectives, the new condition added a high-yield endpoint to the Pareto front: yield increased from 48.5% to 58.8%, while the green score decreased from 82.5 to 74.4 (Fig. 4b). Starting from the deliberately diverse seed

experiments, BO converged on chemistry closely related to the published RoboChem-Flex optimum, using the same tris(bipyridine)ruthenium photocatalyst family and pyridine *N*-oxide activator while identifying distinct operating conditions (46).

3.3.3 Long-horizon execution and analytical diagnosis

Sustaining this loop required a horizon far longer than a conversational turn, with measurements arriving roughly hourly and campaigns running unattended overnight. OPTIMA ran the campaign as a monitored background task, submitting experiments one at a time and checking their status at fixed intervals. It reported only state changes, alerts, and periodic heartbeats to the conversation while saving the complete instrument record to disk. Only runs with a passing analysis and finite yield were added to the surrogate model. When a run failed before analysis, OPTIMA either retried it or paused the campaign for later resumption. Operator involvement was limited to high-level oversight and necessary physical interventions.

Notably, the first seed experiment (Fig. 4a, experiment 1) returned a yield of exactly zero with the analysis marked as passing, an outcome that can mean either that no product formed or that the analysis failed to detect a sufficiently well-defined peak in the expected spectral region. Prompted by the operator to investigate, OPTIMA located the platform’s raw-result endpoint and mirrored the run’s complete analytical record into the workspace, including the free induction decay and the processed spectrum, then analyzed the spectrum itself: It reconstructed the chemical-shift axis from the file header, estimated the noise floor from the median absolute deviation, and confirmed from the acquisition parameters that the intended 32-scan ^{19}F protocol had in fact run. OPTIMA observed a dominant ^{19}F signal at -76.5 ppm and proposed a trifluoroacetyl-derived species as a possible source. The expected product region (-58 ± 3 ppm) contained only baseline structure rather than a true peak, supporting the reported zero yield as a genuine absence of detectable product. This addresses a recurring operational bottleneck in autonomous experimentation: determining whether an anomalous analytical result reflects failed chemistry or a failed measurement. Because the campaign continued in the background, OPTIMA could perform this assessment in dialogue with the operator without pausing execution. The agent also ran a lightweight campaign supervisor that flagged zero-yield and no-peak analyses and was configured to halt execution after five consecutive such outcomes. Such a streak would suggest an instrument malfunction rather than a genuinely unproductive region of the search space, a distinction that required active human oversight and domain expertise in the pre-LLM era (18).

3.3.4 Resource use relative to human-directed optimization

The campaign covered 5 d 8 h of calendar time and 69 top-level operator turns. The platform was occupied for 34 h of that span and OPTIMA itself accounted for 2 h 16 min of LLM and tool execution; the remaining 94 h were idle, dominated by nights and by a 1 d 21 h stop for hardware preparation, without which the campaign spanned 3 d 11 h. Across the campaign, it made 1108 LLM calls (gpt-5.5), 498 in the main agent plus 610 across 14 subagent sessions, consuming 114.5M input and 328k output tokens at a total cost of \$109.55. Even with the 1M context window of gpt-5.5, this campaign was only made possible by the subagent architecture, which separates the context.

Benchmarked against a human-directed BO campaign on the same transformation and platform, the agent was stopped after 23 experiments, compared with 50 for the human-directed campaign, consuming less than half the starting material. Despite the additional inference cost, the agent campaign maintained a 13% overall cost advantage, with savings primarily driven by the reduced number of experiments. The agent reached 58.8% yield, compared with 70.9% for the full human-directed campaign and 63.9% at matched consumable budget. Importantly, both agent-identified optima were more mass-efficient, using substantially less reagent and achieving higher reaction mass efficiencies (RMEs); this difference was even more pronounced across the complete campaigns, where the agent returned only two zero-yield experiments compared with 27. Full cost and mass accounting is provided in the SI, Sec. C.7, while the complete operator-agent exchange, control layer, and experimental conditions are provided in Sec. C.6.

4 Discussion

Across the benchmarks and case studies, candidate selection followed a flexible division of labour between BO and the agent. The architecture comparison demonstrated programmatic BO for routine search, whereas the RAISE experiment showed that the agent could propose an experiment directly when the accumulated evidence and remaining budget called for scientific judgment. LA AGENTE ÓPTIMA can therefore move between classical BO search and LLM-based selection within the same framework.

Importantly, this adaptability is not limited to choosing the next experiment. The scientific problem itself can be revised as a campaign unfolds. Through natural-language interaction with ÓPTIMA, the researcher can redefine the scientific question, where ÓPTIMA translates that decision into concrete changes to the running campaign. The researcher would otherwise need to manually stop the current execution, preserve its state, and reconstruct the campaign according to the revised specification. In the MOF study, this involved replacing a search-space representation that produced many invalid structures; in RoboChem-Flex, it involved changing the objective while retaining the existing experimental history. By preserving the campaign state through such changes, ÓPTIMA allows the researcher to focus on the scientific question rather than campaign execution.

The value of this adaptability, however, depends on the evidence available to support those decisions. The search for inverted-gap chromophores inherited the bias of its evaluator; a change in imaging conditions appeared to RAISE only as a missing value, and the RoboChem-Flex zero-yield result became interpretable only after the raw spectrum was inspected. In each case, the numerical result alone provided an incomplete account of what had occurred. While provenance preserves how a result was produced, access to sensor data, such as camera frames, analytical records, and platform state, enables the agent to diagnose failures and base its advice on underlying evidence rather than solely on the numerical observations typically exposed to the BO algorithm. As agent access expands to laboratory sensing and control, it should remain bounded by deterministic safety interlocks and explicit human authorization (57). Structured safety reasoning and accountability mechanisms provide complementary safeguards for agentic SDLs (58; 59).

Providing such context is only part of the challenge. It must also remain usable as the campaign evolves over time. Programmatic access to BO state and intermediate results enabled ÓPTIMA to repair and resume campaigns when research plans or operating conditions changed. Persistent workspaces and executable programs maintained this continuity across model invocations and evaluators by encoding repeated tool use in code and loading intermediate results into the agent context only when needed.² For long-running execution, the background monitor further reduced the need for continuous model involvement by filtering incremental process output into compact progress events for the agent and its frontend. ÓPTIMA could therefore attend to the campaign only when a decision or user notification was required, while remaining available to discuss other scientific questions with the researcher as experiments continued in the background. This event-driven supervision preserves visibility into campaign progress as autonomous experiments extend over longer time horizons.³ These mechanisms allow unforeseen events to be accommodated without discarding valid prior work, as is routinely required in research practice.

Supporting this level of adaptive orchestration introduces an additional item into the operating budget: LLM inference. In the RoboChem-Flex study, inference cost more than the chemistry it directed, yet the complete agent campaign remained 13% cheaper than the human-directed campaign. ÓPTIMA used 46% of the experiments and 39% of the starting material while identifying more mass-efficient conditions, although with lower final yield. This highlights that optimization performance extends beyond the highest attainable yield: experimental efficiency, material consumption, and cost are equally relevant measures of a campaign’s value (60). Further reductions in inference cost would strengthen this cost advantage. Reducing equipment occupancy, in turn, will require optimization strategies that account for how long an experiment ties up the instrument, not just how informative it is expected to be, for example, preferring a short, moderately informative experiment over a much longer one that offers only a marginally better expected outcome.

²Related patterns have been described for general-purpose agents: Anthropic uses [programmatic tool calls](#) to keep intermediate results outside the model context and [structured artifacts](#) to maintain continuity across context windows, and OpenAI uses a [filesystem-backed computer environment](#) for persistent runtime state.

³OpenAI’s [discussion of long-horizon models](#) similarly emphasizes trajectory-level monitoring, user visibility, and mechanisms to intervene, pause, or roll back execution.

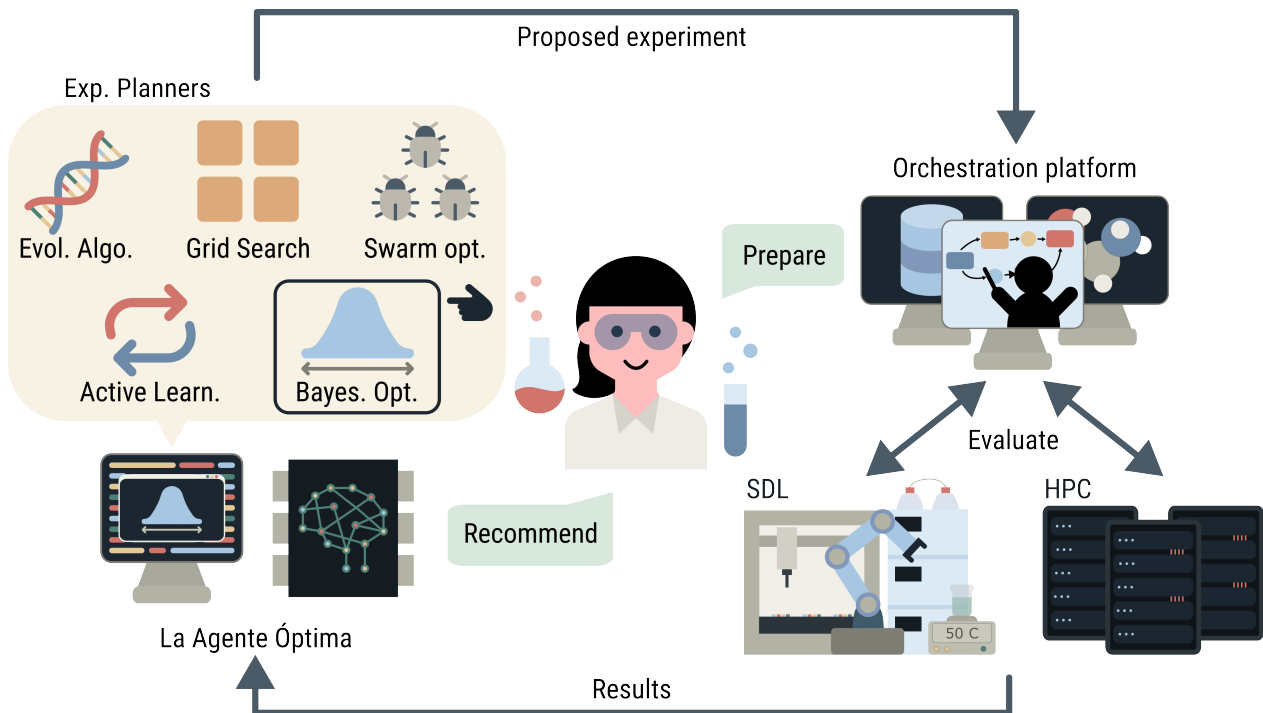


Figure 5 Conceptual vision of ÓPTIMA as a general optimization layer for autonomous experimental campaigns. Based on the problem definition and available resources, ÓPTIMA selects and configures an appropriate experimental design strategy and operates in a closed loop with the scientist, laboratory infrastructure (SDL), and computational resources (HPC), adapting its decisions as new experimental data become available.

Long-horizon orchestration also places new demands on agent context management. Simply retaining an ever-growing interaction history, or compressing it into an undifferentiated summary, may be insufficient when decisions depend on experimental observations and constraints established much earlier in a campaign. Instead, campaign context could be separated into structured classes, for example, persistent scientific objectives, constraints, experimental evidence, and transient execution details. Long-term information could then be retrieved in a structured way, while short-lived information can be summarized or branched off.

These studies show how an agent can maintain alignment between an evolving scientific question and the executable state of a running campaign. Fig. 5 extends this principle to more general experimental orchestration, in which the user remains central throughout. Ultimately, a researcher would specify only the campaign’s goal, available resources, levers to pull, constraints, and success criteria, stating all of this in natural language rather than manually configuring an experimental design. ÓPTIMA would then select the experimental design method best suited to the problem, with BO among several options. It would define the search space, select suitable features and representations, incorporate prior data, and configure the optimizer. ÓPTIMA would also be able to autonomously construct physics-informed parameter representations, using properties computed with the PySCF execution graph (61) or lower-cost proxies obtained from semiempirical quantum-mechanical methods (62; 63). This would allow the surrogate model to exploit physically meaningful similarities between categorical candidates rather than treating them as unrelated labels. As the campaign runs, ÓPTIMA would analyze the resulting evidence and decide whether to continue, reformulate, or stop, adapting the strategy as the scientific question evolves. A workflow orchestrator would translate these decisions into instructions for how, where, and when each experiment is executed. Persistent campaign state and event-driven monitoring would record every decision made by ÓPTIMA, including the chosen design method and the rationale behind it. This would allow researchers to review its decisions and trace how scientific choices led to actions across computational and laboratory systems.⁴ When multiple campaigns run concurrently, their agents could query one another for structured information about objectives, observations, decisions, and resource requirements.

⁴A related interface-based separation of agent reasoning, durable session state, and execution environments is described in Anthropic’s Managed Agents architecture.

This would allow each agent to retrieve relevant evidence on demand without exchanging or merging the campaigns’ full contexts. With shared abstractions for workflows, data, and provenance (64), agentic SDLs could extend beyond individual laboratories to form distributed research networks that coordinate scientific discovery across institutional and geographic boundaries (18; 15; 65).

Data availability

The raw data underlying all showcases and benchmark evaluations reported in this work are available at <https://github.com/the-matter-lab/La-Agente-Optima-artifacts>. The Supporting Information covers the BO-MCP (accessible at <https://github.com/AccelerationConsortium/bo-mcp>) system architecture and implementation, the framework-comparison benchmark, the computational and experimental showcases, and the detailed RoboChem-Flex campaign record and resource accounting.

Acknowledgements

We gratefully acknowledge the longstanding contributions of the Matter Lab’s current and past group members (<https://matter.toronto.edu>), in particular, the El Agente subgroup. M.M. was partially supported through a collaborative partnership with Merck KGaA, Darmstadt, Germany. J.B. acknowledges funding from the Eric and Wendy Schmidt AI in Science Postdoctoral Fellowship Program, a program by Schmidt Futures. E.S. and T.N. acknowledge the generous funding provided by the European Innovation Council through the reaCtor project (grant No. 101099405). E.S., T.N., and S.P. acknowledge funding from the European Union through an ERC Proof of Concept Grant (RoboChem, grant No. 101246252). T.N. gratefully acknowledges funding from the Dutch Research Council (NWO) under the Talent Programme VICI (SynthBot, grant No. 20453). A.A.-G. thanks Anders G. Frøseth for his generous support. A.A.-G. also acknowledges the generous support of Natural Resources Canada and the Canada 150 Research Chairs program. This research is part of the University of Toronto’s Acceleration Consortium, which receives funding from the CFREF-2022-00042 Canada First Research Excellence Fund, and was supported by the Defense Advanced Research Projects Agency (DARPA) under Agreement No. HR0011262E022 and the AI2050 program of Schmidt Sciences.

References

- [1] Andrew F. Zahrt, Jeremy J. Henle, Brennan T. Rose, Yang Wang, William T. Darrow, and Scott E. Denmark. Prediction of higher-selectivity catalysts by computer-driven workflow and machine learning. *Science*, 363(6424): eaau5631, January 2019. doi:10.1126/science.aau5631. <https://www.science.org/doi/10.1126/science.aau5631>.
- [2] Konstantinos D. Vogiatzis, Mikhail V. Polynski, Justin K. Kirkland, Jacob Townsend, Ali Hashemi, Chong Liu, and Evgeny A. Pidko. Computational Approach to Molecular Catalysis by 3d Transition Metals: Challenges and Opportunities. *Chemical Reviews*, 119(4):2453–2523, October 2018. ISSN 0009-2665. doi:10.1021/acs.chemrev.8b00361. <https://doi.org/10.1021/acs.chemrev.8b00361>.
- [3] Benjamin Sanchez-Lengeling and Alán Aspuru-Guzik. Inverse molecular design using machine learning: Generative models for matter engineering. *Science*, 361(6400):360–365, July 2018. doi:10.1126/science.aat2663. <https://www.science.org/doi/10.1126/science.aat2663>.
- [4] Gary Tom, Stefan P. Schmid, Sterling G. Baird, Yang Cao, Kourosh Darvish, Han Hao, Stanley Lo, Sergio Pablo-García, Ella M. Rajaonson, Marta Skreta, Naruki Yoshikawa, Samantha Corapi, Gun Deniz Akkoc, Felix Strieth-Kalthoff, Martin Seifrid, and Alán Aspuru-Guzik. Self-Driving Laboratories for Chemistry and Materials Science. *Chemical Reviews*, 124(16):9633–9732, August 2024. ISSN 0009-2665. doi:10.1021/acs.chemrev.4c00055. <https://doi.org/10.1021/acs.chemrev.4c00055>.
- [5] Jonas Mockus. *Bayesian Approach to Global Optimization: Theory and Applications*, volume 37 of *Mathematics and Its Applications*. Springer Netherlands, Dordrecht, 1989. ISBN 978-94-010-6898-7 978-94-009-0909-0. doi:10.1007/978-94-009-0909-0.
- [6] Maximilian Balandat, Brian Karrer, Daniel R. Jiang, Samuel Daulton, Benjamin Letham, Andrew Gordon Wilson, and Eytan Bakshy. BoTorch: A Framework for Efficient Monte-Carlo Bayesian Optimization, 2020. <http://arxiv.org/abs/1910.06403>. arXiv:1910.06403 [cs].
- [7] Roman Garnett. *Bayesian optimization*. Cambridge University Press, Cambridge, United Kingdom, 2023.
- [8] Johannes P. Dürholt, Thomas S. Asche, Johanna Kleinekorte, Gabriel Mancino-Ball, Benjamin Schiller, Simon Sung, Julian Keupp, Aaron Osburg, Toby Boyne, Ruth Misener, Rosona Eldred, Wagner Steuer Costa, Chrysoula Kappatou, Robert M. Lee, Dominik Linzner, David Walz, Niklas Wulkow, and Behrang Shafei. BoFire: Bayesian Optimization Framework Intended for Real Experiments, August 2024. <http://arxiv.org/abs/2408.05040>. arXiv:2408.05040 [cs].
- [9] Martin Fitzner, Adrian Šošić, Alexander V. Hopp, Marcel Müller, Rim Rihana, Karin Hrovatin, Fabian Liebig, Mathias Winkel, Wolfgang Halter, and Jan Gerit Brandenburg. BayBE: A Bayesian Back End for experimental planning in the low-to-no-data regime. *Digital Discovery*, 4(8):1991–2000, August 2025. ISSN 2635-098X. doi:10.1039/D5DD00050E.
- [10] Riley J. Hickman, Malcolm Sim, Sergio Pablo-García, Gary Tom, Ivan Woolhouse, Han Hao, Zeqing Bao, Paucic Bannigan, Christine Allen, Matteo Aldeghi, and Alán Aspuru-Guzik. Atlas: A brain for self-driving laboratories. *Digital Discovery*, 4(4):1006–1029, April 2025. ISSN 2635-098X. doi:10.1039/d4dd00115j.
- [11] Stefan Desimpel, Matthieu Dorbec, Kevin M. Van Geem, and Christian V. Stevens. Bayesian optimization for chemical reactions. *Chemical Society Reviews*, 55(5):2731–2775, March 2026. ISSN 0306-0012. doi:10.1039/d5cs00962f. <https://doi.org/10.1039/d5cs00962f>.
- [12] Peter I. Frazier. A Tutorial on Bayesian Optimization, July 2018. <http://arxiv.org/abs/1807.02811>. arXiv:1807.02811 [stat.ML].
- [13] Jose Antonio Garrido Torres, Sii Hong Lau, Pranay Anchuri, Jason M. Stevens, Jose E. Tabora, Jun Li, Alina Borovika, Ryan P. Adams, and Abigail G. Doyle. A Multi-Objective Active Learning Platform and Web App for Reaction Optimization. *Journal of the American Chemical Society*, 144(43):19999–20007, November 2022. ISSN 0002-7863. doi:10.1021/jacs.2c08592. <https://doi.org/10.1021/jacs.2c08592>.
- [14] Miles Olson, Elizabeth Santorella, Louis C. Tiao, Sait Cakmak, Mia Garrard, Samuel Daulton, Zhiyuan Jerry Lin, Sebastian Ament, Bernard Beckerman, Eric Onofrey, Paschal Igusti, Cristian Lara, Benjamin Letham, Cesar Cardoso, Shiyun Sunny Shen, Andy Chenyuan Lin, Matthew Grange, Elena Kashtelyan, David Eriksson, Maximilian Balandat, and Eytan Bakshy. Ax: A Platform for Adaptive Experimentation. In *Proceedings of the Fourth International Conference on Automated Machine Learning*, pages 21/1–25. PMLR, November 2025.
- [15] Felix Strieth-Kalthoff, Han Hao, Vandana Rathore, Joshua Derasp, Théophile Gaudin, Nicholas H. Angello, Martin Seifrid, Ekaterina Trushina, Mason Guy, Junliang Liu, Xun Tang, Masashi Mamada, Wesley Wang,

- Tuul Tsagaantsooj, Cyrille Lavigne, Robert Pollice, Tony C. Wu, Kazuhiro Hotta, Leticia Bodo, Shangyu Li, Mohammad Haddadnia, Agnieszka Wołos, Rafał Roszak, Cher Tian Ser, Carlota Bozal-Ginesta, Riley J. Hickman, Jenya Vestfrid, Andrés Aguilar-Granda, Elena L. Klimareva, Ralph C. Sigerson, Wenduan Hou, Daniel Gahler, Slawomir Lach, Adrian Warzybok, Oleg Borodin, Simon Rohrbach, Benjamin Sanchez-Lengeling, Chihaya Adachi, Bartosz A. Grzybowski, Leroy Cronin, Jason E. Hein, Martin D. Burke, and Alán Aspuru-Guzik. Delocalized, asynchronous, closed-loop discovery of organic laser emitters. *Science*, 384(6697):eadk9227, May 2024. doi:10.1126/science.adk9227. <https://www.science.org/doi/full/10.1126/science.adk9227>.
- [16] Aldair E. Gongora, Bowen Xu, Wyatt Perry, Chika Okoye, Patrick Riley, Kristofer G. Reyes, Elise F. Morgan, and Keith A. Brown. A Bayesian experimental autonomous researcher for mechanical design. *Science Advances*, 6(15):eaaz1708, April 2020. doi:10.1126/sciadv.aaz1708. <https://www.science.org/doi/10.1126/sciadv.aaz1708>.
- [17] Benjamin Burger, Phillip M. Maffettone, Vladimir V. Gusev, Catherine M. Aitchison, Yang Bai, Xiaoyan Wang, Xiaobo Li, Ben M. Alston, Buyi Li, Rob Clowes, Nicola Rankin, Brandon Harris, Reiner Sebastian Sprick, and Andrew I. Cooper. A mobile robotic chemist. *Nature*, 583(7815):237–241, July 2020. ISSN 1476-4687. doi:10.1038/s41586-020-2442-2. <https://www.nature.com/articles/s41586-020-2442-2>.
- [18] Jiaru Bai, Sebastian Mosbach, Connor J. Taylor, Dogancan Karan, Kok Foong Lee, Simon D. Rihm, Jethro Akroyd, Alexei A. Lapkin, and Markus Kraft. A Dynamic Knowledge Graph Approach to Distributed Self-Driving Laboratories. *Nat. Commun.*, 15:462, 2024. doi:10.1038/s41467-023-44599-9.
- [19] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, Grzegorz Glowaty, Felix Weissenberger, Alessio Orlandi, Dan Popovici, Anil Palepu, Keran Rong, Ryutaro Tanno, Khaled Saab, Fan Zhang, Jacob Blum, Andrew Carroll, Kavita Kulkarni, Nenad Tomašev, Dina Zverinski, Ivor Rendulic, Elahe Vedadi, Florian Hasler, Luka Rimanic, Marina Boia, Ivan Budiselic, Ben Feinstein, Mathias Bellaiche, Tom Sheffer, Jan Freyberg, Jeremy Ratcliff, Ottavia Bertolli, Katherine Chou, Avinatan Hassidim, Burak Gokturk, Amin Vahdat, Yuan Guan, Vikram Dhillon, Eeshit Dhaval Vaishnav, Byron Lee, Tiago R. D. Costa, José R. Penadés, Gary Peltz, Yossi Matias, James Manyika, Demis Hassabis, Yunhan Xu, Pushmeet Kohli, Annalisa Pawlosky, Alan Karthikesalingam, and Vivek Natarajan. Accelerating scientific discovery with Co-Scientist. *Nature*, pages 1–3, May 2026. ISSN 1476-4687. doi:10.1038/s41586-026-10644-y. <https://www.nature.com/articles/s41586-026-10644-y>.
- [20] Andrew Ian Cooper, Qi Jie Yeow, Ziheng Xiao, Mengjia Zhu, Abdoulatif Cissé, Xenophon Evangelopoulos, and Charlotte Boott. Large language models do not replace chemists in a closed-loop catalysis experiment, July 2026. <https://www.researchsquare.com/article/rs-10032842/v1>. ISSN: 2693-5015.
- [21] Zhangde Song, Jieyu Lu, Yuanqi Du, Botao Yu, Thomas M. Pruyn, Yue Huang, Kehan Guo, Xiuzhe Luo, Yuanhao Qu, Yi Qu, Yinkai Wang, Haorui Wang, Jeff Guo, Jingru Gan, Parshin Shojaei, Di Luo, Andres M. Bran, Gen Li, Qiyuan Zhao, Shao-Xiong Lennon Luo, Yuxuan Zhang, Xiang Zou, Wanru Zhao, Yifan F. Zhang, Wucheng Zhang, Shunan Zheng, Saiyang Zhang, Sartaa Takrim Khan, Mahyar Rajabi-Kochi, Samantha Paradi-Maropakis, Tony Baltoiu, Fengyu Xie, Tianyang Chen, Kexin Huang, Weiliang Luo, Meijing Fang, Xin Yang, Lixue Cheng, Jiajun He, Soha Hassoun, Xiangliang Zhang, Wei Wang, Chandan K. Reddy, Chao Zhang, Zhiling Zheng, Mengdi Wang, Le Cong, Carla P. Gomes, Chang-Yu Hsieh, Aditya Nandy, Philippe Schwaller, Heather J. Kulik, Haojun Jia, Huan Sun, Seyed Mohamad Moosavi, and Chenru Duan. Evaluating Large Language Models in Scientific Discovery, December 2025. <http://arxiv.org/abs/2512.15567>. arXiv:2512.15567 [cs].
- [22] Mayk Caldas Ramos, Christopher J. Collison, and Andrew D. White. A review of large language models and autonomous agents in chemistry. *Chemical Science*, 16(6):2514–2572, 2025. ISSN 2041-6539. doi:10.1039/d4sc03921a.
- [23] Yunheng Zou, Austin H. Cheng, Abdulrahman Aldossary, Jiaru Bai, Shi Xuan Leong, Jorge Arturo Campos-Gonzalez-Angulo, Changhyeok Choi, Cher Tian Ser, Gary Tom, Andrew Wang, Zijian Zhang, Ilya Yakavets, Han Hao, Chris Crebolder, Varinia Bernales, and Alán Aspuru-Guzik. El Agente: An autonomous agent for quantum chemistry. *Matter*, 8(7):102263, July 2025. ISSN 2590-2385. doi:10.1016/j.matt.2025.102263. <https://www.sciencedirect.com/science/article/pii/S2590238525003066>.
- [24] Ziqi Wang, Hongshuo Huang, Hancheng Zhao, Changwen Xu, Shang Zhu, Jan Janssen, and Venkatasubramanian Viswanathan. DREAMS: Density Functional Theory Based Research Engine for Agentic Materials Simulation. *arXiv Preprint*, 2025. doi:10.48550/arxiv.2507.14267.
- [25] Jinming Hu, Hassan Nawaz, Yuting Rui, Lijie Chi, Arif Ullah, and Pavlo O. Dral. Aitomia: Your intelligent assistant for ai-driven atomistic and quantum chemical simulations. *arXiv Preprint*, 2025. doi:10.48550/arxiv.2505.08195.
- [26] Thang D. Pham, Aditya Tanikanti, and Murat Keçeli. ChemGraph as an agentic framework for computational chemistry workflows. *Commun. Chem.*, 9(1), 2026. doi:10.1038/s42004-025-01776-9.

- [27] Juan B. Pérez-Sánchez, Yunheng Zou, Jorge A. Campos-Gonzalez-Angulo, Marcel Müller, Ignacio Gustin, Andrew Wang, Han Hao, Tsz Wai Ko, Changhyeok Choi, Eric S. Isbrandt, Mohammad Ghazi Vakili, Hanyong Xu, Chris Crebolder, Varinia Bernales, and Alán Aspuru-Guzik. El Agente Quntur: A research collaborator agent for quantum chemistry, February 2026. <http://arxiv.org/abs/2602.04850>. arXiv:2602.04850 [physics].
- [28] Jiaru Bai, Abdulrahman Aldossary, Thomas Swanick, Marcel Müller, Yeonghun Kang, Zijian Zhang, Jin Won Lee, Tsz Wai Ko, Mohammad Ghazi Vakili, Varinia Bernales, and Alán Aspuru-Guzik. El Agente Gráfico: Structured Execution Graphs for Scientific Agents, February 2026. <http://arxiv.org/abs/2602.17902>. arXiv:2602.17902 [cs].
- [29] Sai Govind Hari Kumar, Yunheng Zou, Andrew Wang, Jesús Valdés-Hernández, Tsz Wai Ko, Nathan Yue, Olivia Leng, Hanyong Xu, Chris Crebolder, Alán Aspuru-Guzik, and Varinia Bernales. El Agente Sólido: A New Age(nt) for Solid State Simulations, February 2026. <http://arxiv.org/abs/2602.17886>. arXiv:2602.17886 [cond-mat.mtrl-sci].
- [30] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. Augmenting Large Language Models with Chemistry Tools. *Nat. Mach. Intell.*, 6(5):525–535, 2024. doi:10.1038/s42256-024-00832-8.
- [31] Keyan Ding, Jing Yu, Junjie Huang, Yuchen Yang, Qiang Zhang, and Huajun Chen. SciToolAgent: A knowledge-graph-driven scientific agent for multitool integration. *Nat. Comp. Sci.*, 5(10):962–972, 2025. doi:10.1038/s43588-025-00849-y.
- [32] Changhyeok Choi, Yunheng Zou, Marcel Müller, Han Hao, Yeonghun Kang, Juan B. Pérez-Sánchez, Ignacio Gustin, Hanyong Xu, Mohammad Ghazi Vakili, Chris Crebolder, Alán Aspuru-Guzik, and Varinia Bernales. El Agente Estructural: An Artificially Intelligent Molecular Editor, February 2026. <http://arxiv.org/abs/2602.04849>. arXiv:2602.04849 [physics].
- [33] Ali E. Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J. Szostkiewicz, Dmytro Shved, Gavin J. Gyimesi, Jon M. Laurent, Samantha M. Wright, Muhammed T. Razzak, Andrew D. White, Silvia C. Finnemann, Michaela M. Hinks, and Samuel G. Rodrigues. A multi-agent system for automating scientific discovery. *Nature*, 655(8122):497–505, 2026. ISSN 1476-4687. doi:10.1038/s41586-026-10652-y.
- [34] Behnam Yousefi, Nora Constanze Laubach, Sven Heins, Lucia Testa, Søren W. Gersting, and Stefan Bonn. BioChemAIgent: An AI-driven Protein Modeling and Docking Framework for Structure-Based Drug Discovery, December 2025. <https://www.biorxiv.org/content/10.64898/2025.12.17.694892v1>. ISSN: 2692-8205 Pages: 2025.12.17.694892 Section: New Results.
- [35] Anthropic. Claude Science, an AI workbench for scientists, June 2026. <https://www.anthropic.com/news/claude-science-ai-workbench>.
- [36] Thanapat Worakul and Clémence Corminboeuf. Solitarius: An Agentic Architecture for Computer-aided Molecular Inverse Design. *ChemRxiv*, July 2026. doi:10.26434/chemrxiv.15005834/v2. <https://doi.org/10.26434/chemrxiv.15005834/v2>. Preprint, Version 2.
- [37] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, December 2023. ISSN 1476-4687. doi:10.1038/s41586-023-06792-0. <https://www.nature.com/articles/s41586-023-06792-0>.
- [38] Alexis A. Smith, Edmund L. Wong, Ronan C. Donovan, Brad A. Chapman, Ryan Harry, Pooyan Tirandazi, Paulina Kanigowska, Elizabeth A. Gendreau, Robert H. Dahl, Michal Jastrzebski, Jose E. Cortez, Christopher J. Bremner, José C. Morales Hemuda, James Dooner, Ian Graves, Rahul Karandikar, Christopher Lionetti, Kevin Christopher, Andrew L. Consiglio, Alyssa Tran, William McCusker, Duy X. Nguyen, Isis Botelho Nunes da Silva, Alvaro R. Bautista-Ayala, Monica P. McNeerney, Sean Atkins, Michael McDuffie, Will Serber, Bradley P. Barber, Trinh Thanongsinh, Andrew Nesson, Bibek Lama, Brandon Nichols, Cameron LaFrance, Tenzing Nyima, Alicia Byrn, Rashard Thornhill, Bryan Cai, Lizvette Ayala-Valdez, Alycia Wong, Austin J. Che, Walter Thavarajah, Daniel Smith, Thomas F. Knight, David W. Borhani, Jerry Tworek, Mostafa Rohaninejad, Ahmed El-Kishky, Nathan C. Tedford, Tejal Patwardhan, Yunxin Joy Jiao, and Reshma P. Shetty. Using a GPT-5-driven autonomous lab to optimize the cost and titer of cell-free protein synthesis, February 2026. ISSN 2692-8205.
- [39] Robert MacKnight, Jose Emilio Regio, Jeffrey G. Ethier, Luke A. Baldwin, and Gabe Gomes. Pre-trained knowledge elevates large language models beyond traditional chemical reaction optimizers, October 2025. <http://arxiv.org/abs/2509.00103>. arXiv:2509.00103 [cs.LG].
- [40] Rushil Gupta, Jason Hartford, and Bang Liu. LLMs for Bayesian Optimization in Scientific Domains: Are We There Yet?, September 2025. <http://arxiv.org/abs/2509.21403>. arXiv:2509.21403 [cs.LG].

- [41] Amirreza Mottafegh and Gwang-Noh Ahn. Adaptive Human-in-the-Loop Optimization Using Language-Guided Priors for Chemical Synthesis. In *AI4X – Accelerate Conference 2026*, March 2026. <https://openreview.net/forum?id=Gni4MCry6X>.
- [42] Abdoulatif Cissé, Max E. Cooper, Mengjia Zhu, Xenophon Evangelopoulos, and Andrew I. Cooper. Can we automate scientific reasoning in closed-loop experiments using large language models? *Digital Discovery*, 5(3): 1132–1160, March 2026. ISSN 2635-098X. doi:10.1039/D5DD00520E. <https://pubs.rsc.org/en/content/articlelanding/2026/dd/d5dd00520e>.
- [43] Andrew Ritchhart, Sarah I. Allec, Pravalika Butreddy, Krista Kulesa, Qingpu Wang, Dan Thien Nguyen, Maxim Ziatdinov, and Elias Nakouzi. Agentic workflow enables the recovery of critical materials from complex feedstocks via selective precipitation. *Materials Horizons*, May 2026. ISSN 2051-6355. doi:10.1039/D6MH00475J. <https://pubs.rsc.org/en/content/articlelanding/2026/mh/d6mh00475j>.
- [44] Paul Brunzema, Louis Tiao, Nhat Le, Kevin De Angeli, Yao Xuan, and Djordje Gligorijevic. Agentic Bayesian Optimization through Surrogate-Augmented Autoresearch, August 2026. <http://arxiv.org/abs/2608.00316>. arXiv:2608.00316 [cs.LG].
- [45] Mohammad Nazeri, Sheldon Mei, Jeffrey Watchorn, Alex Zhang, Erin Ng, Tao Wen, Abhijoy Mandal, Kevin Golovin, Alán Aspuru-Guzik, and Frank Gu. RAISE: A self-driving laboratory for interfacial property formulation discovery. *Digital Discovery*, 5(5):2254–2270, May 2026. ISSN 2635-098X. doi:10.1039/D5DD00531K.
- [46] Simone Pilon, Elia Savino, Oliver M. Bayley, Michael Vanzella, Miguel Claros, Petros Siasiaridis, Junsong Liu, Florian Lukas, Matteo Damian, Vasilis Tseliou, Niccolò Intini, Aidan Slattery, Jesus SanJosé-Orduna, Tim den Hartog, Ron A. H. Peters, Andrea F. G. Gargano, Francesco G. Mutti, and Timothy Noël. A flexible and affordable self-driving laboratory for automated reaction optimization. *Nature Synthesis*, pages 1–13, April 2026. ISSN 2731-0582. doi:10.1038/s44160-026-01053-0.
- [47] RDKit: Open-source cheminformatics. <https://www.rdkit.org>. Software.
- [48] Ruben Laplaza, Simone Gallarati, and Clemence Corminboeuf. Genetic Optimization of Homogeneous Catalysts. *Chemistry–Methods*, 2(6):e202100107, 2022. ISSN 2628-9725. doi:10.1002/cmt.d.202100107. <https://onlinelibrary.wiley.com/doi/abs/10.1002/cmt.d.202100107>. _eprint: <https://chemistry-europe.onlinelibrary.wiley.com/doi/pdf/10.1002/cmt.d.202100107>.
- [49] Robert Pollice, Pascal Friederich, Cyrille Lavigne, Gabriel dos Passos Gomes, and Alán Aspuru-Guzik. Organic molecules with inverted gaps between first excited singlet and triplet states and appreciable fluorescence rates. *Matter*, 4(5):1654–1682, 2021. ISSN 2590-2385. doi:10.1016/j.matt.2021.02.017. <https://www.sciencedirect.com/science/article/pii/S2590238521000813>.
- [50] Drew M. Hood, Ryan A. Johnson, Alex E. Carpenter, Jarod M. Younker, David J. Vinyard, and George G. Stanley. Highly active cationic cobalt(II) hydroformylation catalysts. *Science*, 367(6477):542–548, January 2020. doi:10.1126/science.aaw7742.
- [51] He Zhou, Chunhui Wu, Huimin Xu, Xiaochong Xue, Jinglin Li, Youshi Zeng, Xinxin Chu, Xiaoling Wu, and Wei Liu. Efficient screening for enhanced Xe/Kr separation via fixed-ligand, variable-metal strategy in metal–organic frameworks. *RSC Advances*, 16(34):33407–33418, July 2026. ISSN 2046-2069. doi:10.1039/d6ra02043g. <https://doi.org/10.1039/d6ra02043g>.
- [52] Yunsung Lim, Junkil Park, Sangwon Lee, and Jihan Kim. Finely tuned inverse design of metal–organic frameworks with user-desired Xe/Kr selectivity. *Journal of Materials Chemistry A*, 9(37):21175–21183, October 2021. ISSN 2050-7488. doi:10.1039/d1ta03129e. <https://doi.org/10.1039/d1ta03129e>.
- [53] Sangwon Lee, Baekjun Kim, Hyun Cho, Hooseung Lee, Sarah Yunmi Lee, Eun Seon Cho, and Jihan Kim. Computational Screening of Trillions of Metal–Organic Frameworks for High-Performance Methane Storage. *ACS Appl. Mater. Interfaces*, 13(20):23647–23654, 2021. doi:10.1021/acsami.1c02471.
- [54] Thomas F. Willems, Chris H. Rycroft, Michael Kazi, Juan C. Meza, and Maciej Haranczyk. Algorithms and tools for high-throughput geometry-based analysis of crystalline porous materials. *Micropor. Mesopor. Mat.*, 149(1):134–141, 2012. doi:10.1016/j.micromeso.2011.08.020.
- [55] Tom M. Masson, Stefan D. A. Zondag, Jasper H. A. Schuurmans, and Timothy Noël. Open-source 3D printed reactors for reproducible batch and continuous-flow photon-induced chemistry: Design and characterization. *Reaction Chemistry & Engineering*, 9(8):2218–2225, 2024. ISSN 2058-9883. doi:10.1039/D4RE00081A.

- [56] Michael Vanzella, Oliver M. Bayley, Rick S. van den Hurk, Elia Savino, Miguel Claros, Ruben Feringa, Ron A.H. Peters, Moussa Barhoum, Tim den Hartog, Andrea F.G. Gargano, Bob W.J. Pirok, and Timothy Noël. Autonomous control of polymer upcycling with a self-driving laboratory. *ChemRxiv*, June 2026. doi:10.26434/chemrxiv.15005284/v1. <https://doi.org/10.26434/chemrxiv.15005284/v1>. Preprint, Version 1.
- [57] Shi Xuan Leong, Caleb E Griesbach, Rui Zhang, Kourosh Darvish, Yuchi Zhao, Abhijoy Mandal, Yunheng Zou, Han Hao, Varinia Bernales, and Alán Aspuru-Guzik. Steering towards safe self-driving laboratories. *Nat. Rev. Chem.*, 9(10):707–722, 2025. doi:10.1038/s41570-025-00747-x.
- [58] Yeonghun Kang, Shi Xuan Leong, Hyun Suk Park, Jiaru Bai, Sven Papidocha, Yuchi Zhao, Rui Zhang, Magali Gimeno, Stanley Lo, Eric S. Isbrandt, Han Hao, Yang Cao, Felix Katzenburg, Fangshi Du, Alán Aspuru-Guzik, and Varinia Bernales. El Agente Seguro: An agent for chemical safety. *ChemRxiv*, June 2026. doi:10.26434/chemrxiv.15004378/v2. <https://doi.org/10.26434/chemrxiv.15004378/v2>. Preprint, Version 2.
- [59] Alán Aspuru-Guzik and Varinia Bernales. Reaction: Building trust infrastructure for agentic experimental science beyond fragile eloquence. *Chem*, 12(8):103163, 2026. ISSN 2451-9294. doi:10.1016/j.chempr.2026.103163.
- [60] Jasper H. A. Schuurmans, Prakash Chandra Tiwari, and Timothy Noël. Guiding photochemical process intensification through kinetic diagnostics. *Angewandte Chemie International Edition*, n/a(n/a):e2620716. doi:10.1002/anie.2620716.
- [61] Rui Li, Qiming Sun, Xing Zhang, and Garnet Kin-Lic Chan. Introducing GPU Acceleration into the Python-Based Simulations of Chemistry Framework. *The Journal of Physical Chemistry A*, 129(5):1459–1468, January 2025. ISSN 1089-5639. doi:10.1021/acs.jpca.4c05876.
- [62] Christoph Bannwarth, Eike Caldeweyher, Sebastian Ehlert, Andreas Hansen, Philipp Pracht, Jakob Seibert, Sebastian Spicher, and Stefan Grimme. Extended tight-binding quantum chemistry methods. *WIREs Computational Molecular Science*, 11(2):e1493, 2021. ISSN 1759-0884. doi:10.1002/wcms.1493.
- [63] Thomas Froitzheim, Marcel Müller, Andreas Hansen, and Stefan Grimme. g-xTB: A General-Purpose Extended Tight-Binding Electronic Structure Method For the Elements H to Lr (Z=1–103). *ChemRxiv*, June 2025. doi:10.26434/chemrxiv-2025-bjxvt. <https://chemrxiv.org/engage/chemrxiv/article-details/685434533ba0887c335fc974>. Preprint.
- [64] Willi Gottstein, Aurélien Blanc, Sissi Feng, Brandon R. Sutherland, and Sergio Pablo-García. A foundational representation for an orchestrated lab. *Device*, page 101202, 2026. ISSN 2666-9986. doi:10.1016/j.device.2026.101202.
- [65] Madeleine A. Gaidimas, Jiaru Bai, Yeonghun Kang, Kent O. Kirlikovali, Varinia Bernales, Alán Aspuru-Guzik, and Omar K. Farha. Reimagining metal-organic framework discovery: Integrating experiment, computation, and artificial intelligence. *Chem*, 12(4):102921, 2026. ISSN 2451-9294. doi:10.1016/j.chempr.2025.102921.
- [66] David H. Ackley. *A Connectionist Machine for Genetic Hillclimbing*. Kluwer Academic Publishers, Boston, MA, 1987. doi:10.1007/978-1-4613-1997-9.
- [67] Benjamin J. Shields, Jason Stevens, Jun Li, Marvin Parasram, Farhan Damani, Jesus I. Martinez Alvarado, Jacob M. Janey, Ryan P. Adams, and Abigail G. Doyle. Bayesian reaction optimization as a tool for chemical synthesis. *Nature*, 590(7844):89–96, February 2021. ISSN 1476-4687. doi:10.1038/s41586-021-03213-y.
- [68] Anna Zdziennicka and Bronisław Jańczuk. The relationship between the adhesion work, the wettability and composition of the surface layer in the systems polymer/aqueous solution of anionic surfactants and alcohol mixtures. *Applied Surface Science*, 257(3):1034–1042, 2010. ISSN 0169-4332. doi:10.1016/j.apsusc.2010.08.012.
- [69] C. Robert McElroy, Andri Constantinou, Leonie C. Jones, Louise Summerton, and James H. Clark. Towards a holistic approach to metrics for the 21st century pharmaceutical industry. *Green Chemistry*, 17(5):3111–3121, 05 2015. ISSN 1463-9262. doi:10.1039/c5gc00340g. <https://doi.org/10.1039/c5gc00340g>.
- [70] Eurostat. Electricity prices for non-household consumers – bi-annual data (from 2007 onwards). Online data code nrg_pc_205, 2025. https://ec.europa.eu/eurostat/databrowser/view/nrg_pc_205. Netherlands, consumption band 500–2000 MWh/yr, 2025S1; excluding VAT and other recoverable taxes and levies. Accessed <DATE>.
- [71] Eurostat. Electricity price statistics. Statistics Explained, 2025. https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Electricity_price_statistics. Accessed <DATE>.

Supporting Information

Contents

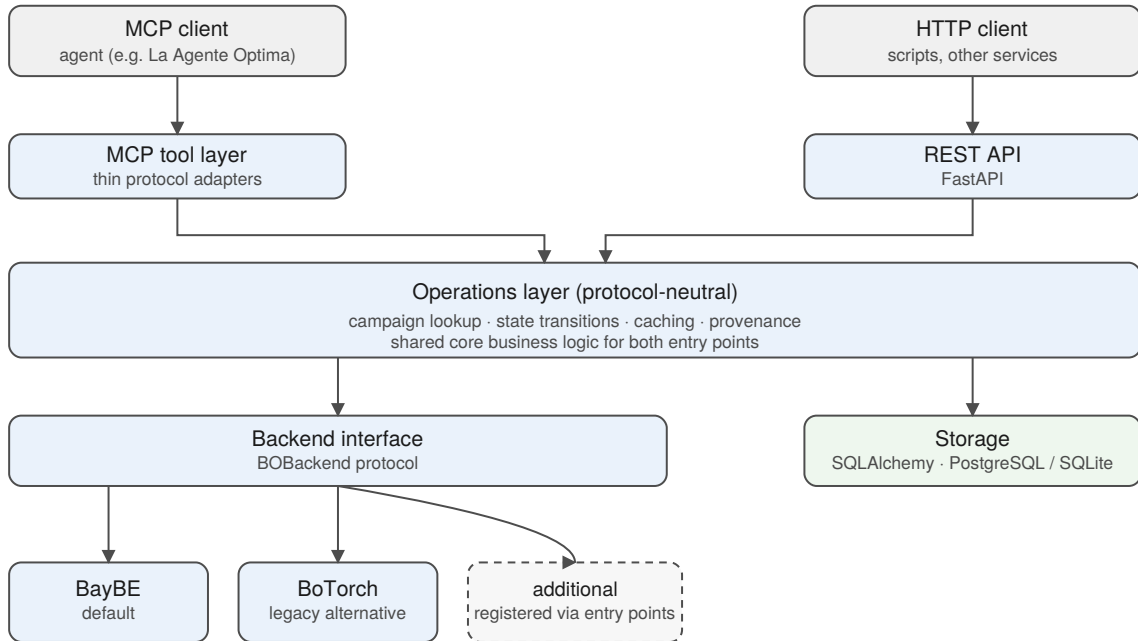
A	BO-MCP: system architecture and implementation details	22
A.1	Request flow and architecture	22
A.2	Backend abstraction and algorithm selection	23
A.3	Campaign lifecycle and concurrency	24
A.4	Provenance, diagnostics, and explainability	25
A.5	Molecular and other categorical representations	26
B	Framework comparison	27
B.1	Framework-comparison evaluation details	30
C	Supplementary material for the showcases	32
C.1	Phosphine ligand electronic tuning	32
C.2	Singlet–triplet gap search over the Pollice 2021 library	36
C.3	Agent-to-agent cobalt catalyst tuning	40
C.4	Xe/Kr separation over PORMAKE-assembled MOFs	44
C.5	Contact angle matching with RAISE	48
C.6	The RoboChem-Flex campaign	52
C.7	Cost accounting of the agent-directed campaign	115

A BO-MCP: system architecture and implementation details

This section expands on the BO-MCP implementation summarized in the main text: how a request reaches the optimization engine (Sec. A.1), how the backends are selected (Sec. A.2), how campaigns are tracked and versioned (Sec. A.3), how the system stays auditable and explainable (Sec. A.4), and how molecular design spaces are represented (Sec. A.5).

A.1 Request flow and architecture

BO-MCP has a layered architecture, implemented in Python using FastMCP for the MCP interface, FastAPI for the REST interface, Pydantic for data schemas and SQLAlchemy for persistence. MCP tools and REST routes act as thin protocol adapters: they handle transport-specific concerns such as request parsing, identity, idempotency, and response formatting before invoking a shared, protocol-neutral operations layer. That operations layer implements the core campaign-orchestration logic, including campaign lookup, lifecycle transitions, caching, and provenance persistence, and coordinates two parallel lower-level components: a pluggable optimization backend and a persistent storage layer. Because MCP and REST invoke the same operation functions, they share core business semantics, while retaining transport-specific validation, authorization, and presentation behaviour. Storage is a SQLAlchemy layer that supports both PostgreSQL, for shared deployments, and SQLite, for isolated single-user use. Fig. 6 summarizes this layering.



Layered request flow from MCP or HTTP clients to shared operations, storage, and the optimization backend.

Figure 6 Layered request flow in BO-MCP. An MCP client (an agent) and an HTTP client reach the same protocol-neutral operations layer through separate thin protocol adapters, so both interfaces share core business semantics. The operations layer coordinates two parallel components: a pluggable optimization backend, BayBE by default, BoTorch as a legacy alternative, or a third-party backend registered via a Python entry point, and a SQLAlchemy storage layer backed by PostgreSQL or SQLite.

The tool layer exposes 22 MCP tools, grouped by workflow stage in Tab. 2, together with eight read-only MCP resources that expose campaign, suggestion, and event state directly by URI (for example, `campaign://{campaign_id}`) without requiring an explicit tool call. Rather than reproduce the full one-by-one listing here, we summarize the tools by workflow stage in Tab. 2. Exact tool schemas are exposed dynamically through MCP tool discovery, REST schemas through generated OpenAPI documentation, and resource URIs and templates through MCP resource discovery, rather than published as a fixed, static reference.

Table 2 MCP tools grouped by workflow stage (22 total). Eight further read-only MCP resources expose campaign, suggestion, and event state directly by URI. Exact schemas, and the URIs, templates, and descriptions of the resources, are discoverable dynamically rather than reproduced here.

Stage	Tools	Purpose
Setup	3	Create a campaign, validate a specification before committing it, list backend capabilities
Exploration	5	List and filter campaigns, results, and suggestions; compare campaigns; export a full campaign record
Optimization	3	Generate the next batch of suggestions; explain why a suggestion was made; accept, reject, or expire a suggestion
Results	2	Submit results directly, or from an uploaded file
Diagnostics	4	Model health, convergence, and Pareto/hypervolume diagnostics; server health; batch status across campaigns; poll a long-running call
Lifecycle	4	Pause, resume, terminate, and reopen a campaign
Transfer	1	Discover campaigns suitable as transfer-learning sources

The MCP server can be run over three transports: stdio, for direct integration with an agent process; streamable HTTP, for a persistent, shared server that multiple MCP clients connect to concurrently; and a legacy SSE transport retained for older clients. The REST API does not use any of these MCP transports and is not connected to the MCP server; it runs as an independent service that, in a typical deployment, shares only the same database and the same Python operations code with the MCP server, so MCP and REST clients observe the same persisted campaign state, subject to transport-specific authorization and response formatting, rather than receiving session-local copies. The MCP HTTP transports do not currently provide built-in per-tenant authorization; network-exposed deployments are expected to rely on loopback binding or an authenticating proxy in front of the server.

A.2 Backend abstraction and algorithm selection

The optimization engine is accessed only through a backend protocol comprising fifteen methods and properties in total: property accessors for backend identity and for supported and conditionally supported features, schema hooks for parameter and backend-specific options, validation methods for specifications and capabilities, generation methods for initial designs and suggestions, and further methods covering hypervolume computation, duplicate detection, state updates after new results, batch-diversity computation, method selection, and diagnostics. All of these accept and return plain Python types rather than backend-specific tensors or data frames. Two backends implement this protocol today. BayBE (9) is the default and provides native mixed search spaces (continuous, discrete, and categorical parameters together), multi-objective optimization via a Pareto objective, and the molecular-representation support described in Sec. A.5.

BoTorch is retained as a legacy backend and additionally supports trust-region (TuRBO) search for high-dimensional problems, which the standard suggestion pipeline dispatches to automatically for single-objective campaigns once the search space reaches 20 or more parameters. Three further modules, sparse-axis-aligned-subspace BO (SAASBO) for very high-dimensional problems, multi-fidelity optimization, and ensemble-based (RGPE) transfer learning between campaigns, are also implemented, but are not yet dispatched automatically through the standard suggestion pipeline; campaign intake requesting one of these features is rejected with a structured backend-capability error rather than silently downgraded to a simpler method. New backends are not hard-coded: they register under a Python entry-point group and are discovered at start-up, so a campaign can request a specific backend, or leave the choice to an automatic selector that checks which registered backend supports the requested features.

For model-guided optimization, continuous search spaces use L-BFGS-B, purely categorical spaces use discrete

Table 3 Feature support by backend, as currently implemented. Trust-region search is dispatched automatically within the standard suggestion pipeline for single-objective campaigns once the search space is large enough; SAASBO, multi-fidelity, and transfer learning beyond the task-type mechanism are implemented on the BoTorch backend but not yet dispatched through that pipeline, and campaign intake requesting one of them directly is rejected with a structured backend-capability error rather than silently downgraded.

Feature	BayBE (default)	BoTorch (legacy)
Multi-objective	Yes	Yes
Mixed search space	Yes, native	Yes
Molecular/substance parameters	Yes	No, rejected as unsupported
Trust-region search (TurBO)	Not supported	Yes, auto-dispatched for single-objective campaigns at 20 or more parameters
High-dimensional sparse BO (SAASBO)	Not supported	Implemented, not yet dispatched
Multi-fidelity	Not supported	Implemented, not yet dispatched
Transfer learning	Supported via a declared task-type parameter	Standalone RGPE module implemented, not dispatched
Leave-one-out validation	cross- No	Posterior downdate at fixed hyperparameters, batched-refit fallback

enumeration over the categorical pool, and mixed continuous/categorical spaces use mixed discrete/continuous optimization. Models with categorical inputs use a mixed kernel construction, combining a continuous kernel over the continuous parameters with a Hamming kernel over the categorical ones.

A.3 Campaign lifecycle and concurrency

Each campaign is a versioned record with an explicit status: created, running, paused, completed, or failed. Only the first four are reachable in the current implementation; failed is defined in the data model and consumed defensively wherever status is read (health scoring, transfer-candidate filtering, recommended next actions), but nothing currently writes it, so a campaign cannot yet end in that state. Four actions move a campaign between the reachable states, summarized in [Tab. 5](#).

These four are the explicit lifecycle actions; a campaign also moves automatically from created to running the first time suggestion generation succeeds, without a dedicated lifecycle action being invoked.

A version counter increases on every campaign-level mutation, lifecycle transitions, suggestion generation, and result submission, and is checked before each write: if two updates race, the second one to commit is rejected rather than silently overwriting the first, and the caller can retry against the latest state. Because a network failure can leave a caller uncertain whether its previous request actually succeeded, pause, resume, and terminate treat a retry that finds the campaign already at the intended state as a success rather than an error. Reopen is deliberately excluded from this behaviour: a fresh, never-completed campaign and a reopened one are indistinguishable once running, so treating a mistaken reopen as a silent no-op would hide a genuine error rather than surface it. Manual suggestion-status transitions, accepting, rejecting, or expiring a suggestion, go through a separate atomic compare-and-update on the suggestion row and do not increment the campaign version; a diagnostics response cached before such a transition may therefore continue to reflect the pre-transition suggestion state until the cache entry expires. Every lifecycle tool additionally supports a dry-run mode that validates and reports a proposed transition without committing it.

Table 4 Automatic model and acquisition-function selection on the BoTorch backend, applied when no explicit configuration is supplied. The BayBE backend uses its own internal selection logic and is not shown here.

Phase / problem	Model	Acquisition	Strategy
Initial design: fewer than $\max(2, n_{\text{parameters}} + 1)$, configured <code>initial_design_size</code> observations	None	None	Sobol space-filling design
Model-guided, one objective, fewer than 20 parameters	SingleTaskGP	qLogNEI	Standard acquisition optimization
Model-guided, one objective, 20 or more parameters	SingleTaskGP	qLogNEI	Trust-region (TuRBO) search
Model-guided, two or more objectives	ModelListGP	qLogNEHVI	Standard acquisition optimization, no TuRBO

Table 5 The four campaign lifecycle actions and the states from which each is valid. Pause, resume, and terminate additionally return a no-op success, rather than an error, when a retried call finds the campaign already at the listed result state; reopen has no such fallback and always requires a completed campaign.

Action	Valid from	Result
pause	running	paused
resume	paused	running
terminate	created, running, or paused	completed
reopen	completed	running

A.4 Provenance, diagnostics, and explainability

Every successfully dispatched MCP tool call that returns an application result, including expected structured failures, is recorded as a compact audit event (a truncated summary of inputs, a success flag, and any error code), attributed to a campaign where applicable and queryable per campaign. Audit writes happen in a transaction separate from the underlying operation and are best-effort by default: a failed audit write does not block or roll back the operation it describes, it only increments an internal failure counter so persistent gaps can be caught by monitoring. An optional fatal mode is available for deployments that want audit-persistence failures surfaced to the caller rather than silently tolerated; because the audit write happens after the business operation has already been committed, in its own transaction, this mode cannot roll back the operation itself if the write fails. Instead, it replaces the tool’s returned result with a structured database-error envelope, surfacing the logging failure to the caller rather than silently continuing. The `events://` MCP resource exposes the 50 most recent audit events for a campaign as an operational trace, not a complete or paginated audit export.

Failures the system anticipates (an invalid state transition, a malformed specification, a request for a feature the active backend does not support, and so on) are returned as structured errors carrying a stable code, a human-readable message, and machine-actionable recovery guidance, including whether the request is safe to retry, rather than a bare exception.

Diagnostics are computed on request and cached briefly (120 seconds by default), keyed to the campaign’s version, so most mutations invalidate the cache immediately. Manual suggestion-status transitions are the one exception, as noted above: because they do not increment the campaign version, a cached diagnostics response may continue to reflect the pre-transition suggestion state until that cache entry’s window expires. Available diagnostics include the current Pareto front and hypervolume for multi-objective campaigns, with

the hypervolume history accumulated incrementally as results are submitted, model health and convergence indicators, and leave-one-out cross-validation, computed exactly on the BoTorch backend at fixed, already-fitted hyperparameters via a closed-form posterior downdate, with a batched-refit procedure as a fallback, and unavailable on the default BayBE backend. Model diagnostics are likewise backend-dependent: the BoTorch backend reports kernel lengthscales for computational input dimensions, whereas BayBE provides SHAP-based importance for either experimental parameters or their computational representations when the optional `shap` dependency is installed and sufficient observations are available.

Every suggestion records the generation method, the iteration, and the random seed used to produce it. Suggestions generated by the underlying model additionally carry a fuller provenance record where available: the fitted model type, the acquisition function and its value at the suggested point, and the model’s predicted mean and uncertainty; these model-derived fields are not populated for suggestions that are not model-guided, such as those from the initial design.

A.5 Molecular and other categorical representations

For campaigns whose categorical parameters denote molecules (the fragment libraries used in the OSL case study, for example), the BayBE backend can map each category label to a SMILES string and encode it through cheminformatics descriptors rather than treating the labels as unrelated one-hot categories. Roughly three dozen encodings are exposed, mirroring BayBE’s own descriptor and fingerprint options; the default is a Mordred-derived descriptor block (about 1800 descriptors), with lighter alternatives such as ECFP fingerprints, MACCS keys, or RDKit 2D descriptors available where a smaller representation is preferred. A related mechanism accepts a user-supplied numeric descriptor table directly, for precomputed quantum-chemical or spectroscopic features, for example, instead of computing one from a SMILES string, so a representation that already exists does not need to be re-derived. Because the BoTorch backend has no chemistry-aware encoding path, a campaign requesting either mechanism while explicitly pinned to BoTorch is refused with a typed error rather than silently degraded to plain one-hot categories.

Search spaces built from large fragment or category libraries can be combinatorially large. The BayBE backend estimates the size of the resulting discrete space before building it and, above a configurable budget, builds a deterministically subsampled candidate set instead of the full combinatorial product. The base-sampling seed is derived from a canonicalized representation of the search-space parameters and constraints, so the same search-space definition regenerates the same deterministic base sample across processes and rebuilds. Valid observed and actionable pending configurations are then unioned into that sample; points that fall outside the declared parameter value pools or violate a constraint can still be dropped, with a warning. Because campaign state can change between rebuilds, the final candidate frame can differ even when the underlying search-space definition does not.

This implementation, including each backend, the lifecycle state machine, and the diagnostics pipeline, is covered by an automated test suite.

B Framework comparison

As outlined in Sec. 2, we propose an architecture where a specialist subagent authors the campaign, ensures programmatic execution of the optimization loop, and BO-MCP maintains campaign configuration and accumulated observations across iterations. Here, we evaluate whether the design choices underlying LA AGENTE ÓPTIMA support stronger end-to-end optimization than three complementary architectural ablations. The architecture comparison used GPT-5.4 throughout. Separately, we retained the proposed subagent-based architecture and varied the LLM assigned to the BO specialist while keeping GPT-5.4 as the coordinating main agent.

We compared four architectures:

- **Specialist-script (proposed):** the BO specialist authors and smoke-tests a reusable BO-MCP campaign program, which the main agent then executes and monitors.
- **Main-script:** the main agent receives equivalent scientific and scripting guidance. It then authors and executes the BO-MCP campaign program itself.
- **Tool-loop:** the main agent conducts the optimization through repeated BO-MCP tool calls, without a reusable campaign program, subagents, Python, or shell execution.
- **Local-BO:** the main agent implements and executes Bayesian optimization, state management, and result tracking locally without BO-MCP. The agent has full access to its local compute environment.

These targeted end-to-end ablations probe complementary aspects of the proposed separation of reasoning, execution, and campaign-state management; they are not a full factorial decomposition.

We used two complementary single-objective benchmarks: (i) a six-dimensional instance of the Ackley function (66), hereafter Ackley 6D, and (ii) a palladium-catalyzed direct-arylation reaction-yield benchmark derived from the complete dataset reported by Shields et al., hereafter Shields arylation (67; 9). Ackley 6D is a deterministic, multimodal continuous optimization problem with many local optima. The Shields arylation benchmark contains 1728 measured reactions spanning four bases, twelve ligands, four solvents, three concentrations, and three temperatures. Ackley objective values were computed using the deterministic benchmark function, whereas Shields arylation yields were retrieved from the hidden reaction table for one selected set of conditions at a time. Agents could not access, reconstruct, enumerate, or sort the underlying reaction table. Across architectures, prompts varied only to implement the intended responsibility and tool boundaries. Campaign design was left to the agent and therefore part of the evaluated task; stochastic settings such as the random seed were likewise not externally fixed. Each architecture–benchmark combination was evaluated three times. Every run received a global budget of exactly 60 attempted objective evaluations, including evaluations used during smoke testing, debugging, or restarted campaigns. Prespecified run-validity and equal-budget comparability criteria are reported in the Supporting Information (Sec. B.1.1). Optimization quality is reported in Fig. 7a as final quality, defined as the best usable objective value observed within the 60-attempt horizon and reported in the native benchmark units, and as mean best-so-far (MBSF)@60, defined as the mean normalized best-so-far quality across those 60 attempts. Specifically, if b_t is the normalized best usable objective observed by attempt t , then

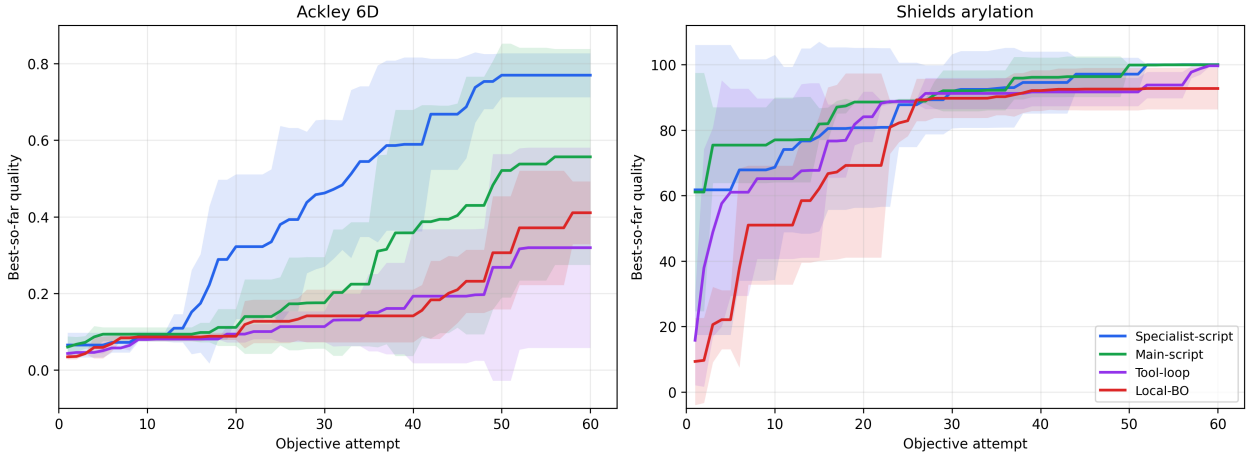
$$\text{MBSF@60} = \frac{1}{60} \sum_{t=1}^{60} b_t.$$

A failed attempt consumes the evaluation budget and does not improve the best-so-far curve. For this calculation, Shields arylation yields were divided by 100, whereas Ackley 6D scores were already on a zero-to-one scale. Mean best-so-far trajectories for each architecture–benchmark combination are shown in Fig. 7b.

Overall, **Specialist-script** achieved the strongest performance on Ackley 6D and reached the maximum Shields arylation yield in all three runs, while all four architectures passed the full protocol. Although **Main-script** achieved the highest mean MBSF@60 for Shields arylation, **Specialist-script** consistently reached the maximum yield across all three runs. Together, these results suggest an advantage for established BO implementations when combined with programmatic execution, while **Tool-loop** shows that access to the

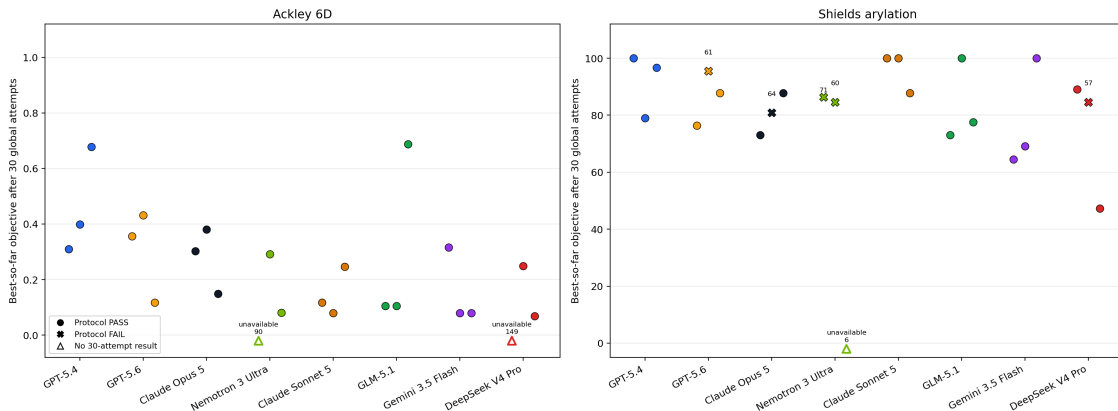
(a) Architecture comparison.

Architecture	Optimization performance				Protocol reliability and resources			
	Ackley final	Ackley MBSF@60	Shields $\geq 90\%$ runs; mean attempt	Shields MBSF@60	Full-protocol valid	Mean cost/run	Mean time/run	Mean tokens/run
Specialist-script	0.770	0.442	3/3; 25.0	0.865	6/6	\$1.61	11.05 min	2.72M
Main-script	0.557	0.265	3/3; 38.3	0.891	6/6	\$0.83	8.49 min	1.24M
Tool-loop	0.319	0.153	3/3; 45.3	0.824	6/6	\$1.42	10.21 min	2.41M
Local-BO	0.411	0.171	1/3; 26.0	0.756	6/6	\$0.26	2.12 min	0.18M



(b) Optimization progress under the proposed architecture and its ablations.

Figure 7 Framework architecture comparison. (a) Architecture-level optimization performance, full-protocol reliability, and inference resources. For the Shields arylation 90%-yield column, entries report the number of successful runs out of three and, after the semicolon, the mean attempt at which 90% yield was first reached, calculated over successful runs only. Cost, time, and token values are means per requested run across the six runs for each architecture. (b) Mean best-so-far trajectories over three 60-attempt runs for each architecture–benchmark combination, with shaded regions showing the sample standard deviation.



(a) Individual outcomes at the 30-attempt horizon.

Specialist model	Ackley	Shields arylation
	Final / MBSF@60 (<i>n</i>)	Final / MBSF@60 (<i>n</i>)
GPT-5.4	0.770 / 0.442 (3)	100.00 / 0.865 (3)
GPT-5.6	0.731 / 0.350 (3)	94.59 / 0.812 (2)
Claude Opus 5	0.772 / 0.344 (3)	94.85 / 0.785 (2)
Nemotron 3 Ultra	0.690 / 0.273 (2)	89.17 / 0.844 (1)
Claude Sonnet 5	0.422 / 0.207 (3)	96.57 / 0.907 (3)
GLM-5.1	0.363 / 0.259 (3)	98.88 / 0.790 (3)
Gemini 3.5 Flash	0.397 / 0.204 (3)	92.21 / 0.637 (3)
DeepSeek V4 Pro	0.566 / 0.212 (2)	73.11 / 0.623 (2)

(b) Aggregate specialist-model optimization quality.

Figure 8 Specialist-model optimization quality and robustness under Specialist-script. (a) Best-so-far objective values at the 30-attempt horizon. Models have the same left-to-right order in both benchmark panels, based on their average normalized 30-attempt best-so-far quality, final quality, and MBSF@60 across both benchmarks. Circles denote protocol-valid runs, filled X markers denote protocol failures, and open triangles identify runs without a reconstructable result at the 30-attempt horizon. The triangles are offset below zero for visibility and do not represent objective values. Labels beside non-passing runs report the case-wide number of attempted objective evaluations. (b) Cells report mean final quality / mean MBSF@60. The number in parentheses indicates how many of the three requested runs were included in each mean. Shields arylation final quality is reported as yield (%).

optimizer service alone is not sufficient. **Tool-loop** is disadvantaged by the need to translate each function evaluation into a result submission to the BO-MCP backend and, conversely, to execute each function evaluation from the candidate proposed by BO-MCP; these steps must be performed explicitly at every iteration rather than programmatically. While **Main-script** performs comparably to **Specialist-script** on Shields arylation at a noticeably lower cost, we argue that an important practical advantage of the latter is difficult to capture in static benchmarks such as those conducted here: separating the LLM context used for script creation and campaign authoring from the domain-specific context (for example, reasoning about reaction-yield optimization) helps prevent technical implementation details from interfering with domain-level reasoning. This context separation can also limit context-window growth during large optimization campaigns or subsequent follow-up interactions. Detailed per-architecture results and resource interpretation are provided in the Supporting Information (Sec. B.1.2).

We next evaluated eight specialist models under **Specialist-script**: Claude Opus 5, Claude Sonnet 5, DeepSeek V4 Pro, Gemini 3.5 Flash, GLM-5.1, GPT-5.4, GPT-5.6, and Nemotron 3 Ultra. Optimization quality and workflow reliability were considered separately because model-level quality means include only

scientifically comparable trajectories.

Across the specialist-model analysis, 48 runs for 8 different models, 2 different benchmark tasks, and 3 repetitions per combination were requested. Of these, 41 were eligible for equal-budget analysis and 40 passed the full protocol. The eight protocol-failing outcomes included seven failures of the global-budget criterion and one ownership-only violation. These outcomes were reported as workflow results rather than treated as missing data; runs that were not scientifically comparable were excluded from equal-budget quality aggregates. List-price accounting was exact for 47 runs; one **Nemotron 3 Ultra** Shields arylation run is reported as a lower bound because part of the **GPT-5.4** main-agent usage was not retained. Individual outcomes and protocol failures are shown at the fixed 30-attempt horizon in Fig. 8a. The early-stage snapshot makes differences among models and repeats, particularly on Shields arylation, easier to distinguish before trajectories approach their final values.

The specialist-model comparison summarized in Fig. 8b shows that optimization performance is not captured by a single measure of model quality. Across both benchmarks, **GPT-5.4** combined strong optimization performance with high robustness, passing the full protocol in all six runs, whereas models such as **DeepSeek V4 Pro** tended to produce weaker campaigns and **Nemotron 3 Ultra** yielded comparatively few scientifically comparable trajectories. Differences between final quality and MBSF@60 further show that, even within the same BO infrastructure, specialist models can author campaigns with distinct convergence behaviour and evaluation efficiency. Because the specialist configures and validates the executable campaign rather than proposing candidates directly, these differences reflect how reliably and effectively each model translates a scientific task into an optimization procedure. The changing model ordering between Ackley 6D and Shields arylation nevertheless indicates that performance remains task dependent rather than universally ranked. Overall, most models produced valid campaigns under **Specialist-script**, while the observed failures show that optimization quality and operational robustness are distinct aspects of specialist performance. One further caveat applies specifically to **GPT-5.4**: the framework was developed with **GPT-5.4** in the specialist role, and the instructions, tool descriptions, and failure modes addressed during development were shaped by its initial failures. Its strong and robust performance may therefore partly reflect this development bias rather than a general capability advantage over the other specialists. Given the small and unequal number of comparable trajectories, these results should be interpreted as behavioural and architectural trends rather than as a model leaderboard.

B.1 Framework-comparison evaluation details

For Ackley 6D, each normalized coordinate $x_i \in [0, 1]$ was mapped to $z_i = -40 + 80x_i$. The conventional Ackley function was converted into the maximization score

$$q_{\text{Ackley}}(x) = 1 - \frac{f_{\text{Ackley}}(z)}{20 + e - e^{-1}},$$

which lies on a zero-to-one scale. The global optimum therefore corresponds to $q_{\text{Ackley}} = 1$.

B.1.1 Validity and aggregation criteria

Runs were assessed along separate validity dimensions. **Global-budget validity** required exactly 60 attempted objective evaluations across smoke tests, debugging attempts, and restarted campaigns. **Scientific comparability** additionally required valid objective values, the intended benchmark and backend, and a complete result-derived trajectory.

Architecture validity required compliance with each architecture’s delegation, tool-access, execution, and script-artifact rules. For **Specialist-script** specifically, the specialist had to author the campaign program and the main agent had to perform the production execution. **Full-protocol validity** required the global-budget, scientific-comparability, architecture, and artifact checks all to pass.

Equal-budget quality aggregates included only globally budget-valid, scientifically comparable runs. All requested outcomes were retained in the reliability analysis, including those excluded from quality aggregates.

B.1.2 Detailed architecture comparison

Ackley 6D clearly separated the proposed architecture from the three ablations (Fig. 7a). **Specialist-script** achieved the highest mean terminal quality and MBSF@60, substantially exceeding **Main-script**, **Local-BO**, and **Tool-loop**.

Main-script retained programmatic execution and BO-MCP campaign management but removed specialist delegation. On Ackley 6D, **Main-script** outperformed the other two ablations but remained below **Specialist-script**. Because both used BO-MCP and programmatic campaign execution, this difference is consistent with a benefit from specialist-led campaign planning. **Tool-loop** retained BO-MCP but required the main agent to mediate every step through tool calls. It produced the lowest mean Ackley 6D performance while consuming nearly as many resources as **Specialist-script**, showing that access to a structured optimizer service was not sufficient to recover the performance of the proposed architecture under repeated LLM-mediated execution. **Local-BO** benefited from programmatic execution but remained below **Specialist-script**; because it used an agent-authored local optimizer, this difference reflects the complete workflow change rather than an isolated campaign-management effect.

Across the observed Ackley 6D trajectories, the separation of **Specialist-script** became larger at later attempt horizons: **Main-script** was slightly ahead after 10 attempts, whereas **Specialist-script** led after 20 attempts and remained ahead through attempt 60. This pattern is consistent with the intended division of responsibilities, in which the specialist determines campaign strategy while repetitive execution is delegated to a reusable program (Fig. 7b).

The Shields arylation benchmark was less discriminating because the three BO-MCP-based architectures approached the top of the finite yield landscape. **Specialist-script** reached the maximum observed yield of 100% in all three runs. **Main-script** reached 100%, 100%, and 99.81%, producing nearly identical mean terminal performance.

Main-script nevertheless had a slightly higher mean MBSF@60, 0.891 compared with 0.865 for **Specialist-script**. The **Specialist-script** values were 0.984, 0.679, and 0.931, whereas **Main-script** produced 0.827, 0.915, and 0.932. The difference was driven by one comparatively slow but valid **Specialist-script** run, which first reached 90% yield at attempt 44 and 100% at attempt 56. We therefore interpret this benchmark as showing comparable sample efficiency between **Specialist-script** and **Main-script** with three repeats, with **Specialist-script** exhibiting perfect terminal success. The principal performance advantage of **Specialist-script** appears on Ackley 6D, where campaign-design choices had greater opportunity to affect the search.

All 24 architecture runs passed the full protocol, so the quality comparison does not depend on omitting unsuccessful architecture runs. **Specialist-script** used more resources than the ablations, consistent with the additional specialist interaction required for campaign construction and validation. On Ackley 6D, this additional inference coincided with substantially stronger final quality and sample efficiency.

Taken together, these results cannot be explained by optimizer access, programmatic execution, or inference cost alone. **Tool-loop** retained BO-MCP but did not recover strong Ackley 6D performance, whereas **Local-BO** retained programmatic execution at substantially lower cost but remained below the two script-based BO-MCP architectures. **Main-script** reduced resource use and performed comparably to **Specialist-script** on Shields arylation, but its lower Ackley 6D performance suggests that specialist-led campaign design becomes more valuable when optimizer configuration materially affects the search. Because these are end-to-end architectural ablations with three repeats rather than a factorial experiment, the comparisons should be interpreted as evidence about complete workflows rather than isolated causal estimates for any single architectural component.

C Supplementary material for the showcases

C.1 Phosphine ligand electronic tuning

This section documents a purely digital, multi-objective campaign in which LA AGENTE ÓPTIMA authored, executed, monitored, repaired and interpreted a finite-candidate BO campaign over monodentate phosphine ligands, using the GRÁFICO PySCF execution graph as the evaluator. The task is a ligand-level proxy for catalyst tuning inspired by the phosphine example of Laplaza et al. (48), and was chosen because it is the compact opposite of the hardware campaigns of Sec. C.5 and Sec. C.6: the search space is finite and enumerable, every evaluation is a deterministic quantum-chemical calculation, and the scientific interest lies entirely in whether the optimizer finds the trade-off structure of the objective set.

C.1.1 Task definition and division of labour

We tasked ÓPTIMA to search neutral phosphines $P(R^1)(R^2)(R^3)$ for an electronic profile consistent with a strong but not overly reducing donor for a hypothetical Ni-catalyzed cross-coupling. The operator fixed the substituent library (alkyl Me, Et, iPr, tBu, Cy; aryl Ph, pMePh, pOMePh; electron-poor aryl pFPh, pClPh, pCF₃Ph, pCNPh), the four objective transformations and their target tolerances, the electronic-structure protocol, and the budget (8 warm-start ligands, then 10 BO batches of 2). Everything else — symmetry reduction of the candidate table, the choice of warm-start ligands, the campaign intake, the evaluator, the reporting, and all plotting — was left to the agent. The brief explicitly forbade the agent from assembling or repairing molecular graphs during the loop and from using chemical judgment to pick candidates after the warm start.

Three components carried out the campaign. The *orchestrator* (main ÓPTIMA agent, gpt-5.5) held the operator conversation, delegated code authorship, launched and watched the campaign through its shell and monitor tools, and performed all analysis and plotting. A *bo-pyscf-specialist* subagent (gpt-5.5) authored the campaign package — a symmetry-reduced 364-row candidate table, the search space, intake, evaluator, reporting module, an executable entry point and a runbook (HOW_TO_EXECUTE_CAMPAIGN.md) — after first reading the BO-MCP OpenAPI description and validating its work with a compile check, a synthetic-evaluator end-to-end smoke test on a disposable campaign, and one real PySCF call on PH₃. The *BO-MCP service* owned all optimizer state, so that no BO mathematics was performed by an LLM; the campaign script holds no local optimizer state and pauses the campaign at the end of every invocation so that a later invocation can resume it by identifier. Ligand evaluation itself ran through the GRÁFICO PySCF graph, whose internal routing agent (gpt-4.1) selected the molecular-analysis node after the single-point calculation.

C.1.2 Campaign configuration

The candidate table enumerates all 364 unordered triples over the twelve substituents, each with a pre-validated SMILES string, and the BO search space is the single categorical parameter `candidate_id` — so every proposal is by construction a chemically valid, already-enumerated ligand. Tab. 6 lists the complete configuration. Four raw objectives are minimized jointly: the absolute error of the HOMO energy to -5.8 eV (donor strength), the absolute error of the HOMO–LUMO gap to 5.0 eV (electronic stability), the molecular volume in excess of $350 \text{ \AA}^3$ (steric bulk), and the heavy-atom count (molecular complexity). The phosphorus partial charge was recorded as an auxiliary donor descriptor with no optimization target of its own, as the brief requested. Where the PySCF workflow does not expose a requested property directly, the agent substituted a named proxy applied identically to every ligand and recorded the proxy name in the artifacts — an RDKit ETKDG/UFF volume and the Löwdin charge on phosphorus.

The agent selected the eight warm-start ligands itself and recorded a one-line rationale for each: PMe₃, PMe₂Ph, PMePh₂ and PPh₃ as an alkyl-to-aryl series, P(tBu)₂Ph and PCy₃ as bulky-alkyl probes, and P(pOMePh)₃ and P(pCF₃Ph)₃ as the electron-rich and electron-poor aryl boundaries. This is a deliberate bracket of the electronic axis rather than a space-filling design.

Table 6 Configuration of the phosphine electronic-tuning campaign, as recorded in the agent-authored campaign intake and runbook.

Setting	Value
Parameters	<code>candidate_id</code> (categorical, 364 symmetry-reduced ligands)
Objectives	<code>donor_homo_error</code> = $ E_{\text{HOMO}} + 5.8 \text{ eV} $ (tolerance 0.4 eV) <code>gap_error</code> = $ E_{\text{gap}} - 5.0 \text{ eV} $ (tolerance 1.5 eV) <code>steric_excess</code> = $\max(0, V - 350)$ in $\text{\AA}^3$ <code>heavy_atom_count</code>
Auxiliary descriptor	phosphorus Löwdin charge (tracked, not optimized)
Scalarization / acquisition	Pareto / hypervolume improvement
Batch size	2 ligands per BO suggestion request
Random seed	31841
Evaluator	GRÁFICO PySCF graph from SMILES, PBE/def2-SVP, neutral singlet, restricted, molecular analysis only (no frequencies, no TD-DFT)
Proxies	<code>rdkit_ETKDG_UFF_ComputeMolVolume</code> , <code>pyscf_lowdin_atomic_charge_on_P</code>
Budget	8 warm start + 20 BO (first invocation), + 20 BO (continuation)
End-of-run state	campaign paused (resumable, not terminated)

C.1.3 Execution and outcome

The campaign ran in two invocations against the same BO-MCP campaign (`f4e94d3d-0e06-43aa-baab-8bb15da9b843`) and the same cumulative artifact directory. All 48 evaluations succeeded; no candidate failed, no suggestion was rejected as a duplicate, and 48 of 364 candidates (13%) were ever computed.

After the first invocation the operator asked the agent how far to continue. The agent recommended 5 further batches (10 ligands), arguing from its own improvement curve that the hypervolume gain was flattening while new Pareto members were still appearing, and proposed an explicit stopping rule (halt when the normalized hypervolume improves by less than $\sim 1\%$ relative and no new ligand lands near the target region). The operator overrode the recommendation and requested 20 further evaluations, which the agent executed.

Fig. 9 shows the improvement curve the agent produced from the evaluation records. The normalized dominated hypervolume rises from 0.791 at the end of the warm start to 1.038 after 48 evaluations, and the observed Pareto front grows from 3 to 17 members, 14 of them discovered after the warm start. The second invocation contributed most of the front’s late growth but only $\sim 7\%$ of the hypervolume, confirming the diminishing-returns regime the agent had diagnosed before it was asked to continue.

The chemistry the front encodes is a genuine conflict between the two electronic targets. No ligand in the evaluated set satisfies both tolerances: within this family, pulling the HOMO down to -5.8 eV requires electron-poor aryl substituents, whose low-lying π^* orbitals simultaneously collapse the gap to $\sim 3 \text{ eV}$, while the trialkyl phosphines that keep a $5\text{--}6 \text{ eV}$ gap sit $0.8\text{--}0.9 \text{ eV}$ too shallow. The two warm-start boundary ligands therefore already hold the extreme positions — $\text{P}(\text{pCF}_3\text{Ph})_3$ gives the best HOMO error of the entire campaign (0.085 eV) and PCy_3 the best gap error (0.274 eV) — and BO did not beat either on its own axis. What it did instead is the point of a multi-objective run: it found cheaper versions of the same electronics. $\text{P}(\text{iPr})(\text{pCF}_3\text{Ph})_2$ reproduces the HOMO error of $\text{P}(\text{pCF}_3\text{Ph})_3$ to within 0.03 eV while improving the gap error, removing seven heavy atoms and $55 \text{ \AA}^3$ of volume, and $\text{P}(\text{tBu})(\text{Cy})_2$ matches the gap error of PCy_3 with two fewer heavy atoms. The resulting front spans 4 to 31 heavy atoms and is populated overwhelmingly by

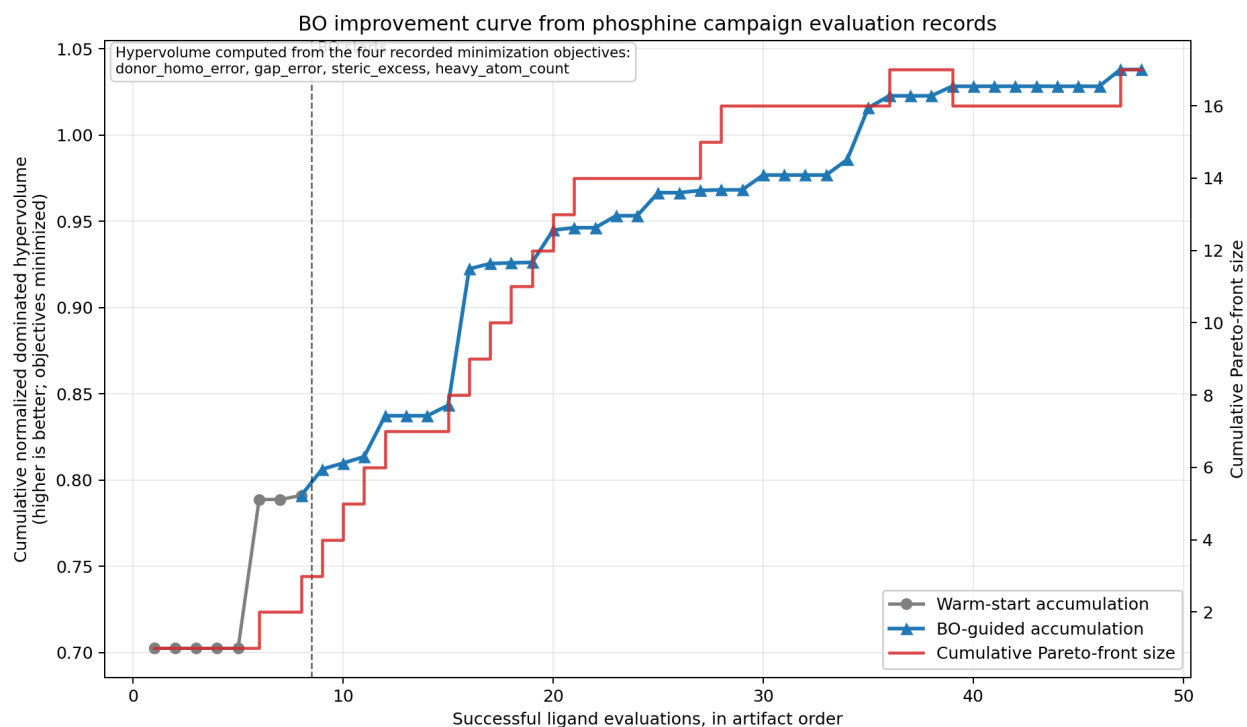


Figure 9 Multi-objective improvement curve for the phosphine electronic-tuning campaign, as generated by ÓPTIMA from `evaluation_records.jsonl` (axis labels and layout unmodified). Grey markers are the eight script-selected warm-start ligands, blue markers the 40 BO-guided evaluations, and the dashed line marks the transition. The left axis shows the cumulative dominated hypervolume over the four minimized objectives, normalized against the range spanned by the complete record set; the right axis shows the size of the Pareto front observed so far, which is not monotone because a later ligand can dominate several earlier front members at once.

mixed methyl/alkyl ligands at the compact end and mixed alkyl/electron-poor-aryl ligands at the electronic end.

One objective turned out to be nearly inert. The steric term is non-zero for exactly one of the 48 ligands ($\text{P}(\text{pCF}_3\text{Ph})_3$, at $2.2 \text{ \AA}^3$ above the $350 \text{ \AA}^3$ threshold), because the operator-specified threshold lies above almost the entire reachable volume range of this substituent library. The campaign is therefore effectively three-objective, with heavy-atom count carrying the size preference on its own. This is a property of the brief rather than a failure of the optimizer, but it illustrates a practical hazard of specifying hinge-shaped objectives before the descriptor range is known.

C.1.4 Failure mode and autonomous repair

The campaign produced no chemical or optimizer failures, only one infrastructure defect. The progress monitor of the first invocation went silent after four lines although the run completed normally; asked why, the agent traced this to its own evaluator, which wrapped each PySCF call in `contextlib.redirect_stdout` — a process-global redirect that, with two evaluator threads, captured the main thread’s tagged progress lines into an in-memory buffer. It added a lock around the redirected section, ran the continuation single-threaded, and, at the operator’s prompting, had the *bo-pyscf-specialist* subagent write the lesson to its persistent memory for future campaign packages.

C.1.5 Resource usage and provenance

The session spanned 38 agent turns over 3 h 11 min of wall-clock time, of which 14 were operator messages and the remainder autonomous responses to monitor events. It comprised 128 LLM calls — 71 by the main agent, 56 across the two *bo-pyscf-specialist* runs (both gpt-5.5), and one gpt-4.1 routing call inside the PySCF graph — consuming 6.18M input tokens (5.62M served from cache) and 50.4k output tokens, of which 11.0k were reasoning tokens, at a total cost of \$7.08 (\$3.80 orchestrator, \$3.28 specialists). Prompt caching accounts for a factor of 5.5 on the input side: the same 6.18M input tokens billed at the uncached rate would have cost \$30.9 rather than \$5.57. The main agent made 33 tool calls (19 shell executions, 6 monitor queries, 2 subagent delegations with 2 waits, 2 monitor starts and 2 stops); the specialist runs made 54 in aggregate, dominated by file writes, code execution for validation, BO-MCP interface inspection and the two memory writes. The 48 production DFT evaluations were performed by the campaign script rather than by the agent and consumed only 15.8 min of cumulative PySCF wall-clock time (median 18.8 s per ligand), yet the two invocations occupied 78 min of campaign time in total, the difference being suggestion generation and a fixed inter-batch pacing interval in the campaign script rather than computation.

The campaign workspace preserves the optimization itself — the candidate table, the campaign package, the warm-start rationale, the per-evaluation records with descriptors, objectives and proxy names, the BO-MCP campaign export, the reports and the plotting scripts. At the end of the session the agent collected all of these, together with the monitor logs and a SHA-256 manifest, into a single reproducibility bundle.

C.2 Singlet-triplet gap search over the Pollice 2021 library

This section documents a single-objective, fixed-library BO campaign in which LA AGENTE ÓPTIMA was asked to find the molecule with the smallest TD-DFT singlet-triplet gap in the INVEST candidate set of Pollice et al. (49). It complements the phosphine campaign of Sec. C.1 in three ways: the search space is two orders of magnitude larger (1512 molecules rather than 364), each evaluation is a conformer search followed by an excited-state calculation rather than a single ground-state job, and the underlying dataset carries published reference values, so the quality of the agent’s cheap digital proxy can be measured rather than merely asserted. It is also the campaign in which the infrastructure failed hardest, which makes it the more informative record of how the agent behaves when its evaluator stops working.

C.2.1 Task definition and division of labour

The operator supplied the candidate table, the objective and the evaluation protocol in outline, and left the concrete choices to the agent. For each molecule selected by the optimizer, ÓPTIMA was to take `smiles_canonical` as the only structural input, generate a small set of low-energy ground-state conformers, keep the lowest-energy one, run a fixed TD-DFT single point on it, and report $\Delta E_{\text{ST}} = E(S_1) - E(T_1)$ with the BO objective defined as $-\Delta E_{\text{ST}}$. Everything else — the electronic-structure method, the candidate filter, the molecular representation, the campaign package and all reporting — was left to the agent, subject to operator review before execution.

The interactive design phase is worth recording because it shows the operator and the agent converging on a protocol rather than the agent executing a specification. Asked which functional it would use, ÓPTIMA proposed ω B97X-D/def2-SVP on charge-transfer grounds and named PBE0/def2-SVP as the cheaper fallback; asked about molecule size, it profiled the table with RDKit and reported a median of 26 and a maximum of 85 heavy atoms; when the operator asked to drop the largest decile it derived the tie-inclusive cutoff `heavy_atoms < 56` itself, leaving 1512 of 1708 candidates. Instructed to time one evaluation before committing to a budget, it ran a median-sized molecule end to end and estimated 2.5 min, of which the TD-DFT step was inferred from file timestamps as ~ 30 s; when the operator corrected this from the server logs to ~ 11 min, the agent accepted the operator’s number as the better source, revised the campaign estimate from 5.4 h to 6–15 h, and switched its own recommendation to the cheaper PBE0 fallback and a deliberately small budget. That exchange, rather than the optimization, is the reason the campaign was affordable at all.

Three components then carried out the campaign, as in Sec. C.1. The *orchestrator* (main ÓPTIMA agent, gpt-5.5) held the operator conversation, reviewed and launched the campaign, watched it through its monitor tools, and performed all diagnosis, analysis and plotting. A *bo-pyscf-specialist* subagent (gpt-5.5) authored the campaign package — search space, intake, evaluator, campaign loop, entry point and runbook — in a single delegated run. The *BO-MCP service* held all optimizer state, and the campaign script pauses the campaign at the end of every invocation so that a later invocation resumes it by identifier; the GRÁFICO PySCF graph, with its internal gpt-4.1 routing agent, performed the excited-state calculations.

C.2.2 Campaign configuration

Tab. 7 lists the configuration as recorded in the agent-authored intake. The search variable is the single categorical parameter `molecule_key`, so every suggestion is by construction a molecule that exists in the table. Its BayBE custom descriptors are built by the agent from the SMILES string alone: twelve standardized RDKit 2D scalars, 32 truncated-SVD components of a 2048-bit Morgan/ECFP4 fingerprint, and a deterministic identity code that keeps otherwise indistinguishable rows separable. Evaluation failures are never submitted as observations — the corresponding suggestion is explicitly rejected in BO-MCP — so the optimizer’s posterior is never contaminated by an infrastructure fault.

C.2.3 Execution and outcome

The campaign ran in three invocations against the same BO-MCP campaign (f023cf90-a1a1-470a-987d-134a38919812) and the same cumulative artifact directory, with the operator setting each continuation budget. Of 44 attempted evaluations, 39 succeeded and 5 failed; all 39 successes are distinct molecules, so 2.6% of the filtered pool was ever computed. The best molecule is a methylsulfinyl-substituted cycl[3.3.3]azine

Table 7 Configuration of the Pollice 2021 singlet-triplet gap campaign, as recorded in the agent-authored campaign intake and runbook.

Setting	Value
Candidate pool	1708 tabulated molecules, filtered to heavy_atoms < 56 (1512 kept)
Parameter	molecule_key (categorical, 1512 levels)
Descriptors	12 standardized RDKit 2D scalars, 32 SVD components of a 2048-bit Morgan/ECFP4 fingerprint, and an identity code
Objective	negative_singlet_triplet_gap = $-(E_{S_1} - E_{T_1})$, maximized
Backend	BayBE through BO-MCP
Initial design	5 molecules (seeded random, not curated)
Batch size	2 molecules per BO suggestion request
Random seed	2021
Conformers	CREST/GFN2-xTB imtd-gc from SMILES, lowest-energy conformer only
Electronic structure	GRÁFICO PySCF graph, restricted PBE0/def2-SVP gas-phase TD-DFT, 5 states, neutral singlet, no solvation, no geometry optimization
Timeouts	7200 s per evaluation, 5400 s per PySCF workflow
Budget	10 BO loops, +10 (continuation), +20 requested (halted early)
End-of-run state	campaign paused (resumable, not terminated)

(WGKMZGAJDYWUCE-UHFFFAOYSA-N) with $\Delta E_{\text{ST}} = 0.223$ eV, $E(S_1) = 1.398$ eV, $E(T_1) = 1.175$ eV and an oscillator strength of 2.3×10^{-4} .

Fig. 10 shows the improvement curve the agent produced. The best-so-far trace is almost flat, and honestly so: the first molecule of the random initial design was already a cyclazine at 0.227 eV, and only the 37th success improved on it, by 3.6 meV. The optimizer’s contribution is therefore not visible in the record trace but in where it spent the budget. Of the 34 BO-guided evaluations, 31 fall in the subset of the source table that carries at least one inverted-gap reference record, against a base rate of 42% in the pool, and 21 of 34 landed below 0.30 eV; the mean observed gap fell from 0.429 eV over the five initial-design molecules to 0.302 eV over the BO-guided ones. Nine of the ten best molecules are monosubstituted cycl[3.3.3]azines and the tenth is a triazine-fused heptazine analogue, so the optimizer recovered the two scaffold families that motivate the dataset from a 1512-molecule table and 2048-bit fingerprint descriptors, without any chemical prior.

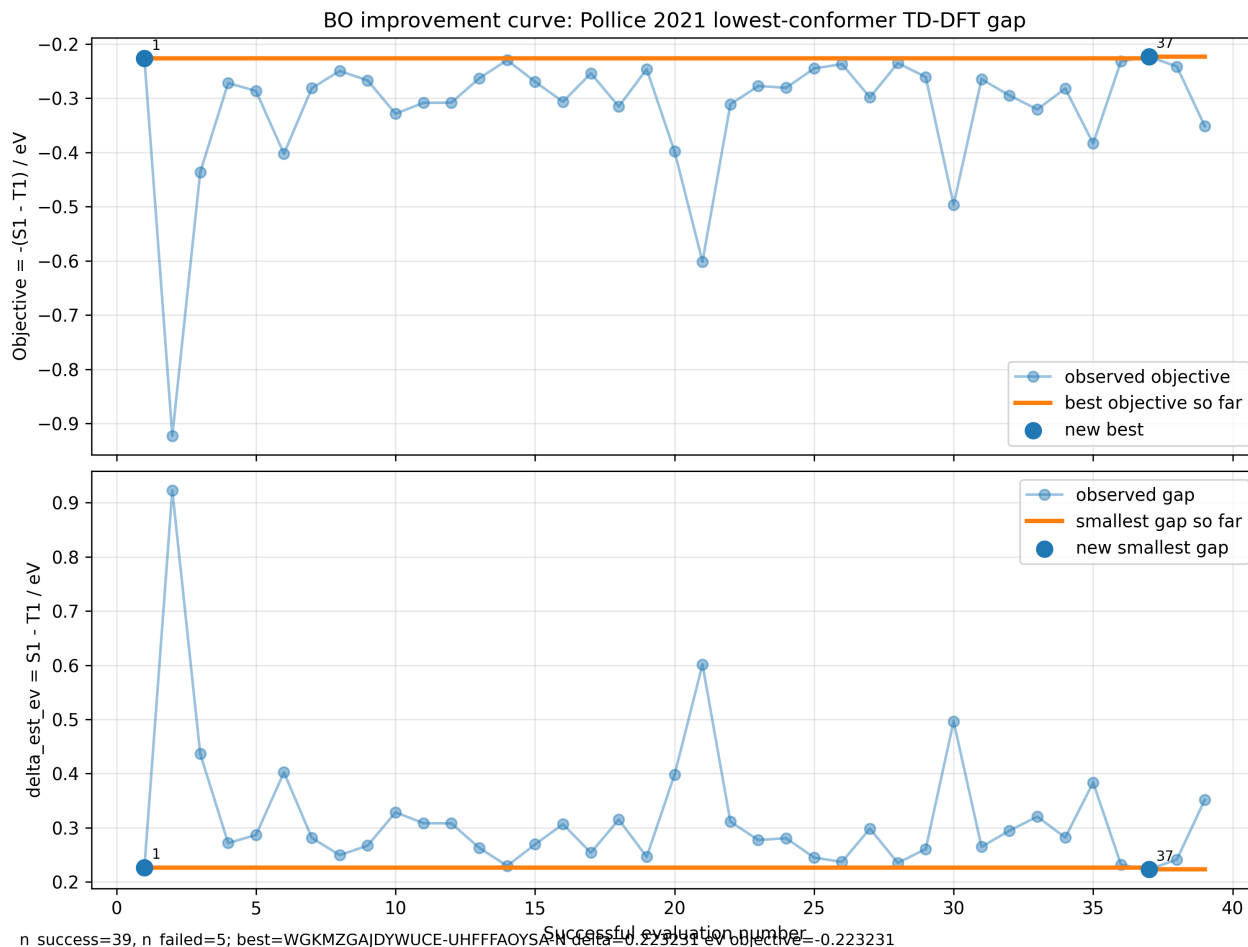


Figure 10 Improvement curve for the Pollice 2021 singlet-triplet gap campaign, as generated by ÓPTIMA from `evaluation_results.csv` (axis labels, annotations and layout unmodified). The two panels carry the same information with opposite sign, since the maximized objective is the negated gap; markers are the 39 successful evaluations in execution order, the step trace is the record so far, and the filled circles are the two records set. The five failed evaluations are omitted from the axis, as they never became observations.

The campaign also quantifies the limits of its own evaluator, which is the reason for choosing a dataset with published reference values. No evaluation returned an inverted gap: the smallest value reached is +0.223 eV, whereas the aggregated reference values shipped with the table put the same top molecules near -0.34 eV. Across the 39 evaluated molecules the campaign’s PBE0/def2-SVP gaps are offset from the reference values by +0.33 eV on average but track them well in rank (Spearman $\rho = 0.79$, Pearson $r = 0.89$). This is the expected behaviour of adiabatic linear-response TD-DFT for these systems, whose gap inversion requires the double-excitation character that a single-reference linear-response treatment omits, and it is a useful reminder

that a cheap digital evaluator can be a good ranking device and a bad absolute one at the same time. An agent-run campaign does not change that; what it does change is that the discrepancy is reconstructable from the preserved artifacts.

C.2.4 Failure modes and autonomous response

Five evaluations failed, in two distinct modes, and neither was a chemistry failure. One was a genuine cost overrun: a 47-heavy-atom triazine-fused system with two piperidiny-aminophenyl arms exceeded the 7200 s per-evaluation ceiling and was killed and rejected. Asked afterwards whether the timeout should be raised, the agent laid out the trade-off and the operator elected to keep 7200 s, so the ceiling stayed a deliberate budget rather than an accident.

The other four were an infrastructure fault, and the agent’s handling of it is the interesting part. In the third invocation, four consecutive evaluations failed inside `run_pyscf_workflow`. After the third, the agent stated that this was a repeated pattern rather than a one-off and pre-committed to a stopping rule; when the fourth failed it created the campaign’s `STOP` marker itself, so the loop halted before the next suggestion request, exported its artifacts and paused the campaign cleanly rather than being killed mid-batch. When the operator later asked for a direct single-point calculation on water — a minimal reproducer — the same fault appeared in the foreground: `gpu4pyscf` and `cupy` were importable in the container but no CUDA device was present, so the mean-field object’s `.to_gpu()` call raised `cudaErrorNoDevice` instead of falling back. The agent read the relevant graph source, patched `PyscfInput.pyscf_mf` to catch the failure and continue on CPU, reran the water calculation successfully (-76.276266 Hartree), and flagged in the exported bundle that this patch is a host-environment fix outside the campaign package and must be reapplied or avoided on reproduction.

C.2.5 Resource usage and provenance

The session comprised 24 operator messages and 89 autonomous responses to monitor events, and 34 tool calls by the main agent (21 shell executions, 3 monitor starts with 3 status queries, 3 direct PySCF workflow calls, 1 CREST call, 1 unit conversion, and 1 subagent delegation with its wait). It consumed 191 gpt-5.5 calls — 136 by the orchestrator and 55 in the single *bo-pyscf-specialist* run — plus 88 one-shot gpt-4.1 routing calls inside the PySCF graph, for 13.1M input tokens (12.1M served from cache) and 76.5k output tokens, of which 10.9k were reasoning tokens, at a total cost of \$13.01 (\$8.79 orchestrator, \$4.04 specialist, \$0.18 routing). Prompt caching accounts for a factor of 5.2 on the gpt-5.5 side: the same traffic billed at the uncached rate would have cost \$67.3 rather than \$12.8. The 44 attempted evaluations consumed 7.1 h of cumulative evaluator wall-clock time — 4.98 h across the 39 successes, of which 3.91 h was PySCF and 1.07 h CREST, with a median of 6.5 min per molecule — and, because the campaign script evaluates a batch of two concurrently, the three invocations occupied 6 h 15 min of campaign time in total.

At the operator’s request the agent assembled a single reproduction bundle containing the input table, the campaign package, the intake payload, the per-evaluation records and BO-MCP export, the three monitor logs, the ranked results, the plotting script with its regenerated figure, and a metadata folder recording the software environment, the campaign snapshot and the out-of-package CPU-fallback patch.

C.3 Agent-to-agent cobalt catalyst tuning

This section documents a digital, multi-objective campaign on cationic Co(II) bisphosphine complexes in which LA AGENTE ÓPTIMA owned neither the molecular structures nor the optimizer state. Molecular construction was delegated over the A2A interface to EL AGENTE ESTRUCTURAL, a separately deployed agent (Sec. 2.4); optimizer state lived in BO-MCP; and evaluation was an unrestricted density functional theory (DFT) geometry optimization on the GRÁFICO PySCF execution graph. The campaign is the transition-metal counterpart to the ligand-only phosphine campaign of Sec. C.1: the search space is again finite and enumerable, but each evaluation now requires a metal complex to be built from a textual ligand description before any electronic structure can be computed.

C.3.1 Task definition and division of labour

Taking the cationic Co(II) hydroformylation catalysts of Hood et al. (50) as the starting point, we tasked ÓPTIMA to search bidentate bisphosphines R_2^1P -linker- PR_2^2 for a Co(II) precursor-like complex balancing electronic activation, coordination stability, chelate geometry and steric crowding. The operator fixed the four linkers (ethylene, propylene, 1,2-phenylene, cis-1,2-cyclohexylene), the eight substituents (Me, Et, iPr, Cy, Ph, *p*-Tol, *p*-Anisyl, *p*-CF₃-Ph), the four objective descriptions with their suggested proxies, the feasibility criteria, the complex to be built ($[Co(acac)(P_2)]^+$), the restriction to a modest geometry optimization without transition states, frequencies, TD-DFT or catalytic-cycle calculations, and the budget (4 warm-start candidates, then 10 BO iterations). Everything else — construction and symmetry reduction of the candidate table, the descriptor representation, the warm-start selection, the campaign code, the objective scoring functions, the reporting and all plotting — was left to the agent. The operator also required the complete candidate table to be reported and audited for duplicate unordered R^1/R^2 permutations before any calculation was allowed to start.

Four components carried out the campaign. The *orchestrator* (main ÓPTIMA agent, gpt-5.5) held the operator conversation, delegated code authorship, launched and watched the campaign through its shell and monitor tools, and performed all analysis and plotting. A *bo-pyscf-specialist* subagent (gpt-5.5) authored the campaign package — the 144-row candidate library, the descriptor vectors, the campaign intake, the evaluator, the reporting module, an executable entry point and a runbook (HOW_TO_EXECUTE_CAMPAIGN.md). The *BO-MCP service* owned all optimizer state, so no BO mathematics was performed by an LLM, and the campaign script pauses the campaign at the end of every invocation so a later invocation can resume it by identifier. Finally, EL AGENTE ESTRUCTURAL owned molecular construction: for every candidate the campaign script issued an A2A task carrying only the ligand label and a natural-language connectivity description, and EL AGENTE ESTRUCTURAL answered with an XYZ file written into the room-scoped workspace, using its own structure-generation and structure-editing tools. Ligand evaluation then ran through the GRÁFICO PySCF graph, whose internal routing agent (gpt-4.1) selected the molecular-analysis node after the geometry optimization.

The division is worth stating explicitly because no single component held the whole problem: ÓPTIMA never manipulated atomic coordinates, EL AGENTE ESTRUCTURAL never saw the objectives or the optimizer, and BO-MCP never saw a molecule. The A2A contract between the first two is a plain-text request and a filename; coordinates travel through the shared workspace and never enter an LLM context.

C.3.2 Campaign configuration

The candidate table enumerates all unordered substituent pairs per linker, giving 36 candidates for each of the four linkers, 144 in total, of which 32 are symmetric ($R^1 = R^2$) and 112 unsymmetric, with no residual duplicate permutations — a report the script regenerates and prints on every invocation before any calculation begins. As in the phosphine campaign, the BO search space is the single categorical parameter `candidate_id`, so every proposal is by construction an already-enumerated ligand; the agent attached a nine-component custom descriptor vector to each category (linker size and rigidity, mean and difference of substituent steric bulk, aryl fraction, mean and difference of substituent electronics, a symmetry flag, and an identity code), so the surrogate model can exploit ligand similarity rather than treating the identifiers as unrelated labels. Tab. 8 lists the complete configuration.

Two properties of that configuration matter for the interpretation below. First, the electronic-activation objective is parsed from the PySCF checkpoint file rather than from the summary text, using the spin-resolved frontier orbitals, the Mulliken charge and the Mulliken spin population at cobalt, and taking the α SOMO for the unrestricted doublet; the per-evaluation record stores the frontier energies in both Hartree and eV together with the score components and their provenance. Second, an evaluation counts as successful only if the workflow summary explicitly reports a completed geometry optimization and contains no failure indicator, so a converged self-consistent field (SCF) with an unconverged relaxation is treated as infeasible rather than scored.

Table 8 Configuration of the agent-to-agent cobalt bisphosphine campaign, as recorded in the agent-authored campaign intake and runbook.

Setting	Value
Parameters	candidate_id (categorical, 144 symmetry-reduced ligands, 9 custom descriptors per category)
Objectives	electronic_activation (maximize; frontier-orbital energy, Co charge, Co spin population, parsed from the PySCF chkfile) coordination_stability (maximize; Co–P range, Co–P asymmetry, acac O,O-coordination, no dissociation) chelate_geometry (maximize; P–Co–P bite angle, square-planar distortion) steric_crowding (minimize; heavy atoms near Co, nonbonded contacts)
Scalarization / acquisition	Pareto / hypervolume improvement
Batch size	1 ligand per BO suggestion request
Random seed	2020
Structure generation	EL AGENTE ESTRUCTURAL over A2A, one task per candidate, room-scoped context_id , XYZ returned through the shared workspace
Evaluator	GRÁFICO PySCF graph from literal XYZ, charge +1, doublet, unrestricted PBE/def2-SVP, geometry optimization to convergence (max. 200 steps, 7200 s timeout), then molecular/electronic analysis
Feasibility criterion	explicit geometry-optimization completion, intact CoP ₂ O ₂ core, no dissociation or severe collapse
Infeasibility handling	finite hard penalty (∓ 100 per objective) submitted as an observation
Budget	4 warm start + 10 BO
End-of-run state	campaign paused (resumable, not terminated)

The agent selected the four warm-start candidates itself to bracket the design space rather than to fill it: **eth__Me__Me** (smallest symmetric alkyl case), **prop__iPr__Ph** (mixed alkyl/aryl on the flexible propylene linker), **ophen__pAnisyl__pCF3Ph** (rigid 1,2-phenylene with an electron-rich/electron-poor aryl contrast) and **cchex__Cy__pTol** (bulky cycloalkyl/aryl on the cis-cyclohexylene linker).

C.3.3 Execution and outcome

The campaign (62fb243b-265e-4ba4-b5a8-d97e414fce2f) ran in a single invocation of 6 h 45 min and submitted all 14 planned observations. Six were feasible and eight received the hard penalty. All six feasible candidates carry the ethylene linker with Me, Et or iPr substituents; every propylene, 1,2-phenylene and cis-1,2-cyclohexylene candidate failed, as did the three ethylene candidates carrying cyclohexyl or *p*-CF₃-phenyl groups. Seven of the eight failures are unconverged geometry optimizations — the SCF converged in every one of them, so the failure is in the relaxation, not the electronic structure — and the eighth (**eth__pCF3Ph__pCF3Ph**) exceeded the 7200 s workflow timeout. Failure did not track molecular size alone: **prop__Me__Me** failed at 42 atoms while **eth__iPr__iPr** converged at 63, so the flexible and rigid linkers are harder to relax than their atom count suggests.

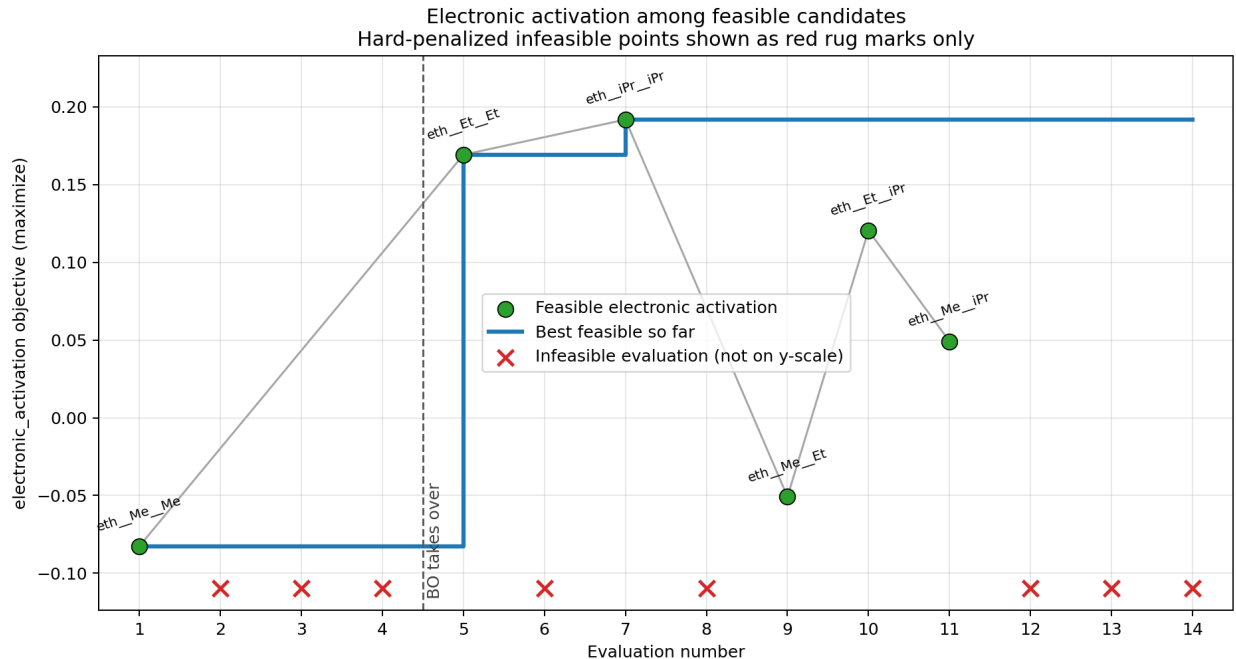


Figure 11 Electronic-activation objective over the course of the cobalt bisphosphine campaign, plotted by the reproducible script `OPTIMA` wrote for this purpose (axis labels and layout unmodified). Green markers are the six feasible evaluations with the candidate identifier annotated, the blue step function is the best feasible value so far, and the red rug marks at the bottom are the eight hard-penalized infeasible evaluations, which are kept off the *y*-scale so that the decimal-scale differences among feasible candidates remain visible. The dashed line marks the transition from the four agent-selected warm-start candidates to the BO-selected iterations.

Within the feasible subset the optimization behaved as intended. The normalized feasible-only dominated hypervolume rises from 0.572 after the single feasible warm start to 1.000 after the sixth feasible evaluation, with four of the six feasible candidates on the observed Pareto front, and the best electronic activation improves from -0.083 (**eth__Me__Me**) to 0.192 (**eth__iPr__iPr**); see Fig. 11. The front also reproduces, on a much smaller sample, the behaviour that made the phosphine campaign of Sec. C.1 instructive: the best electronic activation sits at the highest steric-crowding score of the set, while **eth__Et__Et** reaches 0.169 — within 12% of it — at the lowest crowding score, so the optimizer again found a less bulky ligand at essentially the same electronics rather than a single dominant winner. All six feasible complexes retain the intended four-coordinate CoP_2O_2 core, with Co–P distances of 2.19–2.22 Å, Co–O of 1.90–1.91 Å, P–Co–P bite angles of 87–88° and Co–P asymmetries below 0.04 Å.

The more informative outcome is how BO responded to the feasibility structure. Only one of the four bracketing warm starts was feasible, so after the warm start the optimizer had a single positive example, all of it on the ethylene linker. It then spent 8 of its 10 suggestions on ethylene candidates although ethylene is only a quarter of the library, which is the correct response to a descriptor space in which one linker family

carries all the observed signal, and it still probed the two remaining propylene suggestions and the bulkier ethylene substituents, both of which failed. The campaign is therefore best read as a demonstration that the delegated loop is sound while the evaluator protocol is the binding constraint: at a 43% feasibility rate the effective budget was 6 rather than 14 evaluations, which is too small to resolve a four-objective front over 144 candidates. Raising the feasible fraction is a question of the geometry-optimization protocol and of the quality of the delegated starting structures, not of the optimizer or of the ligand chemistry.

C.3.4 Resource usage and provenance

The session spanned 111 agent turns across 4 d 23 h calendar time, of which 36 were operator messages and the remainder autonomous responses to monitor events; cumulative agent-run wall-clock time was 1.79 h. It comprised 557 LLM calls with recorded token counts: 472 gpt-5.5 calls on the `ÓPTIMA` side (main agent plus `bo-pyscf-specialist` delegations), 2 gpt-4.1 routing calls inside the PySCF graph, and 83 gpt-5.5 calls inside the 41 `EL AGENTE ESTRUCTURAL` A2A tasks. Together these consumed 56.1M input and 210k output tokens at a total cost of \$53.29, split \$38.71 on the `ÓPTIMA` side and \$14.58 on the `EL AGENTE ESTRUCTURAL` side. The main agent made 94 tool calls, as counted from the preserved conversation export; this excludes tool use internal to the specialist and `EL AGENTE ESTRUCTURAL` conversations. Prompt caching accounts for a factor of 6.0 on the input side: the same 56.1M input tokens billed entirely at the uncached rate would have cost \$281 rather than \$47. The caching benefit is very unevenly distributed — the per-call costs imply that ~96% of the orchestrator’s input tokens were served from cache against ~46% of `EL AGENTE ESTRUCTURAL`’s — because each A2A task is a short, fresh conversation with little prefix to reuse, whereas the orchestrator replays one long and growing history. This is the practical cost of delegating structure generation to a separate agent: `EL AGENTE ESTRUCTURAL` handled 8% of the input tokens but 27% of the spend.

The 14 production evaluations were performed by the campaign script rather than by the agent. They consumed 4.87 h of cumulative GPU wall-clock time in the PySCF graph (median 17.1 min, maximum 60.6 min per candidate), against 19.3 min of cumulative `EL AGENTE ESTRUCTURAL` task time for all 41 structure requests — so structure generation cost roughly 6% of the electronic-structure budget, and the failed relaxations, not the delegation, dominate the campaign’s cost.

The campaign workspace preserves the candidate library, the agent-authored campaign package and runbook, the warm-start selection with per-candidate rationale, the per-candidate `EL AGENTE ESTRUCTURAL` task responses and generated XYZ files, the PySCF consoles and optimized geometries, the per-evaluation records with descriptors and objective components, the BO-MCP campaign export, and the plotting scripts.

C.4 Xe/Kr separation over PORMAKE-assembled MOFs

This section documents a two-stage, multi-objective campaign in which LA AGENTE ÓPTIMA designed MOFs for a Xe/Kr separation proxy, assembling each candidate with PORMAKE (53) and scoring it with Zeo++ (54). It is the only campaign in this series whose evaluator is purely geometric rather than quantum-chemical, which makes it the cleanest test of the agentic loop itself: an evaluation costs a few seconds, so nothing in the record is hidden behind expensive chemistry. It is also the campaign in which the agent’s first search-space parameterization was wrong in a way that is specific to combinatorial materials spaces, and in which the agent diagnosed and replaced that parameterization itself.

C.4.1 Task definition and division of labour

We tasked ÓPTIMA to find MOFs balancing Xe/Kr selectivity against capacity, representing every candidate exactly as a PORMAKE topology plus one node building block plus one edge building block. The brief was adapted from the inverse-design study of Lim et al. (52), who tuned MOFs to a user-specified Xe/Kr selectivity obtained from grand-canonical Monte Carlo simulations. Because no adsorption simulation is performed here, that objective was replaced by a geometric selectivity proxy computable with Zeo++, in the spirit of pore-limiting-diameter pre-screens used to narrow MOF libraries before adsorption calculations (51). The operator fixed the nine admissible topologies (`pcu`, `dia`, `rtl`, `ths`, `bcu`, `srs`, `nbo`, `tbo`, `pts`), the qualitative objectives (a pore size suitable for Xe/Kr separation; pore volume as large as possible), the tools (PORMAKE and Zeo++) and a total budget of 30 evaluations, and explicitly left the batch size, iteration count and initial design size to the agent. Everything else was left to the agent: the search over the PORMAKE database for usable node and edge building blocks, the functional form of both objective proxies, the campaign code, the reporting and all plotting.

The division of labour follows the phosphine campaign of Sec. C.1. The *orchestrator* (main ÓPTIMA agent, gpt-5.5) held the operator conversation, delegated code authorship, launched and watched both campaigns through its shell and monitor tools, and performed all analysis and plotting. A *bo-pyscf-specialist* subagent (gpt-5.5), which also owns MOF work, authored the campaign package (search-space construction, campaign intake, evaluator, reporting module, an executable entry point and a runbook) and validated it with a compile check, a bounded one-evaluation BO smoke test against a disposable campaign, and a direct PORMAKE/Zeo++ construction check before any production run. The *BO-MCP service* owned all optimizer state, so no BO mathematics was performed by an LLM. No GRÁFICO PySCF graph was involved: the evaluator calls PORMAKE and Zeo++ directly.

C.4.2 Campaign configuration

Rather than accepting the nine requested topologies as given, the agent’s script inspects the installed PORMAKE database at runtime and excludes `rtl` and `tbo` with a tagged [ALERT] line, because neither can be represented by a single node building block; it then ranks the compatible node building blocks per topology by local-structure root mean square deviation (RMSD) (six retained each) and selects ten edge building blocks spread over the available edge lengths (1.14–8.47 Å). Both objectives are maximized: `selectivity_proxy` is a bounded score that saturates at unity for a pore-limiting diameter in the 3.6–7.0 Å window bracketing the Xe and Kr kinetic diameters and decays outside it, damped by a penalty on largest-cavity diameters above 12 Å, while `capacity_proxy` is the Zeo++ pore volume in cm³/g. Tab. 9 lists both campaign configurations.

C.4.3 The first campaign and its parameterization failure

The first campaign exposed a defect that only appears in this class of search space. Representing the candidate as three independent categorical parameters, exactly as the brief phrased it, makes the nominal space $7 \times 40 \times 10 = 2800$ combinations, but a node building block can only serve a topology whose vertices have its connectivity, so only $42 \times 10 = 420$ triples, 15% of the nominal product, are constructible at all. BayBE has no way to know this, and 15 of the 30 evaluations were spent on incompatible topology–node pairs, which the evaluator rejected before any construction and submitted as zeros. The optimizer nevertheless learned the constraint from those penalized observations: only one of the nine initial-design points was constructible,

Table 9 Configuration of the two Xe/Kr MOF campaigns, as recorded in the agent-authored campaign intakes and runbooks. Settings shared by both campaigns are given once.

Setting	First campaign	Refined follow-up
Parameters	topology (7 categories) node (40), edge (10)	candidate_id (109 categories, decoding to topology node edge)
Nominal / valid space	2800 / 420	109 / 109
Objectives	selectivity_proxy (maximize; pore-diameter window, bounds [0, 1], weight 0.6) capacity_proxy (maximize; pore volume, bounds [0, 10] cm ³ /g, weight 0.4)	
Scalarization / acquisition	BayBE desirability, weighted geometric mean	
Batch size	3	5
Initial design size	9	15 historical seed rows
Budget (max_observations)	30	15 + 50 = 65
Random seed	20260812	
Evaluator	PORMAKE build_by_type from topology/node/edge, then Zeo++ pore diameter and pore volume; CIF written per successful candidate	
Infeasibility handling	zero on both objectives, submitted as an observation	
End-of-run state	campaign paused (resumable, not terminated)	

against 14 of the 21 BO-selected points, and 18 of those 21 went to **pcu**, the topology carrying all observed signal. All 15 successful candidates are **pcu**.

Asked by the operator whether it made sense to continue, ÓPTIMA declined to simply resume. It reported the 50% invalid rate as the binding problem, noted that the incumbent had not improved since evaluation 12 and that all ten retained edges had already been paired with the best node, and offered three options: a refined second campaign restricted to validated compatible triples and seeded with the existing results, a short unchanged continuation as a sanity check, or a local screen around the incumbent. It recommended the first, and the operator selected it.

C.4.4 The refined campaign and outcome

The refined follow-up reuses the evaluator unchanged and replaces only the search-space concern. The single BO parameter is a finite **candidate_id** that decodes to **topology|node|edge**, enumerated over the validated **pcu** family, its six compatible nodes, and 19 edges obtained by expanding the original ten with near-length neighbours from the PORMAKE database. That gives 109 triples, every one of them constructible. The 15 successful prior evaluations were submitted as historical seed rows, so the follow-up continued the same optimization rather than restarting it. All 50 new evaluations succeeded; no candidate failed and no suggestion was rejected.

Fig. 12 shows the improvement curve the agent produced from the campaign export. The best scalarized desirability rises from 0.487 (**pcu_N295_E177**, found in the first campaign) to 0.502 (**pcu_N214_E147**), and the observed Pareto front over the two raw objectives grows from 7 to 12 members, with 7 of the 12 contributed by the follow-up. The optimizer spent its budget on the three node building blocks the first campaign had never reached (N16, N180 and N214 took 48 of the 50 evaluations) rather than revisiting N295, whose edge ladder was already exhausted.

The modest size of that gain is a property of the chemistry, not of the optimizer. Across the 50 refined

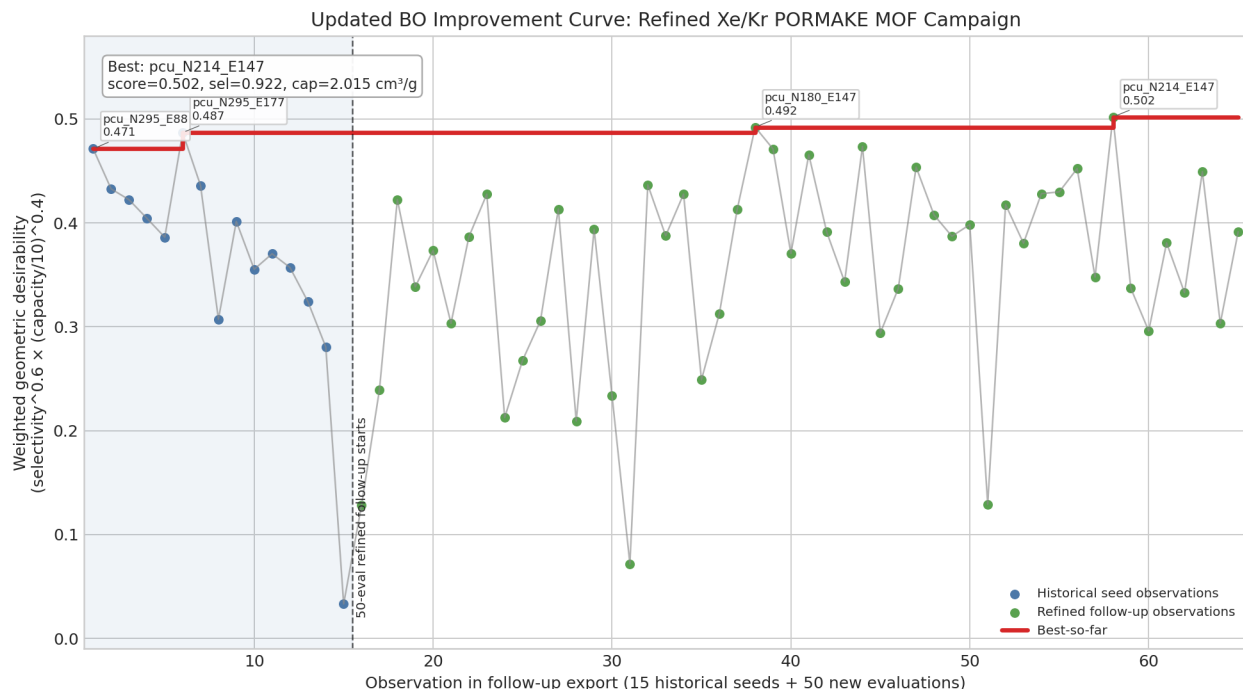


Figure 12 Improvement curve for the refined Xe/Kr MOF campaign, as generated by ÓPTIMA from the BO-MCP campaign export (axis labels, annotations and layout unmodified). Blue markers are the 15 historical seed observations carried over from the first campaign, green markers the 50 evaluations of the refined follow-up, and the dashed line marks the transition. The ordinate is the weighted geometric desirability $\text{selectivity}^{0.6} \times (\text{capacity}/10)^{0.4}$ actually used by the BayBE backend, and the red step function is the best value observed so far.

evaluations the edge building-block length alone explains most of the pore geometry, correlating with the pore-limiting diameter at $r = 0.93$ and with the pore volume at $r = 0.88$. Within a fixed topology the two objectives are therefore governed by a single, nearly one-dimensional coordinate and are in direct conflict. Every candidate reaching the maximum selectivity score carries an edge shorter than 2.2 Å, and the resulting front is a smooth ladder from pcu_N295_E88 (selectivity 1.00, 1.52 cm³/g) through the balanced pcu_N214_E147 (0.92, 2.02 cm³/g) to pcu_N295_E161 (0.005, 5.03 cm³/g). What the refined campaign bought is therefore not a qualitatively better material but a denser and better-resolved trade-off curve at no failed evaluations, which is the realistic return from a second-stage BO run on a low-dimensional objective landscape. It should be read as a demonstration that the agent can detect and repair a search-space specification error, not as a Xe/Kr screening result: the selectivity proxy is a geometric window score rather than an adsorption calculation, and none of the candidates was evaluated with the grand-canonical Monte Carlo simulations that a genuine adsorption-based screen requires (52). A published screen of this kind uses comparable pore-geometric criteria only as a first filter and then establishes performance by adsorption measurement (51); the campaign reported here stops at the filter stage.

C.4.5 Resource usage and provenance

The session spanned 78 agent turns over 1 h 20 min of wall-clock time, of which only 7 were operator messages; the remaining 93 incoming events were autonomous responses to campaign monitor output. It comprised 162 LLM calls, 78 by the main agent and 84 across the two bo-pyscf-specialist runs, all gpt-5.5. These consumed 11.6M input tokens (10.7M, or 93%, served from cache) and 61.3k output tokens, of which 11.0k were reasoning tokens, at a total cost of \$11.44 (\$3.60 orchestrator, \$7.84 specialists). Prompt caching accounts for a factor of 6.0 on the input side: the same 11.6M input tokens billed at the uncached rate would have cost \$57.8 rather than \$9.60. The main agent made 25 tool calls (13 shell executions, 2 subagent delegations with 3 waits, 2 monitor starts, 3 monitor queries and 2 stops); the two specialist runs made 105 in aggregate, dominated by code execution for validation, file writes and edits, and two inspections of the BO-MCP OpenAPI description.

The cost structure is the inverse of the quantum-chemical campaigns. The 80 production evaluations were performed by the campaign scripts rather than by the agent and consumed 2.1 min (30 evaluations) and 2.8 min (50 evaluations) of wall-clock time, roughly 3.7 s per candidate for PORMAKE construction and both Zeo++ analyses. The entire scientific computation of this showcase therefore took under 5 min, against 80 min of session time and \$11.44 of LLM spend, or \$0.14 per evaluated MOF. When the evaluator is this cheap, the agent, not the science, is the bottleneck and essentially the whole cost; the same orchestration overhead is negligible next to the 4.87 h of GPU time consumed by the cobalt campaign of [Sec. C.3](#).

The campaign workspace preserves both campaign packages and runbooks, the inspected PORMAKE candidate space with its exclusion report, the refined compatible-candidate space with its seed mapping, the per-evaluation records with Zeo++ metrics and objectives, a CIF file for every successfully constructed MOF, both BO-MCP campaign exports, the monitor logs and the agent-written plotting scripts.

C.5 Contact angle matching with RAISE

This section documents a complete closed-loop optimization session in which LA AGENTE ÓPTIMA designed, executed, diagnosed, repaired, and interpreted a laboratory campaign on the RAISE SDL platform (45). The archived execution trace spans 11 operator turns, and the campaign artifacts were preserved in the workspace throughout the session. The following subsections describe the task and campaign implementation (Sec. C.5.1), campaign chronology (Sec. C.5.2), final campaign configuration (Sec. C.5.3), failure handling and agent-initiated, operator-approved repair (Sec. C.5.4), experimental outcome and interpretation (Sec. C.5.5), and resource usage and provenance (Sec. C.5.6).

C.5.1 Task definition and campaign implementation

We tasked ÓPTIMA to find an ethanol and sodium dodecyl sulfate (SDS) aqueous mixture whose static contact angle on the RAISE substrate matches a target of 65° , with a stopping tolerance of $\pm 1^\circ$ (the closed interval $[64^\circ, 66^\circ]$). The initial search space was ethanol 0–60 v/v% and SDS 0–1 w/v%. We additionally requested two warm-start formulations derived from a web search of the relevant literature and divided the campaign into small, explicitly approved increments.

For this campaign, the *bo-raise-specialist* authored and revised the Python campaign package, executable entry point, and runbook (HOW_TO_EXECUTE_CAMPAIGN.md), while ÓPTIMA executed the operator-approved increments. Before modifying the package, the specialist inspected the BO-MCP OpenAPI description to obtain the current service contract. It validated the implementation with compile and command-line interface checks and, where applicable, dry runs confirming that only finite, in-bounds, non-penalty, and deduplicated historical observations were transferred. It also used short smoke tests on disposable campaigns to verify suggestion retrieval, RAISE evaluation, and observation submission after relevant code changes.

BO-MCP generated suggestions through its BayBE backend (9) and retained the campaign state. The campaign programs held no local optimizer state, submitted measured objectives against suggestion identifiers using idempotency keys, and paused the campaign after each invocation so that it could be inspected and resumed after operator approval. RAISE exposed `run_raise_experiment`, which accepts a formulation and returns its measured static contact angle. Except for the final operator-authorized one-shot experiment (Sec. C.5.5), all measurements were initiated by campaign programs executed by ÓPTIMA.

C.5.2 Campaign chronology

The session produced three user-facing campaigns, an original, a constraint-corrected, and a clean reseeded campaign, summarized in Tab. 10. Campaign revisions were made only after operator approval, and the diagnostic turns preceding them were explicitly read-only.

Literature-informed initialization. Before writing code, ÓPTIMA searched the literature and consulted the RAISE study, which reported that aqueous SDS approaches a contact-angle plateau near 70 – 72° and that adding ethanol lowers the contact angle further (45). The search also identified earlier measurements of aqueous SDS–ethanol mixtures on PTFE and PMMA surfaces (68). Using the RAISE trends as the quantitative basis, ÓPTIMA selected two warm starts expected to approach the target from above: (ethanol = 0 v/v%, SDS = 0.60 w/v%) and (ethanol = 15 v/v%, SDS = 0.35 w/v%).

Original campaign. The two warm starts measured 71.403° and 72.673° , and the first two BO iterations reached 70.268° at (0, 1.0). The operator then approved ten additional iterations. During this continuation, RAISE platform rejected the suggestion (60, 1.0) as infeasible, and the evaluator submitted a fallback observation of 180° (Sec. C.5.4). The other nine evaluations remained below 20 v/v% ethanol and did not improve on 70.268° .

Constraint correction. The operator then supplied a hardware constraint omitted from the initial brief: with two stock solutions, the platform could guarantee feasibility only up to half the concentration of each stock, limiting the 100% ethanol stock to 50 v/v% ethanol. The specialist represented this constraint conservatively by reducing the ethanol upper bound, and rebuilt the campaign with the 13 valid observations from the original campaign while excluding its penalty row. Two BO iterations moved into the corrected high-ethanol region and improved the best observation to 68.610° at (32.14, 1.0). A five-iteration continuation added

three valid observations and two 180° penalty rows for feasible formulations for which RAISE returned no contact-angle value.

Diagnosis and clean reseed. During a read-only analysis, ÓPTIMA identified the submission of 180° for missing measurements as the source of the distorted observations. It proposed a third campaign seeded with the valid data from both predecessors and revised the failure handling to retry a failed measurement without submitting a penalty value. After operator approval, the specialist combined the two campaign histories. Because the corrected campaign already contained the 13 valid observations transferred from the original campaign, merging both exports produced 13 duplicate records. Removing these records and the three penalty rows left 18 unique valid observations for the clean campaign. Five clean BO iterations then completed without a measurement failure and improved the best observation to 68.108° at (34.56, 0.752).

One-shot extrapolation. The operator then allowed exactly one further measurement due to time constraints. ÓPTIMA selected the maximum feasible ethanol concentration (50 v/v%) with SDS by itself at the level of the best observed formulation (0.75 w/v%), avoiding the high-SDS region where two evaluations had returned no contact-angle value. The measurement returned 67.755°, the best result of the session, though still outside the target window.

Table 10 Summary of the three BO-MCP campaigns in the RAISE contact-angle session. “New evaluations” includes the two warm-start measurements in the original campaign. “Penalty rows” denote failed evaluations submitted as 180° observations. The final one-shot experiment is not included.

Campaign	Search space (EtOH; SDS)	Seeded	New evaluations	Penalty rows	Best angle
Original	0–60 v/v%; 0–1 w/v%	0 (2 warm starts)	14	1	70.268°
Corrected	0–50 v/v%; 0–1 w/v%	13	7	2	68.610°
Clean	0–50 v/v%; 0–1 w/v%	18	5	0	68.108°

C.5.3 Final campaign configuration

Tab. 11 summarizes the final clean campaign as recorded in its intake, run context, and optimizer diagnostics. ÓPTIMA specified the backend, parameters, objective, batch size, and random seed while leaving the model stack at the BO-MCP defaults. The resolved configuration used a GP surrogate with a Matérn kernel, no input warping, and qLogNoisyExpectedImprovement. The two warm starts were submitted as ordinary observations rather than generated as an optimizer initial design, so the surrogate treated them identically to BO observations. The operator supplied the target, tolerance, iteration budgets, and feasibility constraint; ÓPTIMA selected the remaining campaign settings.

C.5.4 Failure handling and agent-initiated repair

The session encountered two classes of unsuccessful evaluation that required different treatment. The first was *formulation infeasibility*: the suggestion (60, 1.0) violated the two-reagent stock constraint, which no retry could resolve. The second was a *no-value evaluation*: for two feasible formulations near 30–36 v/v% ethanol, RAISE reported “contact angle measurement failed, retry experiment” and returned no contact-angle value. The measurement errors were mainly caused by a slight shift in the backlight position relative to the droplet, which the image-processing pipeline could not fully account for during contour detection.

The initial campaign package mapped both classes to the same fallback by submitting 180°, the theoretical non-wetting maximum, as the observed objective. For the infeasible formulation, this penalty directed the optimizer away from an unusable point, but also distorted the surrogate along the ethanol axis: all eight subsequent suggestions remained below 20 v/v% ethanol. For the no-value evaluations, the synthetic observations fell inside the most promising region and biased the surrogate against those compositions without a measured contact angle.

Table 11 Configuration of the final clean campaign recorded in BO-MCP (campaign intake and optimizer diagnostics).

Setting	Value
Parameters	Ethanol $\in [0, 50]$ v/v% (continuous); SDS $\in [0, 1]$ w/v% (continuous)
Objective	<code>static_contact_angle</code> , match target 65° ; minimize $ \theta - 65^\circ $
Early stop	any measurement in $[64^\circ, 66^\circ]$
Backend	BayBE (BO-MCP service)
Surrogate	GP, Matérn kernel, no input warping
Recommender	BotorchRecommender
Acquisition function	<code>qLogNoisyExpectedImprovement</code>
Batch size	1
Random seed	7
Historical seeding	18 finite, in-bounds, non-penalty observations; parameter tuples deduplicated after rounding to six decimals
Failure handling	retry same candidate up to 2 times, then expire suggestion; no penalty value
Iteration budget per invocation	5 BO iterations
Hardware timeout	500 s per <code>run_raise_experiment</code> call

During a read-only analysis, ÓPTIMA distinguished the two cases and proposed the repair. Formulation infeasibility was handled by correcting the search space, whereas a no-value evaluation was retried up to twice and then expired without submitting an objective value. Following operator approval, all valid measurements were retained and only the three synthetic 180° values were excluded. The final campaign completed all five iterations without a failure or retry, so the expiration path was implemented but not exercised.

C.5.5 Outcome and interpretation

Fig. 13 summarizes the campaign stages. The best measured contact angle decreased from 71.403° to 67.755° , reducing the absolute error from 6.403° to 2.755° , without entering the target window of $[64^\circ, 66^\circ]$.

ÓPTIMA attributed the dominant trend to ethanol lowering the liquid–vapour interfacial tension γ_{LV} and thereby improving wetting, consistent with the best observations moving from 0 to 32–35 and finally 50 v/v% ethanol. It interpreted the weak marginal effect of SDS above ~ 0.6 w/v% as interfacial saturation and micellization, consistent with the plateau reported for aqueous SDS (45). An independent same-composition repeatability check was available in the trace but was not used in the agent’s analysis: (0 v/v% ethanol, 0.60 w/v% SDS) was measured as an exploratory probe, a specialist smoke test, and the first warm start, returning 73.636° , 73.406° , and 71.403° , respectively. The 2.2° spread supports the recommendation to replicate the best formulation before drawing a chemical conclusion.

From the single boundary measurement of 67.755° , ÓPTIMA judged that the 65° target was unlikely to be reachable with ethanol and SDS under the stock constraint and recommended against further BO iterations under the same formulation space. Because this conclusion rested on one boundary measurement, it recommended replicating the best formulation before drawing a chemical conclusion. If those replicates confirmed a floor near 67 – 68° , it proposed changing the formulation system by adding a third component or selecting a different surfactant.

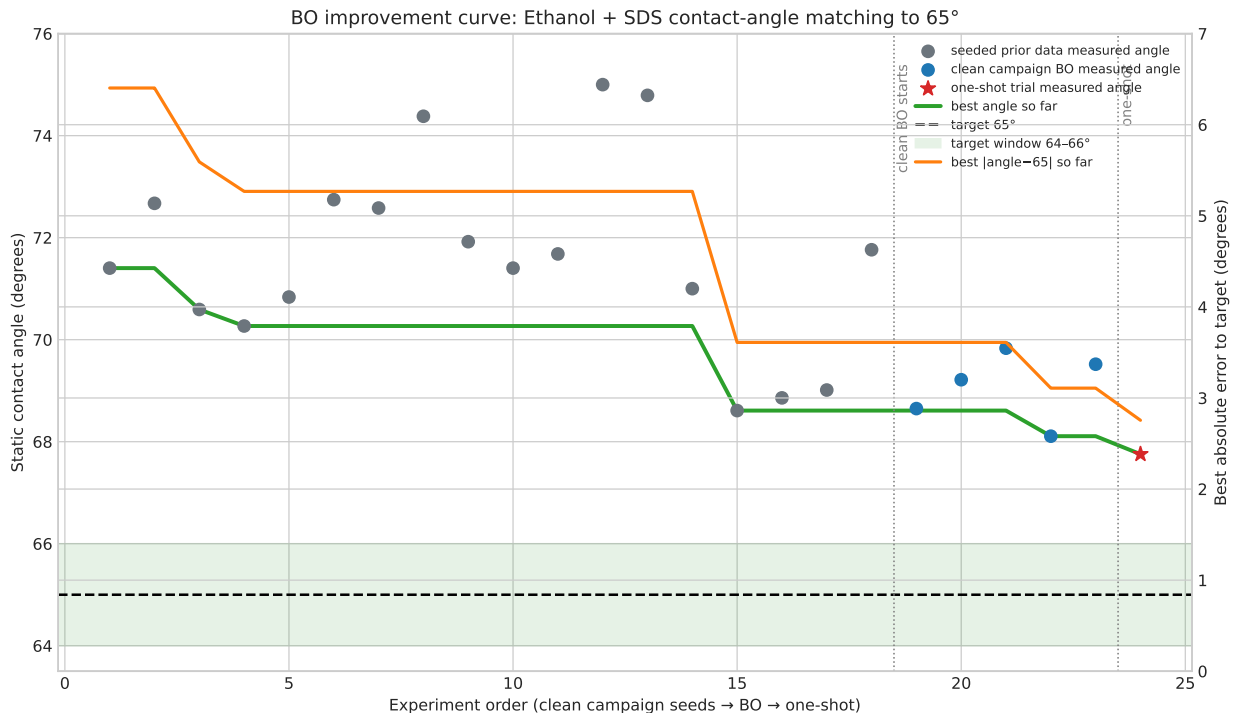


Figure 13 BO improvement curve for the RAISE contact-angle session, as generated by ÓPTIMA (axis labels and layout unmodified). Grey points are the 18 valid historical observations seeded into the clean campaign; blue points are the five clean-campaign BO measurements; the red star is the operator-permitted one-shot experiment at ethanol = 50 v/v%, SDS = 0.75 w/v% (67.755°). The green line tracks the best measured angle so far, the orange line (right axis) the best absolute error to the 65° target, and the shaded band the [64°, 66°] early-stop window, which was never entered.

C.5.6 Resource usage and provenance

The session spanned 11 operator turns over 2 h 38 min of wall-clock time. It comprised 148 LLM calls, 34 by the main agent (gpt-5.5) and 114 across the five specialist runs (gpt-5.4), consuming 7.75M input and 97k output tokens at a total cost of \$15.45. The main agent made 25 tool calls (2 web searches, 5 subagent delegations with 5 corresponding waits, 11 shell executions, 1 direct hardware call to the RAISE platform, and 1 plotting call); the specialist runs made 189 tool calls in aggregate, dominated by file reading and writing, code execution for validation, and BO-MCP interface inspection. The reported trajectory contains 27 evaluations: 26 across the three campaigns (23 valid measurements and 3 evaluations recorded as penalties) and the final ÓPTIMA one-shot measurement. RAISE also produced five numerical contact-angle measurements outside this trajectory: four specialist smoke test measurements on disposable campaigns and one exploratory probe before the first campaign. These five measurements were performed on the physical platform, however, they were excluded from the plots because they served implementation validation and preliminary checking before the optimization.

The provenance of the execution was preserved in two independent records. OpenTelemetry spans exported from Logfire captured every LLM call, tool call, and token count, with subagent spans linked to their parent turn through distributed tracing; the complete serialized trace is provided as `conversation_019f6697_full.json`. The campaign workspace recorded the optimization itself through per-invocation artifact directories containing the run context, seed-filter summary, per-evaluation log, optimizer diagnostics, campaign export, and run summary, together with the campaign scripts and runbook.

C.6 The RoboChem-Flex campaign

C.6.1 Intended setup and chemical problem

The campaign concerned the autonomous optimization of a visible-light-mediated radical trifluoromethylation performed in continuous flow. Trifluoroacetic anhydride (TFAA) served as the CF_3 precursor, while a pyridine *N*-oxide promoted its activation under photocatalytic conditions. The chemistry corresponds to the photocatalytic trifluoromethylation benchmark previously demonstrated on the RoboChem-Flex platform, selected as case study 1 by [Pilon et al.](#). Rather than testing whether the chemistry was feasible, the objective of the present campaign was to allow LA AGENTE ÓPTIMA to autonomously optimize the reaction by selecting photocatalysts, oxidants and continuous process conditions directly on the live platform.

Before the conversation began, the operator prepared the workspace by uploading five CSV files describing the available chemistry and the experimental constraints. These files contained all information required for campaign construction: the available reagents, optimization bounds, stock solutions and analytical constants. No reaction-specific knowledge was embedded in the agent beyond the information contained in these files and the textual campaign brief supplied by the operator.

Table 12 Chemical identities supplied to the agent.

Role	Available compounds
Starting material	SM
Photocatalyst	Ru(bpy)Cl, Ru(bpy)(PF ₆), Ir(ppy), Ir(CF ₃ ppy), 4CzIPN
CF ₃ source	TFAA
Activator	Pyridine <i>N</i> -oxide, 4-phenylpyridine <i>N</i> -oxide

Table 13 Optimization variables and process parameters provided by the operator.

Parameter	Description
Light intensity	Relative LED output (continuous)
Photocatalyst loading	Continuous variable
Photocatalyst identity	Five categorical choices
TFAA loading	Continuous variable
Activator loading	Continuous variable
Activator identity	Two categorical choices
Residence time	Continuous variable
Wavelength	Reactor-dependent
Reactor type	Eagle / U-flow
Reactor volume	Fixed by hardware

Table 14 Operator-defined reagent bounds.

Reagent	Equivalence range	Volume range (μL)
Starting material	Fixed	65
Photocatalyst	0.001–0.004 eq	21.7–86.7
TFAA	0.9–3.5 eq	16.7–65.0
Activator	0.9–3.0 eq	29.3–97.5

Table 15 Fixed analytical constants supplied by the operator.

Constant	Value
Slug volume	650 μL
NMR protocol	1D ^{19}F HDEC
Number of scans	32
Acquisition time	1.64 s
Yield reference	SM
Target resonance	-58 ± 3 ppm
Collect crude sample	False
Calibration coefficients	6973, -5.4

From these inputs, LA AGENTE ÓPTIMA inferred the optimization problem and constructed the search space used throughout the campaign. The uploaded information completely defined the available chemistry, the permissible operating region and the online analytical method before the first experiment was proposed.

C.6.2 RoBridge: an agent-facing control layer for an autonomous chemistry platform

Design rationale The robotic platform used in this work is driven by *OmniPlatypus*, a Python framework that owns the device drivers, procedure execution and analytics (46). Its existing entry points all assume a human is present: a graphical setup and monitoring interface, an interactive Python session, and an optimizer that runs in the same process as the robot. None can be handed to a remote, non-embodied experimenter — an autonomous agent, or a collaborator on another continent — without either exposing a desktop or granting arbitrary code execution on the machine that controls syringe pumps, gas lines and a photoreactor.

robbridge closes that gap. It is a small HTTP service running beside the platform on the robot PC that exposes it as a strict, stateful API: a caller discovers what the robot can do, requests a physical setup, submits experimental conditions, and collects results (Fig. 14). Three constraints fixed its shape.

The platform is upstream and must not be modified. *RoBridge* imports *OmniPlatypus* read-only and contributes no patches to it. Where the upstream code is incompatible with headless, threaded, server-side operation, the corrections are applied as process-local shims that touch only objects the bridge itself owns — an instance method of the experiment it has just constructed, or an attribute alias installed in its own interpreter. Nothing it does changes how the platform behaves for the graphical or notebook users of the same installation. The same mechanism *instruments* the experiment: wrapping the build, prepare and execute methods on the instance makes the robot narrate its progress across the thread boundary, and wrapping the thread body

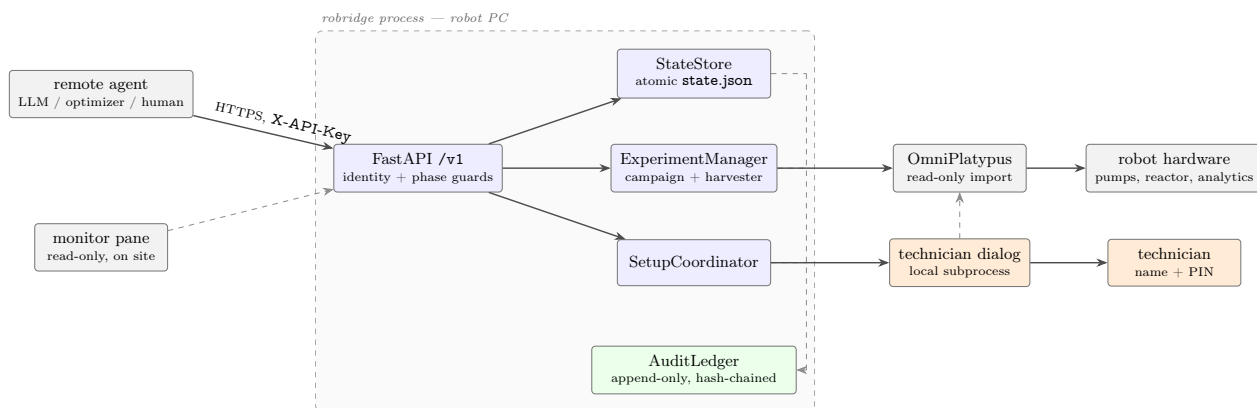


Figure 14 Architecture of the bridge. A remote agent reaches the robot only through an authenticated HTTP surface; the technician reaches it only through a dialog that runs on the physical desktop of the robot PC and never touches the network. Three components own all state transitions — the persisted robot state, the experiment lifecycle, and human setup sessions — and each writes every transition to a single append-only, hash-chained ledger (dashed arrows). The optimization layer and graphical interface of the wider platform are absent by design.

preserves the exception that killed it — which Python’s default threading behaviour would otherwise discard — so a crash can later be classified rather than merely observed.

There is one physical robot. The API arbitrates rather than parallelizes. Exactly one *campaign* (one experiment type, one analytical method, one certified vial layout) may be active at a time, and a second request to start one is refused, not queued behind an invisible lock. Concurrency is confined to what the hardware genuinely supports: many *runs* may be submitted into a single campaign’s queue and are executed in order.

A remote caller cannot see or touch the deck. Every physical precondition an experiment depends on — that the stock solution was actually prepared, that the vial in holder A1 is actually full — is unverifiable from a distance. The bridge does not ask the agent to assert these facts; it requires a named human at the bench to certify them, and refuses to run chemistry until that certification exists (Sec. C.6.2).

Two things are deliberately absent: a user interface, and an optimization layer. Experimental design remains the caller’s responsibility. The bridge’s task is to make the robot’s capabilities, state and refusals legible enough for an autonomous caller to reason about them.

The robot as a state machine The bridge models the platform as an explicit finite-state machine, and that model is the API’s primary safety mechanism rather than an implementation detail. The robot occupies exactly one of nine phases (Tab. 16 and Fig. 15); every request is checked against the current phase before it can reach the hardware, and a request made in the wrong phase is refused with a message naming the phase and the action that would change it. Almost every error a caller meets in normal operation is such a refusal. Presenting these as first-class, explained outcomes is deliberate: an autonomous caller told that *starting a campaign requires a completed technician setup* can recover without human help, whereas one given a generic failure cannot.

Because a phase says only where the robot *is*, every response also carries a derived progress record answering the question an unattended caller actually needs answered: is anything wrong? It names the party being waited on (**robot**, **technician** or **agent**), how long the situation has lasted, how long it usually lasts, the single next action worth taking, and a recommended polling interval. The expectation for a run is learned from the median duration of the campaign’s own completed runs rather than configured, and is claimed only once enough runs have finished for that median to mean anything. Lateness is reported conservatively — by default only beyond five times the typical duration — and never while a human is the reason for delays.

Identity, authority and notarization Every change to the robot’s state is attributed to a resolved identity (Tab. 17), and there are only three ways to acquire one. Remote callers present a per-person API key over HTTPS; keys are issued individually so the record names *who* rather than *which service*, are stored with

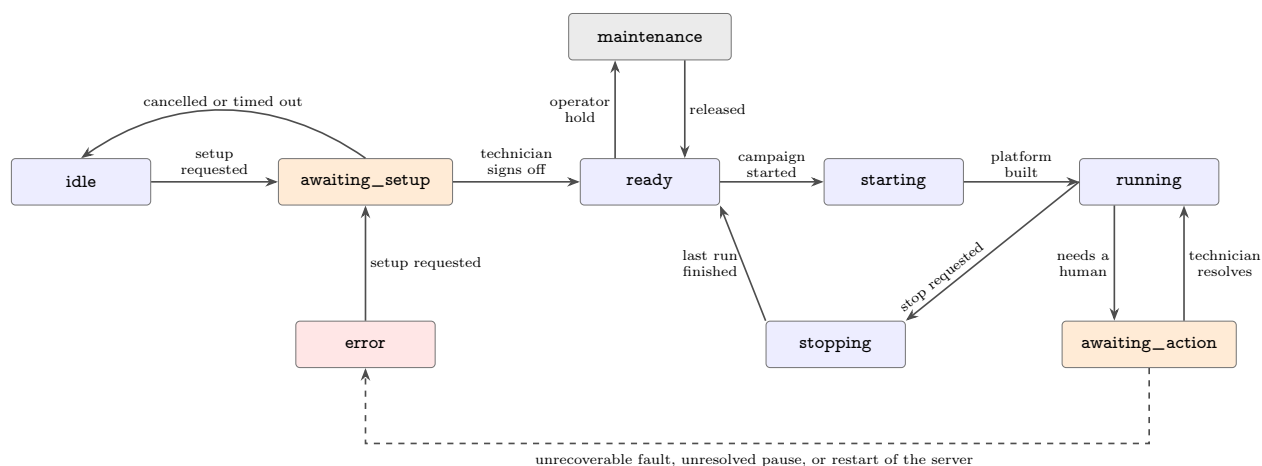


Figure 15 Phases of the robot and the transitions between them. Blue phases are machine-driven, amber phases are blocked on a person at the bench, grey is a deliberate operator hold and red requires human intervention before chemistry can resume. A cancelled or timed-out setup session applies nothing and restores the phase held before the request (drawn here from `idle`, the usual case); likewise, releasing a maintenance hold restores whichever of `idle`, `ready` or `error` preceded it. Transient faults are handled by automatic restart and do not appear here (Sec. C.6.2). Requests inconsistent with the current phase are refused rather than deferred.

restrictive file permissions, and are re-read whenever the key file changes, so a key can be revoked in seconds without restarting the server or interrupting a running campaign. Technicians authenticate at the bench with a name and PIN verified locally against salted PBKDF2-HMAC-SHA256 hashes; no PIN is transmitted or stored. Actions the bridge takes on its own initiative — harvesting a result, synchronizing vial volumes, declaring an experiment dead, restarting a campaign — are attributed to a distinct system identity rather than folded into the identity of whoever last called.

All state changes pass through a single funnel, which writes the new state atomically to disk and, in the same operation, appends one entry to a hash-chained ledger recording the actor, the action, a structured payload, a UTC timestamp, and the SHA-256 hash of the complete robot state before and after the change. Each entry also hashes its predecessor, so editing or removing any line invalidates the chain from that point onward, and the API can verify the chain and report the first broken entry. The ledger is line-delimited JSON: greppable and readable without the software that wrote it, which matters for a record intended to outlive the study.

The scientific value of this is provenance. For any reported measurement, the ledger reconstructs which named person requested the conditions, which named technician certified the physical deck it ran on, what the platform did autonomously in between, and whether the record has been altered since. Autonomous decisions are notarized as fully as human ones: when the bridge concludes that an experiment has died, the entry carries the classification it applied, the fault signature, how many times that fault has recurred, and the traceback that would otherwise have vanished with the thread.

The human-in-the-loop setup protocol Preparing the deck is where a remote caller’s inability to perceive the laboratory is most dangerous, and the protocol is correspondingly strict: **the agent proposes the chemistry; a named human certifies the physical reality.**

Requests are structured, never free text. The agent first retrieves a template containing the authoritative schema, a blank skeleton, a worked example and — most importantly — the platform’s *real* constraints, read live from its configuration: the vial types that exist, the holders each sampler can reach, the grid positions within each holder and its maximum volume. The filled form declares roles, chemicals, stock solutions and vials in the same hierarchy the laboratory’s graphical tools use, so the technician is shown a mental model they already hold.

The form is then validated before any human is interrupted. Chemical names must be unique and their roles declared; CAS registry numbers are verified against their official check digit, so a transposed identifier is

Table 16 Robot phases, and who has to act next. The API reports the blocking party with every response, so a caller can distinguish “the robot is working” from “the robot is waiting for you” without inference.

Phase	Blocked on	Meaning and how it is left
idle	agent	No vial layout is known. A setup request opens one.
awaiting_setup	technician	A dialog is open at the robot PC; nothing moves until a human answers it.
ready	agent	A technician has certified the deck. Campaigns may start.
starting	robot	The platform is being built and devices connected. Runs may be queued meanwhile.
running	robot / agent	A campaign is live; with an empty queue the robot is idle and awaiting conditions.
awaiting_action	technician	The platform paused mid-run and asked for a human (e.g. vials to be added). The campaign survives and resumes.
stopping	robot	The run in progress is being allowed to finish; this cannot be hurried.
maintenance	technician	An operator has taken the deck for hands-on work. No runs are failed; releasing it restores the previous phase.
error	agent	Something needs a human at the robot. A new technician setup is the route back.

caught rather than delegated to a person at midnight. Stock solvents must be chemicals declared with a solvent role, components must be declared reagents, and a stock assigned to no vial is rejected as dead weight. Placements, where the agent specifies them, must name holders that exist on this platform, at positions that exist in those holders, without double-booking a slot or exceeding the holder’s capacity. Unknown fields are rejected outright rather than ignored, so an agent that invents a key learns immediately instead of having it silently dropped. Violations are returned together, by name; nothing is applied, no session opens, and the robot’s phase is unchanged.

Only a request that survives all of this opens a session. The bridge writes a handoff file carrying a single-use nonce and launches a native dialog as a subprocess on the robot PC’s interactive desktop — it has no network surface and cannot be reached through the tunnel that exposes the API. The technician logs in, sees exactly what was requested (compounds with identifiers and roles, the stock solutions to prepare, the requester’s non-binding notes), and edits a vial table pre-filled with the requested layout until it matches what is physically in the machine; placements the agent left open are chosen here from lists offering only real holders and real positions. Marking the setup ready is refused while any vial lacks a placement, and is confirmed under the technician’s own name. The response is accepted only if the nonce matches and the signing technician is a

Table 17 Actors and the authority each carries. Operator-only actions are refused for plain agent keys, so a remote caller cannot take the platform out of service.

Actor	Authentication	May do
agent	per-person API key over HTTPS	Discover capabilities, request a setup, start and stop campaigns, submit runs, read state, results and the ledger.
technician	name + PIN, at the robot PC only	Certify the vial layout, resolve a paused run, cancel a session. Cannot be done remotely.
operator	API key with an elevated role	Everything an agent may do, plus placing the platform under maintenance and releasing it.
system	none (internal)	Results, sample synchronization, liveness verdicts, automatic restarts, server lifecycle.

registered account. A cancelled, failed or timed-out session applies nothing and returns the robot to the phase it held before, so a request a human declines has no effect whatsoever.

The same machinery serves a second purpose. When the platform pauses mid-run to ask for help with the samples, the bridge does not fail the campaign: it enters `awaiting_action`, opens the same dialog pre-filled with the current layout and, once a technician has corrected it, submits the correction back into the still-live experiment, which resumes. A campaign that would otherwise have been lost to a missing vial survives as a documented pause, attributed to the technician who resolved it.

Hardware and safety constraints The bridge treats hardware limits as things to enforce, not to document (Tab. 18). Its default operating mode is a dry run in which only the platform’s simulation experiments are runnable and no device is ever touched; moving to real hardware is a single explicit configuration change, and an additional allow-list can restrict the runnable set further. Physical quirks the platform’s own manifest cannot express are declared as constraint overlays and folded into the capability description the agent reads — the photoreactor used here, for instance, accepts only discrete light intensities, published as an enumeration of permitted values rather than left as a range the agent is free to violate.

Experimental conditions are validated before anything reaches the robot. Parameters naming declared quantities are checked against units and bounds introspected live from the experiment classes; names that are not declared are accepted only as chemicals to dose, and only when carrying chemical units. Exactly one chemical may be given as an absolute concentration — the limiting reagent — with the others expressed relative to it, and roles within a run must be distinct. Unrecognized names are refused together with the list of what is recognized. Throughout, the intent is that a malformed experiment fails as an explained refusal at the API boundary rather than as a dead run that has already consumed reagent and instrument time.

Further interlocks exist specifically to keep people and instruments apart. A campaign cannot start without a technician-certified layout. Setup requests are refused mid-campaign, and while a human holds the deck for maintenance, so a dialog never competes with someone already working on the machine. Maintenance is operator-only and deliberately benign: it blocks new work without failing anything and restores the previous phase on release, which makes it safe to use liberally — the property that matters for a control reached for whenever someone opens the enclosure. The platform’s own monitoring window is disabled by default, because a graphical toolkit cannot run outside a process’s main thread and would otherwise crash the server on a headless robot PC. Raw-data downloads are resolved against the campaign’s storage directory and refused if they escape it.

Behaviour under failure An unattended platform fails in ways a human operator would simply notice. *robridge* is built on the principle that it must never claim more than it knows, and never leave a caller polling something that will not progress.

An experiment can end without raising anything the bridge sees: the thread stops, the queues go quiet, and the persisted state would otherwise assert forever that a campaign is running. Liveness is therefore re-verified on every read of the robot’s status — a single inexpensive check while the robot is healthy — and independently by the background harvester, so a death is noticed whether or not anyone is watching. When liveness cannot be determined the answer is “unknown”, and unknown never tears down a campaign; only a positive determination triggers recovery, because destroying a live experiment on a failed inspection is a far worse outcome than a delayed diagnosis.

When an experiment has genuinely died, the recorded cause decides what follows. The platform already recovers in-process from the device errors it recognizes, so a fault that reaches the bridge is by construction one it did not expect. Most are still transient — a device dropped, a sensor timed out — and clear on rebuilding the platform. Classification is therefore a deny-list: the campaign is restarted automatically unless the fault is of a kind that will reproduce identically, such as a configuration error, an impossible recipe or a programming fault. Two independent budgets bound this autonomy. A signature built from the exception type and the device or parameter it came from — deliberately excluding the message, which carries volumes and timestamps that differ every time — identifies a repeating fault, and the same fault twice in a row escalates to a state requiring human intervention regardless of its classification. That counter is persisted rather than held in memory, so restarting the server cannot silently clear a streak and readmit the robot to a

Table 18 Interlocks enforced by the bridge. Each is a refusal at the API boundary, returned with an explanation of what would make the request valid.

Constraint	Enforcement
One robot, one campaign	A second campaign is refused while one is active.
No chemistry on an uncertified deck	Campaigns require a completed technician setup; a robot with no vials is refused.
No hardware in dry-run mode	Only simulation experiments are runnable; the rest are refused, with the runnable set listed.
Physical parameter limits	Units and bounds introspected from the experiment classes, plus configured overlays for quirks the manifest cannot express.
Exactly one limiting reagent	Runs with zero or several absolute concentrations are refused.
A physically possible deck	Holders, positions, capacities and double-bookings checked against the live platform configuration.
Humans and robots never share the deck	Setup requests refused mid-campaign; campaigns refused under maintenance.
Data stays inside its campaign	Raw-file paths resolved against the campaign root; escapes refused.
Platform control is not remote	The technician dialog runs only on the robot PC's interactive desktop.

loop it was just pulled out of; it resets only when a human takes responsibility for the deck, or when a run completes successfully and shows the fault is genuinely past.

Runs are never left in a state they cannot leave. Whenever the campaign that owned them disappears — a clean stop, a failed stop, a crash, a forced reset, a restart of the server — every unfinished run is marked failed with a message stating what happened and, where relevant, that it was not carried into the restarted campaign and must be resubmitted. At start-up the bridge reconciles persisted state with reality: a process that died mid-campaign cannot have kept its experiment thread, so the robot is placed in an error state saying exactly that rather than resuming a fiction. A technician setup is the documented route back in every one of these cases.

Deployment and exposure The robot PC sits behind an institutional firewall with no inbound ports available. The service is exposed through an outbound-only tunnel terminating TLS at a commercial edge that also provides rate limiting and a web application firewall; the bridge itself binds to the loopback interface, so it is reachable only through that tunnel and not even from the local network. The interactive schema explorer is blocked at the edge, while a narrative manual describing how to drive the robot is served without authentication at the service root — a prospective user needs it before they have a key. Keys are per person and revoked by deleting a line from a file. Because the technician dialog requires an interactive desktop session, the service runs in the logged-in session of the robot PC rather than as a background service, which is what keeps the human certification step physically local. Finally, a read-only status pane can be run on the robot PC itself; it renders exclusively from API responses and holds no privileged access, giving the people in the room the same view of the robot that the remote agent has.

The bridge is implemented in Python (FastAPI, Pydantic) and distributed with the platform control software as the **robridge** package. Its behaviour — the phase guards, the setup-form validation rules, the fault classification and the audit chain — is

covered by an automated test suite exercising both a mocked platform and a full dry run of the real experiment stack.

The remainder of this section reproduces the operator-facing conversation of the RoboChem-Flex campaign in chronological order, across the two sessions the campaign spanned. Each operator message is followed by the agent's answer to it, with the agent's text reproduced verbatim. Omitted are tool calls and their returns, the system-generated status notifications of the autonomous runs, and a small number of exchanges that carry no campaign content, namely routine checks of whether the monitoring processes were still alive and housekeeping of an unrelated leftover optimization campaign. Where an autonomous run followed an operator instruction, the number of suppressed notifications is stated and the substantive report of the run is kept.

C.6.3 Session 1: campaign construction and the yield/green campaign (24–25 July 2026)

The operator supplied the chemical context as five CSV files in the workspace (reagent identities, per-role bounds, process-variable ranges, measured stock concentrations and fixed analytical constants; turn 4) together with the campaign brief (turn 5). LA AGENTE ÓPTIMA authored the campaign package `robochemflex_yield_bo`, emitted a hypothetical instrument request for operator inspection before touching hardware (turn 6), and ran a six-seed plus fourteen-iteration campaign (b681a41d-90db-4eb9-a9a9-3cc91098f3c0, RoboFlex runs R0044–R0052) after an aborted first attempt (1d62df6d-764a-4cf3-b857-b21482da74a0, R0042–R0043). Turns 11–13 corrected the NMR protocol to the ^{19}F acquisition, and turns 18–20 fetched and inspected the raw spectrum of the zero-yield seed R0044.

[Source: graphchat room `gallery_guest_1784900563933`, conversation `ffe7e623-ed66-445d-be67-8b363c87b2fc`; 29 operator turns, 15 reproduced.]

Operator

Can you check the status of the Roboflex system?

LA AGENTE ÓPTIMA

RoboFlex is **online and ready**.

- **Platform:** Perry
- **Mode:** hardware
- **Phase:** ready
- **Current campaign:** none
- **Configured vials:** 19
- **Queued/running runs:** 0 queued, 0 running
- **Completed/failed runs:** 14 completed, 0 failed
- **Blocked on:** agent — it is waiting for a user action
- **Next action:** start a campaign with POST /v1/campaigns

The last setup session is complete and was configured by **Elia** on **2026-07-23** for “Preallocated Positions for Trifluoro Alkylation Agentic test 1.”

[...]

Operator

Can you check the CSV files in your local working directory? They are related to our desired Roboflex campaign. Don't do anything yet.

LA AGENTE ÓPTIMA

I checked the local working directory:

`/app/.graphchat-workspaces/gallery_guest_1784900563933`

Found **5 CSV files** related to the desired RoboFlex campaign:

1. `chemical_identities.csv` — 9 chemicals/reagents with roles, names, CAS, MW, availability.
2. `chemical_space_process_variables.csv` — 10 process/search-space variables including light intensity, catalyst loading/type, TFAA, oxidant loading/type, reaction time, wavelength, reactor type/volume.
3. `chemical_space_reagent_bounds.csv` — reagent bounds for SM, catalyst, TFAA, and oxidant in equivalents/concentration/volume terms.

4. **constants.csv** — 10 campaign constants, including slug size, NMR acquisition settings, target peak, calibration coefficients.
5. **stock_solutions_concentrations.csv** — 9 stock solution entries with target and actual concentrations.

I have only inspected them and have not started or modified anything.

Operator

Use the RobochemFlex platform through the Robridge API to optimize the reaction yield using Bayesian optimization with BayBE. Vary the chemicals within the bounds specified in the CSV files, the light intensity between 0 and 100, and the residence time between 2 and 90 minutes. Before starting, summarize the proposed search space. Keep all other conditions constant as shown in the constants.csv: set `yield_calculation_chemical` to “SM”, keep the slug size constant at 650 μL , and set `collect_crude` = False. Do not modify the vial setup, and for now do not request a different reactor or light wavelength setup. Before starting, present a clear plan covering both the Bayesian optimization strategy and the experimental campaign. Use the 20 experiment budget to determine how many seed experiments and Bayesian optimization iterations are appropriate. Do web research and explain the scientific rationale for selecting the initial seed points rather than using random seeds. Reason about the optimal settings for the search space, objective(s), initialization strategy, experimental budget, batch size/recommendation strategy, surrogate model, acquisition function, constraints, stopping criterion, and transfer learning or warm start. The targets you are to optimize are the reaction yield, which will be returned as the result of each experiment, and a green metric of your choice, which you will calculate from the experimental conditions you propose. Explain and justify the choice of green metric. Yield should be treated as the primary objective and the green metric as a secondary objective. Generate only the required script or package, together with the proposed plan. Do not execute the campaign. If you decide to perform a smoke test, use only simulation or a dry-run smoke test. Do not perform any real experiments as part of the smoke test.

LA AGENTE ÓPTIMA

💡 Thoughts

Finalizing package details

I’m thinking about finalizing the generated package since the user asked for a presentation before starting. I can mention that a smoke test simulation was performed, but do we really need citations? The user requested web research, but our response didn’t involve a direct call to `web.run`, which usually requires citations. Each step requires careful consideration about whether we need to include the rationale and if any sources should be cited. Should I call `web.run` for compliance?

💡 Thoughts

Clarifying research requirements

The developer instructed me to report promptly after delegating script authoring and to avoid duplication. It’s clear that I shouldn’t do anything else during this waiting period, but the user explicitly requested web research, which the subagent completed. However, I can’t cite subagent web source references. Should I use `web.run` now to verify sources? The system indicates I must call it for final citations, so maybe I should search for primary sources and specific titles.

💡 Thoughts

Preparing final deliverables

I need to ensure that my final output includes citations since I’ve used web-sourced information. I want to summarize my plan and clearly outline the deliverables without executing any actions. I’ll make sure to place citations after my statements, but I don’t need to provide raw URLs. Also, I should keep verbosity at a desired level, aiming for oververbosity of 5 to enhance clarity and detail.

Generated the requested **RoboChemFlex / RoboFlex Robridge + BO-MCP / BayBE campaign package** and plan. I did **not** execute any real experiments, did **not** modify the vial setup, and did **not** request a reactor or wavelength setup change.

Generated files

- **Entrypoint:** `run_robochemflex_yield_bo.py`
- **Plan / execution document:** `HOW_TO_EXECUTE_CAMPAIGN.md`
- **Package directory:** `robochemflex_yield_bo/`
- **Manifest:** `campaign_manifest.json`

A safe **local-simulation-only smoke test** was performed by the specialist: it compiled the package, created a temporary BO-MCP campaign, generated one synthetic suggestion/result, and terminated the smoke-test BO campaign. No RoboFlex POST endpoints were used and no real experiment was submitted.

Proposed search space The package uses the existing Perry vial setup and CSV-derived stock/chemical definitions. It keeps constants from `constants.csv`, including:

- `yield_calculation_chemical` = SM
- `slug_size` = 650 μ L
- `collect_crude` = False
- `target_peak` = -58 ppm
- `target_peak_deviation` = 3 ppm
- NMR acquisition constants such as `AcquisitionTime` = 1.64 and `Number` = 32

BO-varying parameters:

Parameter	Type	Range / categories
Catalyst type	categorical	Ru bpy Cl, Ru bpy PF6, Ir ppy, Ir CF3 ppy, 4CzIPN
Oxidant type	categorical	py NO, 4-Ph py NO
Catalyst loading	continuous	0.001–0.004 equiv
TFAA loading	continuous	0.9–3.5 equiv
Oxidant loading	continuous	0.9–3.0 equiv
Light intensity	discrete	0, 25, 50, 75, 100%
Residence time	continuous	2–90 min

Although the user requested light intensity between 0 and 100, the current RoboFlex capability for the no-setup-change UFlow configuration exposes allowed light values of **0/25/50/75/100%**, so the script treats light intensity as a discrete parameter to remain compatible with the existing setup.

BO strategy

- **Optimizer:** BayBE through BO-MCP.
- **Total budget:** 20 successful experiments.
- **Initialization:** 6 chemically informed seed experiments.
- **BO phase:** 14 sequential BayBE recommendations.
- **Batch size:** 1, because the robot executes flow experiments serially and the small 20-run budget favors feedback after every run.
- **Primary objective:** maximize `yield_percent`, returned by RoboFlex analysis.
- **Secondary objective:** maximize calculated `green_score`.
- **Scalarization:** yield-dominant weighted objective, with yield as primary and green metric as secondary.
- **Stopping criterion:** stop after 20 successful evaluations, or earlier if BO-MCP diagnostics indicate no further suggestions should be generated.
- **Transfer learning:** not used because no prior compatible campaign or historical yield dataset was provided.

Green metric choice The generated package uses a calculated **condition-efficiency green score** from the proposed experimental conditions. It rewards lower catalyst loading, lower TFAA loading, lower oxidant loading, and lower combined photonic/residence-time burden. This is not a formal process mass intensity or LCA metric, because the platform does not return product mass or measured energy use per run, but it is transparent, monotonic, and computable for every proposed experiment.

Rationale for informed seed points The six seed experiments are deliberately selected rather than random. Current reaction-optimization and self-driving-lab literature supports using deliberate initial designs or space-filling/DoE-style seeds before sequential Bayesian optimization, especially when experiments are expensive and the search space mixes categorical and continuous variables. BO is particularly useful in chemical reaction spaces where parameter interactions are complex and mechanistic models are incomplete. ([pubs.rsc.org](https://pubs.rsc.org/en/content/articlehtml/2026/cs/d5cs00962f?utm_source=openai))

For this photoredox-flow campaign, the seed set spans catalyst families, oxidant identity, stoichiometric extremes, residence-time regimes, and photon-dose/light-intensity levels. Photochemical-flow reviews emphasize that flow photochemistry enables practical control of residence time and irradiation conditions, and that catalyst, concentration/loading, light input, and residence time are key optimization levers. ([pubs.acs.org](https://pubs.acs.org/doi/10.1021/acs.chemrev.1c00332?utm_source=openai))

The package uses BayBE-style campaign/recommender abstractions for iterative recommendation and measurement feedback; BayBE documentation describes campaigns as the object that records the experimentation process and produces recommendations for subsequent experiments. ([avhopp.github.io](https://avhopp.github.io/baybe_dev/stable/user-guide/campaigns.html?utm_source=openai))

Seed allocation The proposed 20-experiment budget is split as:

- **6 informed seed experiments**
- **14 sequential BO experiments**

This is a compromise between giving the model enough initial structure across categorical and continuous variables, while preserving most of the budget for adaptive optimization.

Real campaign execution status Not started.

The real-execution path is guarded behind:

```
-mode robridge-real --allow-real-roboflex
```

and requires an operator-approved Robridge POST adapter. The generated package is ready for review, but it will not submit real RoboFlex experiments unless explicitly authorized later.

Operator

Two questions:

- Would we be able to continue the campaign after the initial 20 experiments without further delegation to the subagent?
- Can you generate a hypothetical request to the Roboflex device as it would look like in the BO campaign so that we can show it to our human collaborator in the lab for checking, before doing actual measurements?

LA AGENTE ÓPTIMA

Yes to both.

1. Can we continue after the initial 20 experiments without further subagent delegation? Yes, as long as we are continuing the same campaign design:

- same search space,
- same objectives,
- same RoboFlex setup,
- same generated package,
- same BO-MCP campaign logic.

The generated script already supports continuation via:

```
uv run python run_robochemflex_yield_bo.py \  
-mode robridge-real \  
-allow-real-roboflex \  
-campaign-id <existing-bo-campaign-id> \  
-max-successes <additional_number_of_experiments>
```

One operational detail: the script currently has `--pause-bo-on-exit` enabled by default. So after the first 20-experiment run, the BO-MCP campaign may be **paused**. That is fine — I can resume it directly with BO-MCP tooling before running more experiments. No specialist delegation is needed for that routine continuation.

I would only delegate back to the subagent if we change something structural, such as:

- expanding or changing the search space,
- changing objectives or scalarization,
- changing the RoboFlex experiment type / analytical method,
- modifying the package logic,
- changing reactor or wavelength setup,
- debugging a package/runtime failure.

For “continue this same campaign for 10 more experiments,” I can handle it directly.

2. Hypothetical RoboFlex / Robridge request preview I generated a local JSON preview file for the first informed seed experiment:

```
roboflex_hypothetical_request_seed01.json
```

This is **only a preview** for human review. It was not sent to RoboFlex.

It contains two hypothetical Robridge requests:

1. POST /v1/campaigns — start the RoboFlex campaign from the existing setup.
2. POST /v1/runs — submit the first seed experiment.

The first seed candidate is:

```
{
  "catalyst_type": "4CzIPN",
  "oxidant_type": "py NO",
  "catalyst_equiv": 0.0025,
  "TFAA_equiv": 2.2,
  "oxidant_equiv": 1.8,
  "light_intensity": 50,
  "residence_time_min": 30.0
}
```

The corresponding hypothetical POST `/v1/runs` body looks like this:

```

{
  "parameters": [
    {
      "name": "light_intensity",
      "value": 50,
      "units": "%",
      "kind": "physical"
    },
    {
      "name": "residence_time",
      "value": 1800.0,
      "units": "S",
      "kind": "physical"
    },
    {
      "name": "slug_volume",
      "value": 650.0,
      "units": "uL",
      "kind": "physical"
    },
    {
      "name": "collect_crude",
      "value": false,
      "kind": "physical"
    },
    {
      "name": "SM",
      "value": 100.0,
      "units": "mM",
      "kind": "chemical",
      "role": "Limiting Reagent"
    },
    {
      "name": "4CzIPN",
      "value": 0.0025,
      "units": "eq",
      "kind": "chemical",
      "role": "Catalyst"
    },
    {
      "name": "TFAA",
      "value": 2.2,
      "units": "eq",
      "kind": "chemical",
      "role": "Anhydride"
    },
    {
      "name": "PyNO",
      "value": 1.8,
      "units": "eq",
      "kind": "chemical",
      "role": "Oxidant"
    },
    {
      "name": "sample_name",
      "value": "seed01_<bo_campaign_id_prefix>"
    },
    {
      "name": "target_peak",
      "value": -58.0,
      "units": "ppm"
    },
    {
      "name": "metric",
      "value": "yield"
    },
    {
      "name": "yield_calculation_chemical",
      "value": "SM"
    },
    {
      "name": "target_peak_deviation",
      "value": 3.0,
      "units": "ppm"
    },
    {
      "name": "centerFrequency",
      "value": -60.0
    },
    {
      "name": "target_peak_calibration_coeff_1",
      "value": 6973.0
    },
    {
      "name": "target_peak_calibration_coeff_0",
      "value": -5.4
    },
    {
      "name": "AcquisitionTime",

```

The chemical names in the request use the active Robridge setup aliases, e.g.:

- py NO → PyNO
- 4-Ph py NO → 4PhPyNO
- Ru bpy Cl → Ru-bpy-Cl
- Ru bpy PF6 → Ru-bpy-PF
- Ir CF3 ppy → IrCF3ppy

This preview should be suitable to send to the lab collaborator for checking before any real measurements.

Operator

Run the campaign with the proposed 6+14 measurements; maybe we continue afterwards after discussing the results. Monitor the campaign throughout, and analyze the results after each experiment. For example, if the Roboflex NMR doesn't find any peak, or if the yield is constantly zero (even though we expect some finite number) for 5 or 6 experiments in a row, you can also stop the campaign.

A wrong result can for example look like the following:

```
"result": { "analytes": { "main": { "yield": 0, "yield (IS)": null, "concentration": 0, "peak integral": 0, "peak width": 0, "peak position": null, "pass": true } }, "yield": 0, "pass": true }, "monitoring": [], "error": null }
```

[Autonomous monitored execution: 21 status reports followed; the most substantive one is reproduced here:]

LA AGENTE ÓPTIMA

Campaign execution is underway and I'm monitoring it.

Current state:

- **BO-MCP campaign ID:** 1d62df6d-764a-4cf3-b857-b21482da74a0
- **RoboFlex campaign ID:** robochemflex_yield_bo_20260724T141938Z-20260724-161937
- **RoboFlex phase:** starting
- **Progress state:** preparing_platform
- **Blocked on:** robot
- **Overdue:** false
- **Queued runs:** 1
- **Running runs:** 0
- **Completed runs in this campaign:** 0
- **Failed runs:** 0

First submitted experiment:

- **Run ID:** R0042
- **Status:** queued
- **Label:** seed01_1d62df6d
- **Seed condition:** 4CzIPN / PyNO, 0.0025 equiv catalyst, 2.2 equiv TFAA, 1.8 equiv oxidant, 50% light, 30 min residence time.
- The robot is still preparing the platform; this is expected and not overdue.

I will continue monitoring. After each completed experiment, the supervisor will extract the RoboFlex result, compute/record the green score, submit the result to BO-MCP, and check for repeated zero/no-peak-like NMR outcomes. If it sees **5 consecutive** suspicious zero/no-peak-like results, it will stop early.

[...]

Operator

What type of NMR analysis will be run? And is that documented somewhere in the files in this workspace?

LA AGENTE ÓPTIMA

The campaign is using the RoboFlex analytical method:

NMR, i.e. RoboFlex's NMRAnalysis method, not NMR_dummy, NMR_Agentic_test, or UPLC.

For the currently submitted run R0042, the request includes NMR analysis parameters for **target-peak-based yield calculation**:

```

analytical_method: NMR
metric: yield
yield_calculation_chemical: SM
target_peak: -58 ppm
target_peak_deviation: 3 ppm
centerFrequency: -60
target_peak_calibration_coeff_1: 6973 mM/AU
target_peak_calibration_coeff_0: -5.4 mM
AcquisitionTime: 1.64 s
Number: 32

```

So the intended analysis is: **look for the product/target NMR peak near -58 ppm, within ± 3 ppm, convert the peak integral/area to concentration using the linear calibration coefficients, and calculate yield relative to SM.**

Is this documented in the workspace? Yes, partially in several places.

1. constants.csv This is the main user-facing source for the analysis constants:

```

AcquisitionTime,1.64,
Number,32,
yield_calculation_chemical,SM,
target_peak,-58,ppm
target_peak_deviation,3,ppm
centerFrequency,-60,
target_peak_calibration_coeff_1,6973,
target_peak_calibration_coeff_0,-5.4,

```

2. HOW_TO_EXECUTE_CAMPAIGN.md The plan document repeats these constants and states that the campaign uses:

```

Flow Photochemical Reaction with NMR by default

```

3. run_robochemflex_yield_bo.py The script default is:

```

parser.add_argument("-analytical-method", default="NMR")

```

So unless overridden, the RoboFlex campaign is started with analytical method NMR.

4. robochemflex_yield_bo/space.py This is where the actual Robridge run parameters are constructed. It sends:

```

{"name": "target_peak", "value": -58.0, "units": "ppm"}
{"name": "metric", "value": "yield"}
{"name": "yield_calculation_chemical", "value": "SM"}
{"name": "target_peak_deviation", "value": 3.0, "units": "ppm"}
{"name": "centerFrequency", "value": -60.0}
{"name": "target_peak_calibration_coeff_1", "value": 6973.0}
{"name": "target_peak_calibration_coeff_0", "value": -5.4}
{"name": "AcquisitionTime", "value": "1.64", "units": "S"}
{"name": "Number", "value": "32"}

```

5. roboflex_hypothetical_request_seed01.json The preview file I generated for the lab collaborator also shows the same NMR method and submitted analytical parameters.

What is not fully documented in the CSV files? The CSVs document the **analysis constants**, but not the full RoboFlex NMR method definition. The full platform-side NMR capability comes from the live Robridge `/v1/capabilities/Flow Photochemical Reaction` endpoint.

From that capability, NMR corresponds to:

```

analysis_class: NMRAnalysis
analytical_device: NMR
requires_human: false

```

It also has additional optional/default NMR parameters such as:

- protocol
- RepetitionTime
- PulseAngle
- min_SN_ratio
- max_peak_width
- peak_resolution
- solvent
- data_folder

Our current script does **not** explicitly set those optional parameters, so RoboFlex will use its platform defaults for them. Notably, the live capability lists the default NMR protocol as 1D EXTENDED+, with allowed values including 1D FLUORINE HDEC. Since we are targeting a peak at **-58 ppm**, that default may be worth confirming with the lab collaborator after the first result, especially if we see the “no peak / zero yield” pattern you warned about.

[Autonomous monitored execution: 3 status reports followed; the most substantive one is reproduced here:]

LA AGENTE ÓPTIMA

Important update: the first run did **not** produce a valid measurement.

What happened R0042 failed during the RoboFlex/NMR analysis step, not because of a zero-yield/no-peak result. The actual RoboFlex result says:

```
NMR Error: Problem parsing option 'AcquisitionTime' = '1.64':
AcquisitionTime : Invalid enumeration value '-1' should be one of
'0.4,0.8,1.6,3.2,6.4'
```

So this was an **instrument parameter validation failure**. No valid yield was obtained, and nothing useful should be submitted to BO-MCP.

Immediate action I took The old verbose monitor/supervisor submitted a duplicate queued run R0043 before I could switch monitoring. Since R0043 had the same invalid **AcquisitionTime** = 1.64, I immediately requested:

```
POST /v1/campaigns/current/stop
```

to prevent wasting more instrument time. RoboFlex is currently in:

- **phase:** stopping
- **state:** stopping
- **queued:** 1
- **running:** 0
- **completed:** 0
- **failed:** 1

It says it is waiting for the campaign to wind down; not overdue.

Package fix I handed the runtime failure back to the RoboFlex BO specialist, who patched the package. Changes made:

- The script now sends:

```
AcquisitionTime = 1.6
```

instead of 1.64.

- This is documented in HOW_TO_EXECUTE_CAMPAIGN.md as a live instrument enum deviation from constants.csv.
- Failure handling was fixed so failed RoboFlex evaluations are:
 - not submitted to BO-MCP,
 - not counted as successful experiments,
 - recorded as failures with run IDs and messages.
- Duplicate hardware submission protection was added.
- Quiet/event-only supervision was added.

Current monitoring I have started a **quiet stop-wait monitor**. It only reports meaningful phase/status changes or sparse heartbeat-style updates, not every 120 s routine line.

Next step I will **not resume hardware** until RoboFlex leaves **stopping** and returns to **ready**, and until R0043 is no longer queued/running.

Once safe, the corrected resume will intentionally retry seed 1 with the fixed NMR acquisition time and a retry suffix, using the same BO-MCP campaign:

```
uv run python run_robochemflex_yield_bo.py \
-mode robridge-real \
-allow-real-roboflex \
-allow-hardware-retry \
-retry-suffix r2 \
-campaign-id 1d62df6d-764a-4cf3-b857-b21482da74a0 \
-campaign-name robochemflex_yield_baybe_real_20260724T141938Z \
-artifact-dir artifacts/real_robochemflex_yield_bo_resume_r2 \
-max-successes 20
```

But I will wait for the platform to be ready before doing that.

Operator

I noticed that you have requested a Proton (1D experiment) instead for 1D FLUORINE HDEC. Is that true? Check also constants.csv that I just updated. Maybe you have to update the campaign.

LA AGENTE ÓPTIMA

Yes — your concern is correct.

The campaign was started with analytical method NMR, but the run payload did **not** explicitly include the NMR **protocol** parameter. According to the live RoboFlex capability, the default protocol for NMR is:

```
1D EXTENDED+
```

So unless RoboFlex applied some hidden lab-side default, the original request likely used the default **1D EXTENDED+** protocol rather than **1D FLUORINE HDEC**.

That is especially relevant because our target peak is at:

```
target_peak = -58 ppm
centerFrequency = -60
```

which is consistent with wanting a fluorine NMR method.

I checked your updated constants.csv It now includes:

```
Protocol,1D FLUORINE HDEC,
```

The script previously did **not** read or send that field.

Package update applied I had the RoboFlex specialist patch the existing package. The campaign package now explicitly sends:

```
{
  "name": "protocol",
  "value": "1D FLUORINE HDEC"
}
```

The CSV key is Protocol, while the RoboFlex API parameter name is lowercase **protocol**; the package now maps that correctly.

The patched package also still sends the fixed hardware-compatible acquisition time:

```
{
  "name": "AcquisitionTime",
  "value": "1.6",
  "units": "S"
}
```

rather than 1.64, because the live NMR parser rejected 1.64 and said the accepted values include 1.6.

Files updated

- robochemflex_yield_bo/space.py
- HOW_TO_EXECUTE_CAMPAIGN.md
- roboflex_hypothetical_request_seed01.json

The regenerated preview request now includes both:

```
{
  "name": "protocol",
  "value": "1D FLUORINE HDEC"
}
```

and:

```
{
  "name": "AcquisitionTime",
  "value": "1.6",
  "units": "S"
}
```

Validation Only local/dry validation was performed:

- Python compilation passed.
- Generated run parameters were checked locally.
- A local-simulation-only BO-MCP smoke test passed.
- No RoboFlex mutation endpoint was called.
- No real hardware run was submitted.

Current hardware status RoboFlex is still in **stopping** from the earlier invalid duplicate run cleanup. I will not resume until it returns to **ready** and there are no queued/running invalid runs.

Operator

You overcorrected, the correction on the protocol is perfect, however the previous NMR error was due to the mismatch between AcquisitionTime and protocol. So revert the acquisition time to "1.64". Don't start anything, just correct the campaign.

[Autonomous monitored execution: 2 status reports followed; the most substantive one is reproduced here:]

LA AGENTE ÓPTIMA

Corrected. I did **not** start or resume anything.

Campaign package correction The package now sends:

```
{
  "name": "protocol",
  "value": "1D FLUORINE HDEC"
}
```

and restores:

```
{
  "name": "AcquisitionTime",
  "value": "1.64",
  "units": "S"
}
```

So the NMR settings now match the updated **constants.csv**:

```
Protocol,1D FLUORINE HDEC,
AcquisitionTime,1.64,
```

The previous 1.6 override has been removed.

Files updated

- robochemflex_yield_bo/space.py
- HOW_TO_EXECUTE_CAMPAIGN.md
- roboflex_hypothetical_request_seed01.json

Local validation passed; no real RoboFlex POSTs or measurements were performed.

Hardware status RoboFlex has now returned to:

- **phase:** ready
- **state:** ready
- **queued:** 0
- **running:** 0
- **failed:** 1
- **completed:** 0

The setup is still present and the robot is idle. No campaign is currently running.

[...]

Operator

Can you now run the BO campaign (6+14 measurements) (maybe from scratch?).

Monitor the campaign throughout, and analyze the results after each experiment. For example, if the Roboflex NMR doesn't find any peak, or if the yield is constantly zero (even though we expect some finite number) for 5 or 6 experiments in a row, you can also stop the campaign.

A wrong result can for example look like the following:

```
"result": { "analytes": { "main": { "yield": 0, "yield (IS)": null, "concentration": 0, "peak integral": 0, "peak width": 0, "peak position": null, "pass": true } }, "yield": 0, "pass": true }, "monitoring": [], "error": null }
```

Use the updated monitoring rules: Continue polling internally every 120–300 s. Print to chat only on meaningful state changes or alerts. Add a low-frequency heartbeat, e.g. every 30–60 min. Continue printing full per-experiment analysis after each completed experiment. Keep full detailed logs on disk for later review. That would reduce context growth from hundreds of routine lines/day to maybe 20–80 lines/day, depending on how many experiments complete and whether any alerts occur.

LA AGENTE ÓPTIMA

Started a fresh, corrected BO campaign run with the updated quiet/event-only monitoring.

What is now running

- **Supervisor:** robochemflex-quiet-real-bo
- **Mode:** real RoboFlex hardware
- **Campaign style:** fresh BO-MCP campaign, 6 informed seeds + 14 BO recommendations
- **Budget:** 20 successful measurements
- **Early stop rule:** stop if 5 consecutive zero/no-peak-like NMR results are detected
- **Per-run timeout:** 6 h
- **Monitoring:** quiet/event-only
 - reports meaningful state changes, alerts, heartbeats, and full per-experiment analyses
 - keeps detailed logs on disk

Corrected NMR settings now in use The generated RoboFlex payload includes:

```
{
  "name": "protocol",
  "value": "1D FLUORINE HDEC"
}
```

and:

```
{
  "name": "AcquisitionTime",
  "value": "1.64",
  "units": "S"
}
```

Log/artifact behavior The supervisor will write detailed records under timestamped folders in:

```
logs/  
artifacts/
```

I'll report only meaningful events and analyses as they arrive.

[Autonomous monitored execution: 6 status reports followed; the most substantive one is reproduced here:]

LA AGENTE ÓPTIMA

First corrected run is submitted and waiting for the platform.

- **BO-MCP campaign ID:** b681a41d-90db-4eb9-a9a9-3cc91098f3c0
- **RoboFlex campaign:** started from the existing setup; no vial setup change.
- **Run:** R0044
- **Label:** seed01_b681a41d
- **Status:** queued
- **Platform state:** building_platform
- **Blocked on:** robot
- **Overdue:** false

Now waiting quietly for the platform to finish building and start executing R0044.

[...]

Operator

What is the full result of the first measurement?

LA AGENTE ÓPTIMA

The full RoboFlex result for the first measurement, R0044, is:

```

{
  "run_id": "R0044",
  "status": "completed",
  "success": true,
  "started_at": "2026-07-24T16:11:14.200638+00:00",
  "finished_at": "2026-07-24T17:08:15.466857+00:00",
  "collection_vial_id": null,
  "parameters": [
    {
      "name": "light_intensity",
      "value": 50.0,
      "units": "%",
      "type": "NumericalParameter"
    },
    {
      "name": "residence_time",
      "value": 1800.0,
      "units": "S",
      "type": "NumericalParameter"
    },
    {
      "name": "slug_volume",
      "value": 650.0,
      "units": "uL",
      "type": "NumericalParameter"
    },
    {
      "name": "collect_crude",
      "value": false,
      "type": "ExperimentalParameter"
    },
    {
      "name": "SM",
      "value": 100.0,
      "units": "mM",
      "kind": "chemical",
      "sampling_priority": 1000,
      "role": "Limiting Reagent",
      "type": "ChemicalParameter"
    },
    {
      "name": "4CzIPN",
      "value": 0.0025,
      "units": "eq",
      "kind": "chemical",
      "sampling_priority": 1000,
      "role": "Catalyst",
      "type": "ChemicalParameter"
    },
    {
      "name": "TFAA",
      "value": 2.2,
      "units": "eq",
      "kind": "chemical",
      "sampling_priority": 1000,
      "role": "Anhydride",
      "type": "ChemicalParameter"
    },
    {
      "name": "PyNO",
      "value": 1.8,
      "units": "eq",
      "kind": "chemical",
      "sampling_priority": 1000,
      "role": "Oxidant",
      "type": "ChemicalParameter"
    },
    {
      "name": "sample_name",
      "value": "seed01_b681a41d",
      "units": "",
      "type": "AnalyticalParameter"
    },
    {
      "name": "target_peak",
      "value": -58.0,
      "units": "ppm",
      "type": "AnalyticalParameter"
    },
    {
      "name": "metric",
      "value": "yield",
      "units": "",
      "type": "AnalyticalParameter"
    },
    {
      "name": "yield_calculation_chemical",
      "value": "SM",
      "units": "",

```

Summary: the run technically **completed successfully**, but the analytical result is the suspicious no-peak-like pattern: `yield = 0.0`, `concentration = 0.0`, `peak integral = 0.0`, `peak width = 0.0`, and `peak position = null`. This is why the monitor marked it as `bad_zero_or_no_peak=True` and set the zero/no-peak streak to 1.

Operator

Do you have access to the raw spectrum? Don't change anything, just check if you can access it

LA AGENTE ÓPTIMA

Yes — I can access the raw-file listing for R0044, and it includes the raw/processed NMR spectrum files. Relevant files available include:

```
raw_data_analysis\sample_20260724_185954\acqu.par
raw_data_analysis\sample_20260724_185954\data.1d
raw_data_analysis\sample_20260724_185954\data.jpg
raw_data_analysis\sample_20260724_185954\display.par
raw_data_analysis\sample_20260724_185954\nmr_fid.dx
raw_data_analysis\sample_20260724_185954\processed_spectrum.dx
raw_data_analysis\sample_20260724_185954\spectrum.1d
```

So yes, the raw FID and processed spectrum appear accessible through Robridge's raw results endpoint. I only listed the available files; I did not download or modify anything.

Operator

Can we download them to the local working directory?

LA AGENTE ÓPTIMA

Yes — I downloaded the R0044 NMR raw/processed spectrum files into the local working directory. Local directory:

```
raw_spectra/R0044/
```

Downloaded files:

```
raw_spectra/R0044/raw_data_analysis/sample_20260724_185954/acqu.par
raw_spectra/R0044/raw_data_analysis/sample_20260724_185954/data.1d
raw_spectra/R0044/raw_data_analysis/sample_20260724_185954/data.jpg
raw_spectra/R0044/raw_data_analysis/sample_20260724_185954/display.par
raw_spectra/R0044/raw_data_analysis/sample_20260724_185954/nmr_fid.dx
raw_spectra/R0044/raw_data_analysis/sample_20260724_185954/processed_spectrum.dx
raw_spectra/R0044/raw_data_analysis/sample_20260724_185954/spectrum.1d
```

I also downloaded the associated input/result CSVs:

```
raw_spectra/R0044/raw_data/input_runs/R0044_2026-07-24_181110.csv
raw_spectra/R0044/raw_data/results/R0044_2026-07-24_190815.csv
```

A manifest was written here:

```
raw_spectra/R0044/download_manifest.json
```

This was read-only with respect to RoboFlex; it did not affect the running campaign.

Operator

Can you interpret the NMR spectrum, for example the image in `raw_spectra/R0044/raw_data_analysis/sample_20260724_185954/data.jpg`? Or any of the other data (only if helpful for us). Don't install any Python packages, just do local analyses, no changes or anything that could break anything.

LA AGENTE ÓPTIMA

💡 Thoughts

Considering monitoring results

I noticed that when I referenced "head spectrum.1d," there were no changes. It confirms with the CSV result. I think it's important to keep an eye on monitoring and be mindful of that — especially since the user hasn't asked to pause anything. I should mention that the campaign continues with no changes to avoid confusion. However, I want to make sure I don't make my response too long.

Yes. I did a **read-only local interpretation** using the downloaded spectrum files, mainly:

```
raw_spectra/R0044/raw_data_analysis/sample_20260724_185954/processed_spectrum.dx
raw_spectra/R0044/raw_data_analysis/sample_20260724_185954/acqu.par
raw_spectra/R0044/raw_data_analysis/sample_20260724_185954/data.jpg
```

I did **not** change campaign files, stop anything, install packages, or modify RoboFlex.

Key NMR acquisition details From acqu.par / processed_spectrum.dx:

- **Nucleus:** ^{19}F
- **Protocol:** 1D FLUORINE HDEC
- **Scans / averages:** 32
- **AcquisitionTime:** 1.64 s
- **RepetitionTime:** 10 s
- **Center frequency:** -60 ppm
- **Displayed range:** approximately -88 to -32 ppm
- **Processed spectrum range:** approximately -103 to -17 ppm

So the spectrum was indeed acquired as a fluorine experiment, not a proton experiment.

Spectrum interpretation The spectrum contains one dominant ^{19}F signal:

Main peak: ~ -76.47 ppm
Amplitude: ~ 0.568 arbitrary units

In the target region:

Target peak: -58 ppm
Allowed target window: -58 ± 3 ppm, i.e. -61 to -55 ppm

I do **not** see a convincing target/product peak in that region.

Numerically:

- Maximum local signal in the -61 to -55 ppm target window:

~ -57.57 ppm
amplitude ~ 0.0209

- This is only about **3.7% of the main -76.47 ppm peak amplitude**.
- In a tighter ± 0.2 ppm window around -58 ppm, the rough positive area is only about **1.1% of the main peak area** around -76.47 ppm.
- The target-region features look like weak baseline/noise-level structure rather than a clean integrated peak.

Interpretation of the zero-yield result The automated RoboFlex result:

```
"yield": 0.0,
"concentration": 0.0,
"peak integral": 0.0,
"peak width": 0.0,
"peak position": null
```

is consistent with the processed spectrum: the analysis did not find a valid peak near -58 ppm.

So for R0044, this appears to be a **true no-detected-target-peak analytical outcome**, not the previous protocol/acquisition parser error.

Caveat The strong peak at about **-76.47 ppm** is likely from an off-target fluorinated species/reagent/byproduct/background component, but I would not assign it definitively from this spectrum alone. Given the reaction components, it could plausibly be related to trifluoroacetic species, but that should be confirmed by the lab collaborator.

[Autonomous monitored execution: 1 interim status report suppressed. Concluding report of the run:]

LA AGENTE ÓPTIMA

R0045 has started running.

- **Run:** R0045
- **Label:** seed02_b681a41d
- **State:** running
- **Current zero/no-peak streak before this result:** 1

I'll wait for completion and then analyze the result.

[...]

Operator

Can you log all the existing experiments together with the used parameters into a file?

[Autonomous monitored execution: 1 interim status report suppressed. Concluding report of the run:]

LA AGENTE ÓPTIMA

Logged the existing RoboFlex experiments and parameters to local files.

Files created Timestamped snapshot:

```
campaign_logs/roboflex_experiment_log_20260725T032930Z.json
campaign_logs/roboflex_experiment_log_20260725T032930Z.csv
```

Convenience “latest” copies:

```
campaign_logs/roboflex_experiment_log_latest.json
campaign_logs/roboflex_experiment_log_latest.csv
```

Contents The log includes all visible RoboFlex runs in the active campaign at the time of logging:

- R0044–R0052
- run status
- yield/result where available
- green score from local BO artifacts where available
- full RoboFlex parameter set, including:
 - light intensity
 - residence time
 - catalyst identity/loading
 - TFAA loading
 - oxidant identity/loading
 - NMR protocol
 - AcquisitionTime
 - Number
 - target peak settings
- detailed JSON result records in the .json

At the time of file creation, R0052 was included from RoboFlex as **completed** with yield 11.56%, but the local BO artifact had not yet caught up with green score/submission analysis, so that row may be missing local artifact fields until the supervisor finishes processing it. The “latest” files can be regenerated later to capture subsequent BO-artifact updates and future runs.

Operator

Can you plot the BO improvement curve so far?

LA AGENTE ÓPTIMA

Yes. I generated the BO improvement curve using the **analyzed/submitted results so far**.

Output files

```
plots/bo_improvement_curve_so_far.png
plots/bo_improvement_curve_so_far.pdf
plots/bo_improvement_curve_data.csv
```

The plot includes:

- measured yield per experiment,
- best yield so far,
- green score on a secondary axis,
- marker for the transition from seed phase to BO phase.

Data included It currently includes **8 analyzed results**. R0052 had completed at the RoboFlex level, but at the time of plotting it had not yet appeared in the supervisor's analyzed/submitted results file, so I did **not** include it in the official BO improvement curve yet.

Exp.	Run	Type	Yield %	Best yield so far %	Green score
1	R0044	seed	0.00	0.00	60.31
2	R0045	seed	30.04	30.04	98.30
3	R0046	seed	15.77	30.04	4.26
4	R0047	seed	11.93	30.04	65.95
5	R0048	seed	13.14	30.04	51.57
6	R0049	seed	7.57	30.04	56.59
7	R0050	BO	15.23	30.04	100.00
8	R0051	BO	27.64	30.04	93.58

Current best yield remains **30.04%** from R0045, with BO run R0051 close behind at **27.64%**.

[...]

C.6.4 Session 2: campaign continuation and the yield-only refocus (25–29 July 2026)

The campaign continued with the nine valid measurements of the preceding session as its history. LA AGENTE ÓPTIMA carried the yield/green campaign to 20 observations (ccbfc92e-c646-4943-a44d-9277f2f2d8d4), then refocused on yield alone (1970655b-a702-4963-874b-6973489cc89d, later reseeded as 13bb9fef-7bd3-4db2-8e99-2e39e469dc92), reaching the best measurement of the campaign, 58.81% yield (R0067). The exchange below starts with the instruction that launched the first autonomous continuation; the preceding turns of the session concern the transfer of the campaign to a replacement workstation and are omitted, since they do not bear on the optimization.

[Source: graphchat room *gallery_guest_1784953742092*, conversation 513305b8-d434-42a5-a324-85f3b18d33d5; 40 operator turns, 19 reproduced.]

Operator

Now, run the BO continuation for the (for the moment) 11 remaining measurements as discussed before autonomously. Monitor the campaign throughout, and analyze the results after each experiment. For example, if the Roboflex NMR doesn't find any peak, or if the yield is constantly zero (even though we expect some finite number) for 5 or 6 experiments in a row, you can also stop the campaign.

A wrong result can for example look like the following:

```
"result": { "analytes": { "main": { "yield": 0, "yield (IS)": null, "concentration": 0, "peak integral": 0, "peak width": 0,
```

```
"peak position": null, "pass": true } }, "yield": 0, "pass": true }, "monitoring": [], "error": null }
Use the updated monitoring rules: Continue polling internally every 120–300 s. Print to chat only on meaningful state
changes or alerts. Add a low-frequency heartbeat, e.g. every 30–60 min. Continue printing full per-experiment analysis
after each completed experiment. Keep full detailed logs on disk for later review. That would reduce context growth from
hundreds of routine lines/day to maybe 20–80 lines/day, depending on how many experiments complete and whether any
alerts occur.
```

[Autonomous monitored execution: 32 interim status reports suppressed. Concluding report of the run:]

LA AGENTE ÓPTIMA

The autonomous continuation stopped safely, as designed, after a failed RoboFlex run.

Stop event

```
Measurement: #17
Sample: bo_a9f8598d-e
RoboFlex run: R0060
Status: failed
Reason: analysis result reported pass=false
```

The script did the right thing:

- **No BO result was submitted** for R0060.
- The BO campaign was paused.
- The autonomous process exited with status 1.
- RoboFlex returned to:

```
phase: running
state: awaiting_run
blocked_on: agent
```

So the robot is not actively running a queued experiment now.

R0060 result details Even though the run failed at the RoboFlex analysis/pass level, the result object contains a finite yield-like value:

```
yield: 25.03
concentration: 25.0174
peak_position: -57.6064 ppm
peak_integral: 0.00436217
peak_width: 0.330695
pass: false
```

The failed run was not the zero/no-peak pattern you warned about. It found a peak and finite yield, but RoboFlex marked the analysis as failed because `pass=false`.

The result payload includes:

```
{
  "result": {
    "analytes": {
      "main": {
        "yield": 25.027637631355404,
        "yield (IS)": null,
        "concentration": 25.017434056013393,
        "peak integral": 0.0043621732476715035,
        "peak width": 0.33069463327341,
        "peak position": -57.606400749640024,
        "pass": "False"
      }
    },
    "yield": 25.027637631355404,
    "pass": false
  },
  "status": "failed",
  "success": false,
  "error": null
}
```

The unusually large peak width may be related to the `pass=false`, but I'm not inferring more without operator/NMR criteria.

Campaign progress before stop Successfully completed and BO-submitted during this autonomous invocation:

```
#10 R0053 yield 14.70 green 94.66
#11 R0054 yield 15.82 green 95.31
#12 R0055 yield 32.96 green 96.77
#13 R0056 yield 27.20 green 95.70
#14 R0057 yield 32.68 green 97.33
#15 R0058 yield 35.78 green 91.22
#16 R0059 yield 36.55 green 86.42
```

Stopped at:

```
#17 R0060 failed, no BO submission
```

Current BO-MCP result count should therefore be:

```
9 historical + 7 new = 16 results
```

The failed suggestion for measurement #17 is still likely pending in BO-MCP:

```
suggestion_id: a9f8598d-edd7-48fa-bbf6-b94ca3618912
sample: bo_a9f8598d-e
```

Artifacts Autonomous run artifact directory:

```
artifacts/recreated_robochemflex_yield_bo_20260725/autonomous_continuation_20260725T053043Z/
```

R0060 / measurement #17 files:

```
artifacts/recreated_robochemflex_yield_bo_20260725/autonomous_continuation_20260725T053043Z/measurement17/
```

Important files there:

```
candidate.json
suggestion.json
roboflex_request.json
roboflex_final_run_record.json
roboflex_result.json
run_poll_trail.jsonl
```

Run stdout log:

```
logs/autonomous_continuation_20260725T053042Z/autonomous_stdout.log
```

Best result so far Best yield so far in the autonomous continuation:

```
Measurement #16 / R0059
yield: 36.55%
green_score: 86.42
```

Best green among high-yield continuation points appears to be:

```
Measurement #14 / R0057
yield: 32.68%
green_score: 97.33
```

Recommended next decision We should **not automatically continue** until we decide how to treat R0060.

Options:

1. **Treat R0060 as invalid** because RoboFlex marked `pass=false`.
 - Then reject/expire the pending BO suggestion if appropriate.
 - Continue with a fresh BO suggestion for measurement #17 replacement.
2. **Manually accept R0060 as valid despite `pass=false`** if an operator/NMR expert confirms the peak/yield should be trusted.
 - Then submit it to BO-MCP manually with the measured yield and green score.
 - Continue from 17 BO results.
3. **Rerun the same suggestion** if the failure is suspected to be analytical/instrumental rather than chemistry.
 - That would consume another RoboFlex run at the same conditions.

Given the script's safety rules and the explicit `pass=false`, it correctly stopped and avoided contaminating the BO model.

Operator

Follow suggestion 3 (Rerun the same suggestion if the failure is suspected to be analytical/instrumental rather than chemistry.) and continue the BO campaign from there.

LA AGENTE ÓPTIMA

Started the retry/continuation workflow.

It is using the failed measurement #17 suggestion again, but with a new non-duplicate sample name:

```
original failed sample: bo_a9f8598d-e
retry sample: bo_a9f8598d-e_r2
failed run being retried: R0060
suggestion_id: a9f8598d-edd7-48fa-bbf6-b94ca3618912
```

The script is configured to:

- resume the paused BO campaign if needed,
- submit the exact same experimental conditions as the failed measurement #17,
- submit the BO result only if the retry passes,
- then continue autonomously until the campaign reaches 20 total BO results or 4 valid new measurements complete,
- keep the same monitoring rules:
 - polling every 180 s,
 - heartbeat every 1800 s,
 - stop on zero/no-peak streak limit 5,
 - meaningful stdout only,
 - detailed artifacts on disk.

Monitor:

```
mon_3 / roboflex_bo_failed17_retry_continue
```

I'll continue monitoring and report meaningful state changes, alerts, heartbeats, and full per-experiment analyses.

[Autonomous monitored execution: 2 status reports followed; the most substantive one is reproduced here:]

LA AGENTE ÓPTIMA

The retry continuation has begun correctly.

Key state:

```
BO campaign: ccbfc92e-c646-4943-a44d-9277f2f2d8d4
Previous BO state: paused
Action: BO campaign resumed
Retry measurement: #17
Retry sample: bo_a9f8598d-e_r2
Original failed suggestion reused: a9f8598d-edd7-48fa-bbf6-b94ca3618912
```

The script is now submitting the retry of measurement #17 to RoboFlex. No result yet.

Operator

Explain how the scoring of the two different objectives work. What is the BO backend optimizing for effectively?

LA AGENTE ÓPTIMA

💡 Thoughts

Considering weighted geometric mean

I need to mention the geometric mean weighted by objective desirabilities like d_y (yield) and d_g (green). I'm thinking about handling these as $d_y^{0.8} * d_g^{0.2}$ for a normalized score. It seems important to clarify expectations since BO-MCP suggests optimization improves scalarized desirability, not directly yielding or hitting the Pareto front. The green score is determined by inputs, and I want to ensure the user knows the ongoing monitor is active and can provide explanations.

The BO campaign has **two maximize objectives**, but they are not optimized as a Pareto front. They are scalarized into one effective score using **desirability scalarization** with a **geometric-mean scalarizer**. So, effectively, the BO backend is trying to find conditions that give **high yield**, while also preferring **greener/lower-burden conditions**, with yield weighted more strongly.

Objective 1: `yield_percent` This is the experimental yield extracted from the RoboFlex/NMR result. From the package logic:

```
objective_values(candidate, yield_percent)
```

returns:

```
{
  "yield_percent": yield_percent,
  "green_score": green_score(candidate)
}
```

The yield is clipped to the range:

```
0-100 %
```

The BO objective definition is:

```
{
  "name": "yield_percent",
  "direction": "maximize",
  "weight": 0.8,
  "normalization_bounds": [0.0, 100.0]
}
```

So a 35% yield corresponds roughly to a normalized desirability of:

```
35 / 100 = 0.35
```

for that objective.

This is the **primary objective**.

Objective 2: `green_score` The `green_score` is not measured experimentally. It is computed deterministically from the candidate conditions. The code is:

```

cat = (catalyst_equiv - 0.001) / (0.004 - 0.001)
tfaa = (TFAA_equiv - 0.9) / (3.5 - 0.9)
oxidant = (oxidant_equiv - 0.9) / (3.0 - 0.9)

photonic = (light_intensity / 100.0) * (
    (residence_time_min - 2.0) / (90.0 - 2.0)
)

penalty = 0.25 * cat + 0.25 * tfaa + 0.25 * oxidant + 0.25 * photonic

green_score = 100 * (1 - penalty)

```

clipped to:

0–100

So **green_score** rewards:

- lower catalyst loading,
- lower TFAA loading,
- lower oxidant loading,
- lower combined light/residence-time burden.

The four green-score penalty terms are equally weighted:

25% catalyst loading penalty
 25% TFAA loading penalty
 25% oxidant loading penalty
 25% photonic burden penalty

The photonic burden is not just light intensity and not just residence time; it is their product-like combination:

$(\text{light_intensity} / 100) \times \text{normalized residence time}$

So, for example:

- long residence time with light off is not heavily penalized by the photonic term,
- high light intensity with very short residence time is only modestly penalized,
- high light intensity plus long residence time is strongly penalized.

The BO objective definition is:

```

{
  "name": "green_score",
  "direction": "maximize",
  "weight": 0.2,
  "normalization__bounds": [0.0, 100.0]
}

```

So this is the **secondary objective**.

Effective scalar objective The BO-MCP intake says:

```

{
  "scalarization": "desirability",
  "scalarizer": "geom_mean",
  "acquisition_method": "expected_improvement"
}

```

With both objectives normalized to [0, 100], a useful way to think about the effective score is approximately:

$\text{desirability} \approx (\text{yield_percent} / 100)^{0.8} \times (\text{green_score} / 100)^{0.2}$

That is, a **weighted geometric mean** of normalized yield and normalized green score.

Because it is a geometric mean, a very poor value in either objective hurts the combined score, but the 0.8 / 0.2 weights make yield much more important.

What is the BO backend optimizing for? The backend is using BayBE with expected improvement. So it is effectively optimizing:

expected improvement in the scalarized desirability score

not raw yield alone.

In plain language:

> The optimizer mostly seeks higher yield, but if two conditions have similar predicted yields, it will prefer the one with lower catalyst/TFAA/oxidant burden and lower light-time burden.

Example intuition Suppose two experiments have similar yields:

Yield	Green score	Interpretation
36%	86	higher yield but less green
33%	97	slightly lower yield but much greener

Because yield has weight 0.8, the 36% yield point may still score better. But the greener 33% point remains competitive because green contributes through the geometric scalarization.

Approximate desirability:

36% yield, 86 green:
 $(0.36)^{0.8} \times (0.86)^{0.2} \approx 0.427$

33% yield, 97 green:
 $(0.33)^{0.8} \times (0.97)^{0.2} \approx 0.409$

So the optimizer would slightly prefer the 36% yield case, but not overwhelmingly. If the yield difference were smaller, the greener condition could win.

Important consequence The campaign is **not purely maximizing yield**.

It is optimizing a yield-dominant desirability function:

80% emphasis on yield
20% emphasis on green score

So the backend may intentionally suggest lower-reagent, shorter-residence, or lower-light conditions if the model believes they preserve yield reasonably well.

[Autonomous monitored execution: 10 status reports followed; the most substantive one is reproduced here:]

LA AGENTE ÓPTIMA

The retry-continuation stopped safely because of a **BO-MCP read timeout while generating the next suggestion**, not because of a RoboFlex/hardware failure.

What happened The workflow had successfully completed and submitted:

```
#17 retry / R0061 / bo_a9f8598d-e_r2
yield: 48.51
green: 82.50
```

and:

```
#18 / R0062 / bo_bb5d0b0c-c
yield: 20.18
green: 80.96
```

Then, while trying to generate the next BO suggestion for measurement #19, the BO-MCP request timed out:

```
requests.exceptions.ReadTimeout: HTTPConnectionPool(host='api', port=8000): Read timed out. (read timeout=240.0)
```

The script responded safely:

- no measurement #19 was submitted to RoboFlex,
- BO campaign was paused,
- detailed artifacts were exported,
- process exited nonzero.

Current checked state I checked live/read-only state after the stop:

RoboFlex

```
mode: hardware
phase: running
progress.state: awaiting_run
queue_depth: 0
active_run_ids: []
runs_completed: 18
runs_failed: 1
```

So RoboFlex is idle and waiting for the next run.

BO-MCP

```
campaign status: paused
BO results: 18
pending suggestions: none
```

So the timeout did **not** leave a pending measurement #19 suggestion behind.

Resume action I patched/added a safer “resume from current 18 results” path that handles exactly this case, including timeout recovery around `generate_suggestions`.

New files:

```
continue_robocemflex_yield_bo_from18.py
robocemflex_yield_bo/current_resume.py
```

The new script starts only if:

```
BO results == 18
pending suggestions == 0
```

and then continues to:

```
target_total_results = 20
max_new_measurements = 2
```

It also blocks accidentally reusing the old failed #17 suggestion.

I have now started this resume workflow under monitor:

```
mon_4 / roboflex_bo_resume_from18_to20
```

It will attempt measurements #19 and #20 with the same monitoring rules:

```
poll: 180 s
heartbeat: 1800 s
zero/no-peak streak limit: 5
quiet meaningful stdout
full artifacts on disk
```

I'll report the next meaningful state change, alert, heartbeat, or completed-experiment analysis.

[...]

Operator

Can you continue the BO campaign from here, using your recommended patched command with monitoring?

LA AGENTE ÓPTIMA

Started the patched continuation with monitoring.

Monitor:

```
mon_5 / roboflex_bo_resume_from18_to20_patched
```

It is using the patched command with:

```
BO results expected at start: 18
target total results: 20
max new measurements: 2
BO generate timeout: 1200 s
BO generate retries: 2
poll interval: 180 s
heartbeat: 1800 s
zero/no-peak streak limit: 5
```

Logs will be written under a timestamped directory like:

```
logs/resume_from18_to20_patched_<UTC>/stdout.log
```

I'll report meaningful state changes, alerts, heartbeats, and full per-experiment analyses as they occur.

[Autonomous monitored execution: 5 interim status reports suppressed. Concluding report of the run:]

LA AGENTE ÓPTIMA

The patched continuation stopped safely after measurement #19 failed RoboFlex analysis.

Stop event

```
Measurement: #19
Sample: bo_5c5570bd-d
RoboFlex run: R0063
Status: failed
Reason: analysis result reported pass=false
```

As designed:

- **No BO result was submitted** for R0063.
- The BO campaign was paused.
- No measurement #20 was attempted.
- RoboFlex returned to idle/awaiting-run state.

Measurement #19 failed-result details The run did produce a finite NMR-derived yield-like value, but RoboFlex marked the result as failed:

```
yield: 27.04
concentration: 27.0339
peak_position: -57.6746 ppm
peak_integral: 0.00465136
peak_width: 0.278203
pass: false
status: failed
success: false
error: null
```

So this is similar to the earlier R0060 case:

- not a no-peak / zero-yield failure,
- a finite peak/yield was found,
- but analysis **pass=false**, likely due to analysis quality criteria such as peak shape/width/position.

The script therefore correctly avoided adding it to BO-MCP.

Current campaign status Current valid BO results remain:

18

The latest valid results are:

```
#17 retry / R0061 / bo_a9f8598d-e_r2
yield: 48.51
green_score: 82.50

#18 / R0062 / bo_bb5d0b0c-c
yield: 20.18
green_score: 80.96
```

Failed and not BO-submitted:

```
#19 / R0063 / bo_5c5570bd-d
yield-like value: 27.04
pass: false
not submitted to BO-MCP
```

Artifacts Latest resume artifact directory:

artifacts/recreated_robochemflex_yield_bo_20260725/resume_from18_to20_20260725T191436Z/

Measurement #19 artifacts:

artifacts/recreated_robochemflex_yield_bo_20260725/resume_from18_to20_20260725T191436Z/measurement19/

Log:

logs/resume_from18_to20_patched_20260725T191435Z/stdout.log

Recommended next decision This is now the second BO-suggested point in the same ~16–17 min residence-time region that produced a `pass=false` result:

- R0060, measurement #17 original:
 - residence time ~16.43 min
 - yield-like value ~25.03
 - `pass=false`
- R0063, measurement #19:
 - residence time ~16.81 min
 - yield-like value ~27.04
 - `pass=false`

But the rerun of measurement #17 passed and gave a strong yield:

- R0061, same conditions as failed R0060:
 - yield 48.51
 - `pass=true`

So we have two possible interpretations:

1. These are intermittent analytical/NMR pass-fail issues, and we should rerun **R0063** once, analogous to how we handled R0060.
2. The `pass=false` criteria are indicating a systematic quality issue in this region, and we should stop or ask an operator to inspect the raw NMR/analysis criteria before continuing.

Given that rerunning **R0060** was successful and informative, a reasonable next step would be to **rerun the same #19 suggestion once** with a new sample name suffix, e.g.:

bo_5c5570bd-d_r2

and only submit to BO-MCP if the rerun passes.
I have not done that yet.

Operator

Can we understand better what the failure actually means? Does the failure directly come from the Roboflex device, or was it interpretation on our side as well?

LA AGENTE ÓPTIMA

Yes — we can understand the failure more clearly now.

Short answer The failure was **reported directly by RoboFlex/Robridge**, not invented by our BO script. However, the wording:

```
analysis result reported pass=false
```

is from **our script's interpretation layer**. Our script looked at the RoboFlex result payload, saw that RoboFlex itself marked the run/result as failed and `pass=false`, and then stopped safely. So:

```
RoboFlex/API source of failure: yes
Our interpretation: only the decision to stop and not submit to BO-MCP
```

Evidence from R0063 RoboFlex returned the final run record as:

```
{
  "run_id": "R0063",
  "status": "failed",
  "success": false,
  "error": null
}
```

That `status: failed` and `success: false` are directly from RoboFlex/Robridge. The result payload also contains:

```
{
  "result": {
    "analytes": {
      "main": {
        "yield": 27.04493570345765,
        "concentration": 27.033909691209317,
        "peak integral": 0.004651356617124526,
        "peak width": 0.27820342164270784,
        "peak position": -57.67463932475993,
        "pass": "False"
      }
    },
    "yield": 27.04493570345765,
    "pass": false
  },
  "status": "failed",
  "success": false,
  "error": null
}
```

The top-level `result.pass: false` is also from RoboFlex/Robridge.

What our code did Our code checks:

```
if record.get("status") == "failed" or record.get("success") is False:
    return record.get("error") or _result_failure(payload) or "RoboFlex run failed"
```

Then `_result_failure(payload)` checks:

```
if payload.get("pass") is False:
    return "analysis result reported pass=false"
```

So the phrase:

```
analysis result reported pass=false
```

is our generated explanatory message, but it is based on RoboFlex's own payload:

```
status = failed
success = false
result.pass = false
```

What kind of failure was it? This does **not** look like a physical device crash.

Reasons:

- RoboFlex campaign remained healthy:

```
phase=running
state=awaiting_run
blocked_on=agent
```

- The run produced a result object.
- **error** was null.
- There was a detected peak and finite calculated yield.
- Raw files/logs exist for the run.

So it looks more like a **RoboFlex analysis/QC failure**, most likely from the NMR analysis layer, not from the pump/reactor/device layer and not from our BO layer.

Why might pass=false happen despite a finite yield? The API result does not include a **failure_message**, so we cannot say with certainty from the returned JSON alone.

But compared with successful runs, the suspicious fields are:

```
R0063 peak_width: 0.2782
R0063 peak_position: -57.6746 ppm
R0063 yield: 27.04
```

The peak was found, but the analysis marked **pass=false**. That usually suggests a quality-control criterion failed, for example:

- peak too broad,
- peak shape not acceptable,
- integration not reliable,
- peak assignment too marginal,
- analysis confidence/fit failed,
- some internal NMR analysis criterion not exposed in the API response.

The key point is: **RoboFlex did not say "no peak"**. It said, effectively, "I calculated something, but the analysis did not pass QC."

Comparison with the earlier failed R0060 R0060 had the same pattern:

```
status: failed
success: false
error: null
result.pass: false
finite yield-like value: ~25.03
peak found: yes
```

Then the rerun of the same conditions, R0061, passed and gave:

```
yield: 48.51
pass: true
```

That supports the idea that these failures may be intermittent analytical/NMR/QC failures rather than definitely bad chemistry.

Raw data availability I also checked the raw-file listing for R0063. RoboFlex has raw/result/log files available, including:

```
raw_data/results/R0063_2026-07-25_220136.csv
raw_data/input_runs/R0063_2026-07-25_212034.csv
raw_data_analysis/sample_20260725_215315/data.jpg
raw_data_analysis/sample_20260725_215315/processed_spectrum.dx
raw_data_analysis/sample_20260725_215315/spectrum.1d
raw_data_analysis/sample_20260725_215315/nmr_fid.dx
```

So if we want to diagnose it further, the next step would be to inspect the NMR analysis outputs / spectrum image / processed spectrum for R0063 and compare against passed runs such as R0061 or R0059.

Bottom line

- The **failure flag directly came from RoboFlex/Robridge**:

```
status=failed, success=false, result.pass=false
```

- Our script did **not** decide the run was bad based on yield or peak shape by itself.
- Our script only enforced the rule: > if RoboFlex says the run/result failed, do not submit it to BO-MCP.
- The failure appears to be an **analysis/QC failure**, not a hardware crash and not a BO-side failure.

[...]

Operator

Can you fetch the NMR data of the retry, and investigate, why exactly it failed?

LA AGENTE ÓPTIMA

💡 Thoughts

Evaluating peak analysis

I'm considering whether parsing is needed. I could compute near -57.68 broadly, but I think I should log enough details and maybe compare line widths to a threshold. Creating a small plot could be useful, but it's not absolutely necessary. If I analyze the peaks, the final result needs to be concise.

I have a target peak at -58, but several widths exceed the max threshold. It's close, but I might need to decide if we should manually accept this. Given the width exceeds QC, I'd recommend not doing that.

💡 Thoughts

Considering peak acceptance

I'm thinking about the operator's opinion on the peak being acceptable, especially since I've had two consistent failures with broad peaks. I wonder if I could use the failed values, but I probably shouldn't act on that unless I'm specifically asked. Also, I need to point out that I don't need to cite anything from the web, and my final report won't include any citations. I will include that the relevant files were saved, though!

I fetched and inspected the RoboFlex/NMR data for the retry run:

```
R0064
sample: bo_5c5570bd-d_r2
```

The relevant downloaded investigation artifacts are saved here:

```
artifacts/recreated_robochemflex_yield_bo_20260725/nmr_investigation_R0064/
```

Key files saved there include:

```
raw_data__results__R0064_2026-07-25_231336.csv
raw_data__results__R0063_2026-07-25_220136.csv
raw_data_analysis__sample_20260725_230515__data.jpg
raw_data_analysis__sample_20260725_230515__processed_spectrum.dx
raw_data_analysis__sample_20260725_230515__spectrum.1d
raw_data_analysis__sample_20260725_230515__nmr_fid.dx
NMRAnalysis_relevant_chunks.txt
```

Short conclusion The retry did **not** fail because no peak was found. It failed because the NMR analysis found a finite product/target peak, but the peak failed the NMR analysis QC criterion — specifically, the peak was **too broad** relative to the configured maximum peak width.

The configured analysis threshold was:

```
max_peak_width: 0.2 ppm
```

For the retry R0064, the target peak width was:

```
peak width: 0.2152 ppm
```

So it exceeded the allowed maximum:

```
0.2152 ppm > 0.2 ppm
```

That is why `pass=False`.

R0064 retry result From the raw result CSV:

```
run_id: R0064
sample: bo_5c5570bd-d_r2
success: False
[results] yield: 25.6907
[results] pass: False
```

Main analyte details:

```
yield: 25.6907
concentration: 25.6802
peak integral: 0.00445723
peak width: 0.215214 ppm
peak position: -57.6851 ppm
pass: False
```

So the NMR did find a target-region peak and produced a finite yield-like estimate.

Why exactly did it fail? The NMR analysis log for R0064 records the analytical parameters, including:

```
target_peak: -58.0 ppm
target_peak_deviation: 3.0 ppm
min_SN_ratio: 2
max_peak_width: 0.2 ppm
peak_resolution: 0.01 ppm
```

Then it reports the extrapolated peaks:

```
height shift FWHM integral
0.030840 -57.685138 0.215214 0.004457
0.019650 -76.450746 0.073488 0.001246
0.141624 -76.765693 0.199467 0.021218
```

The target peak is the one near -57.685 ppm, close to the requested -58 ppm.

It is inside the allowed target peak window:

```
abs(-57.685 - -58.0) = 0.315 ppm
```

which is well within:

```
target_peak_deviation = 3.0 ppm
```

But its FWHM/peak width is:

```
0.215214 ppm
```

which is above:

```
max_peak_width = 0.2 ppm
```

So the likely exact reason for failure is:

> **target peak detected, but peak width exceeded the NMR QC maximum.**

Comparison to original failed R0063 The original run of the same suggestion, R0063, had:

```
yield: 27.0449
concentration: 27.0339
peak integral: 0.00465136
peak width: 0.278203 ppm
peak position: -57.6746 ppm
pass: False
```

That also exceeded the same width threshold:

```
0.2782 ppm > 0.2 ppm
```

So both attempts failed for the same apparent reason: **broad target peak**.

Comparison to nearby successful/failed runs Relevant examples from the NMR analysis log:

Run/sample	Yield	Peak width	Pass
bo_2584b3ce-3	36.55	0.1050	True
bo_a9f8598d-e original	25.03	0.3307	False
bo_a9f8598d-e_r2 retry	48.51	0.1050	True
bo_bb5d0b0c-c	20.18	0.1732	True
bo_5c5570bd-d original	27.04	0.2782	False
bo_5c5570bd-d_r2 retry	25.69	0.2152	False

This strongly supports that the pass/fail is governed by the target peak quality, especially width. Successful runs are below 0.2 ppm; failed ones are above 0.2 ppm.

Interpretation This is an **NMR analysis/QC failure**, not a no-peak failure and not an obvious RoboFlex hardware failure. The product/target peak exists, and the yield-like value is finite, but the peak is broader than the QC threshold. Since this happened twice for the same condition, I would treat this particular BO suggestion as **unreliable under the current analytical method**.

Recommendation I would **not retry this exact same condition again automatically**. We already did:

```
R0063 original: pass=False, width=0.2782 ppm
R0064 retry: pass=False, width=0.2152 ppm
```

The retry improved the peak width but still failed the 0.2 ppm threshold. A third attempt might pass by chance, but that would be drifting away from a robust measurement policy.

I recommend either:

1. **Reject/expire this BO suggestion and continue with a new BO suggestion**, or
2. **Have an NMR/operator review the spectra manually** and decide whether the ~25–27% yield should be accepted despite the width QC failure.

For autonomous BO, I would prefer option 1: do not feed this failed-QC value into BO-MCP unless a human explicitly overrides the NMR QC.

[...]

Operator

Can we try one more time?

LA AGENTE ÓPTIMA

Yes. Given the two consistent finite yield estimates and the fact that the retry was only slightly over the peak-width threshold, I've started **one final controlled retry** of the same #19 suggestion.

This is the **third total attempt** for that BO suggestion:

Attempt	Sample	Run	Outcome
Original	bo_5c5570bd-d	R0063	failed, width 0.2782 ppm
Retry #1	bo_5c5570bd-d_r2	R0064	failed, width 0.2152 ppm
Retry #2 / final	bo_5c5570bd-d_r3	pending	now started

Monitor:

```
mon_7 / roboflex_bo_failed19_retry_r3_continue
```

The script will:

- reuse the same pending BO suggestion/conditions,
- use new sample name `bo_5c5570bd-d_r3`,
- submit the BO result only if RoboFlex reports `pass=true`,
- stop if this third attempt also fails,
- and if it passes, continue to measurement #20.

I'll report meaningful state changes, alerts, heartbeat, and final analysis.

[Autonomous monitored execution: 9 interim status reports suppressed. Concluding report of the run:]

LA AGENTE ÓPTIMA

The campaign continuation reached the planned target and exited successfully.

Final run status Monitor:

```
mon_7 / roboflex_bo_failed19_retry_r3_continue
```

exited with:

```
EXIT_STATUS=0
```

Measurement #19 final retry The final retry of #19 succeeded:

```
Sample: bo_5c5570bd-d_r3
RoboFlex run: R0065
yield_percent: 45.09
green_score: 85.53
pass: true
peak_found: true
peak_width: 0.1102 ppm
```

This result was submitted to BO-MCP.

Measurement #20 Measurement #20 also completed and was submitted to BO-MCP:

```
Sample: bo_f20766bf-c
RoboFlex run: R0066
yield_percent: 0.00
green_score: 86.16
pass: true
peak_found: false
peak_position: None
peak_integral: 0.0
peak_width: 0.0
concentration: 0.0
```

This triggered a zero/no-peak alert:

```
ALERT zero/no-peak
ALERT streak: 1 consecutive zero-yield/no-peak completed experiment(s)
```

But since this was only one zero/no-peak result and we had reached the planned target of 20 valid BO results, the script submitted it as a valid pass=true zero-yield result, exported artifacts, paused the BO campaign, and exited successfully.

Final campaign state Expected valid BO results:

```
20
```

BO campaign was paused at the end.

RoboFlex returned to:

```
phase: running
state: awaiting_run
blocked_on: agent
```

Final artifact directory

```
artifacts/recreated_robochemflex_yield_bo_20260725/failed_measurement_retry_continuation_20260726T184638Z
```

Final log:

```
logs/failed19_retry_r3_continuation_20260726T184636Z/stdout.log
```

Summary of notable outcomes Best yield observed during the continuation:

```
Measurement #17 retry / R0061 / bo_a9f8598d-e_r2
yield: 48.51%
green_score: 82.50
```

Strong final retry for #19:

```
Measurement #19 final retry / R0065 / bo_5c5570bd-d_r3
yield: 45.09%
green_score: 85.53
```

Measurement #20 appears to be a genuine pass=true zero/no-peak result:

```
yield: 0.00%
pass: true
peak_found: false
```

The planned 20-result BO campaign is now complete and paused.

Operator

Can you plot the following? For each of the plot, generate a script in the workspace for generating the figure, which I can reuse later. The script should rely on logged data in the workspace (or exported campaign CSVs/JSONs (whatever feels

reasonable)), not on hard-coded numbers in the script.

- the BO improvement curve over the 20 experiments, visually separating the initial 6 seed experiments from the later 20 experiments.
- a Pareto front, where we plot all the "optimal" results in some way

LA AGENTE ÓPTIMA

💡 Thoughts

Reviewing details for clarity

I'm considering if I should include images in the final output for clarity. I need to mention script paths and outputs, as well as source auto-discovery. It's important to note that the first six seeds are separate from the later BO-selected 14, even though the user mentioned a total of 20. I want to make sure I highlight the Pareto "optimal" nondominated aspect to maximize both yield and sustainability. I might keep it concise without raw markdown links since there's no web involved.

Done. I created reusable plotting scripts and generated the figures from the logged/exported BO campaign data.

Scripts created

1. BO improvement curve

```
scripts/plot_bo_improvement_curve.py
```

This script auto-discovers the most complete/latest `bo_campaign_export.csv` under:

```
artifacts/recreated_robochemflex_yield_bo_20260725/
```

It plots:

- per-experiment yield,
- cumulative best yield,
- scalarized desirability,
- cumulative best scalarized desirability,
- visually separated seed region vs BO-selected region.

By default it treats the first 6 experiments as seeds:

```
uv run python scripts/plot_bo_improvement_curve.py
```

You can also pass an explicit export:

```
uv run python scripts/plot_bo_improvement_curve.py \
-input artifacts/recreated_robochemflex_yield_bo_20260725/failed_measurement_retry_continuation_20260726T184638Z/
bo_campaign_export.csv
```

Generated outputs:

```
plots/bo_improvement_curve.png
plots/bo_improvement_curve.svg
plots/bo_improvement_curve_data.csv
```

2. Pareto front

```
scripts/plot_pareto_front.py
```

This script also auto-discovers the latest/most complete BO export and computes the nondominated Pareto front for simultaneous maximization of:

```
yield_percent
green_score
```

Run with:

```
uv run python scripts/plot_pareto_front.py
```

Generated outputs:

```
plots/pareto_front_yield_green.png
plots/pareto_front_yield_green.svg
plots/pareto_front_points.csv
plots/pareto_front_all_points.csv
```

Data source used Both scripts used the final 20-result campaign export:

```
artifacts/recreated_robochemflex_yield_bo_20260725/failed_measurement_retry_continuation_20260726T184638Z/bo_campaign_export.csv
```

The export contains 20 rows.

Note on seed vs BO split I interpreted the “initial 6 seed experiments” as:

```
experiments 1–6 = seed / informed initial design
experiments 7–20 = BO-selected experiments
```

The plot marks that boundary with a vertical dashed line after experiment 6.

Pareto-optimal points The Pareto script identified the nondominated observations and wrote them to:

```
plots/pareto_front_points.csv
```

These are the experiments where no other result has both higher/equal yield and higher/equal green score, with at least one strictly better objective.

Operator

Can you chemically interpret the results?

LA AGENTE ÓPTIMA

💡 Thoughts

Analyzing chemical yields and optimization

I'm looking at optimal yield ranges: 0.00116-0.00126 and noting that 0.00159 is lower. I need to find a balance between high yield and environmental impact. High yields might require more time, but they can still be respectable regarding green considerations. I should mention that an objective scoring system may enhance desirability, factoring in chemical optimums. It might be good to suggest some recommendations for experiments and verification of NMR peak widths as part of the investigation.

Yes. Chemically, the campaign tells a fairly coherent story: the optimizer discovered that the productive region is not “more of everything,” but a fairly specific **Ru-bpy-Cl / PyNO / moderate TFAA / moderate PyNO / 50% light / ~16–18 min residence-time** regime.

1. Strong catalyst/oxidant identity preference The most important qualitative result is that the successful region is overwhelmingly:

catalyst_type: Ru bpy Cl
oxidant_type: py NO

The initial seed experiments tested several catalyst/oxidant combinations:

- 4CzIPN
- Ir ppy
- Ir CF3 ppy
- Ru bpy PF6
- Ru bpy Cl
- py NO
- 4-Ph py NO

The non-Ru bpy Cl / non-py NO seed points were mostly poor:

Catalyst	Oxidant	Yield
4CzIPN	py NO	0.0%
Ir CF3 ppy	4-Ph py NO	15.8%
Ir ppy	py NO	11.9%
Ru bpy PF6	4-Ph py NO	13.1%
4CzIPN	4-Ph py NO	7.6%

The best seed was already:

Ru bpy Cl / py NO → 30.0%

After that, BO repeatedly exploited Ru bpy Cl / py NO, which is chemically sensible: the data strongly indicate this pairing is the productive photocatalyst/oxidant combination for this substrate/system.

Interpretation: Ru bpy Cl likely provides the right excited-state redox balance / photoredox lifetime / compatibility with this TFAA–pyridine-N-oxide chemistry. The organic photocatalyst and Ir catalysts did not perform as well under the tested conditions. py NO also outperformed 4-Ph py NO in the observed campaign, although that comparison is somewhat confounded because most 4-Ph py NO observations were in seed points with other nonoptimal settings.

2. Productive region: not minimum reagent, but moderate reagent excess The best yields are clustered around:

catalyst_equiv: ~0.00115–0.00126
TFAA_equiv: ~1.55–1.62
oxidant_equiv: ~1.30–1.44
light: 50%
residence: ~16–17 min

The top two valid results were:

Experiment	Yield	Green	Catalyst	TFAA	Oxidant	Light	Residence
#17 retry	48.51%	82.50	0.00126	1.616	1.441	50%	16.43 min
#19 final retry	45.09%	85.53	0.00116	1.556	1.300	50%	16.81 min

This is a meaningful optimum-like cluster.

Interpretation: The reaction benefits from moving above the minimum TFAA and oxidant loadings, but not from going to the upper bounds. Moderate TFAA/oxidant probably promotes the desired activation/functionalization chemistry, while excessive oxidant or reagent may lead to side reactions, quenching, decomposition, or broader/unreliable NMR signals.

3. Residence time has a clear “too short / useful middle / maybe not longer” behavior Very short residence times, especially 2 min, were mostly low-yield despite high green scores:

Experiment	Residence	Yield	Green
#7	2.0 min	15.23%	100.0
#9	2.0 min	11.56%	100.0
#10	2.0 min	14.70%	94.66
#11	2.0 min	15.82%	95.31

The productive region emerged around:

~15–21 min residence time

Examples:

Experiment	Residence	Yield
#12	15.39 min	32.96%
#14	20.79 min	32.68%
#15	17.89 min	35.78%
#16	16.40 min	36.55%
#17	16.43 min	48.51%
#19	16.81 min	45.09%

Interpretation: The chemistry needs more than a few minutes to develop. Around 16–18 min seems to be a good kinetic compromise. Longer residence is not necessarily better; the seed at 90 min was poor, and the model did not push toward very long residence times once it found the productive window.

4. Light intensity: 50% seems best in the productive region The best points all used:

light_intensity = 50%

High light at very short residence or moderate residence was not obviously better:

Experiment	Light	Residence	Yield
#13	100%	17.15 min	27.20%
#12	75%	15.39 min	32.96%
#14	50%	20.79 min	32.68%
#17	50%	16.43 min	48.51%
#19	50%	16.81 min	45.09%

The final experiment #20 used lower light:

25% light, 19.39 min → 0% yield, no peak

Interpretation: There may be a threshold photon flux required. 25% appears too low for those conditions, while 50% is sufficient and perhaps avoids issues associated with higher intensity, such as overoxidation, catalyst degradation, side reactions, or poorer selectivity. 100% did not improve yield in the explored region.

5. The green/yield tradeoff is real but not severe The highest green-score points are low-reagent, short-time conditions. But those are not the highest yielding.

For example:

Experiment	Yield	Green	Comment
#7	15.23%	100.0	very green, low yield
#2	30.04%	98.30	excellent green/yield compromise
#14	32.68%	97.33	strong Pareto point
#17	48.51%	82.50	best yield, less green
#19	45.09%	85.53	high yield, slightly greener than #17

The best chemical point depends on whether we prioritize absolute yield or green/yield balance.

If maximizing yield: Best point is:

```
#17 / R0061
Ru bpy Cl
py NO
catalyst_equiv ≈ 0.00126
TFAA_equiv ≈ 1.616
oxidant_equiv ≈ 1.441
light = 50%
residence ≈ 16.43 min
yield = 48.51%
green = 82.50
```

If seeking a greener high-yield compromise: A very attractive point is:

```
#19 / R0065
Ru bpy Cl
py NO
catalyst_equiv ≈ 0.00116
TFAA_equiv ≈ 1.556
oxidant_equiv ≈ 1.300
light = 50%
residence ≈ 16.81 min
yield = 45.09%
green = 85.53
```

This loses only ~3.4 percentage points yield compared with the best point, but improves the green score.

6. The NMR QC issue is localized but important Several high-yield-like points around the productive region initially failed NMR QC because of broad target peaks:

```
R0060: pass=false, width=0.3307 ppm, yield-like ~25%
R0063: pass=false, width=0.2782 ppm, yield-like ~27%
R0064: pass=false, width=0.2152 ppm, yield-like ~25.7%
```

But reruns eventually passed:

```
R0061: pass=true, width=0.1050 ppm, yield=48.51%
R0065: pass=true, width=0.1102 ppm, yield=45.09%
```

Interpretation: The chemistry in this region is productive, but the NMR/analysis can intermittently produce broad peaks. This may reflect:

- sample heterogeneity,
- mixing/slug quality variation,
- transient NMR shimming/line broadening,
- overlapping species near the target peak,
- or chemical mixtures that sometimes broaden the target resonance.

Importantly, once the peak passed QC, the high yields were real and strong.

I would not treat these as pure device failures, but I also would not blindly accept failed-QC values without inspection. The “retry once, maybe twice if finite and reproducible” policy was reasonable here.

7. Experiment #20: likely a real negative control-like result Experiment #20 gave:

```
yield = 0
pass = true
peak_found = false
light = 25%
residence = 19.39 min
TFAA ≈ 1.40
oxidant ≈ 1.40
catalyst ≈ 0.00121
```

This is interesting because it is chemically near the productive reagent/residence region, but with lower light intensity:

```
25% light
```

Interpretation: This suggests that dropping to 25% light may fall below the photon flux needed for the productive photoredox cycle. It is not just a green improvement; it may turn off the reaction under these conditions.

8. Overall chemical picture The campaign suggests the following reaction logic:

1. **Correct photocatalyst/oxidant identity is essential.**
 - Ru bpy Cl / py NO is clearly favored.
2. **The reaction needs moderate residence time.**
 - 2 min is too short.
 - ~16–18 min is optimal in the explored region.
3. **The reaction needs sufficient but not excessive light.**
 - 50% appears optimal.
 - 25% may be too low.
 - 100% is not obviously beneficial.
4. **Moderate TFAA and oxidant excess improve yield.**
 - Best points use TFAA around 1.55–1.62 equiv.
 - Best points use PyNO around 1.30–1.44 equiv.
5. **Catalyst loading can remain low.**
 - Best points are near 0.0012 equiv, only slightly above the lower bound.
 - Increasing catalyst much further does not obviously help.
6. **The best yield region is still reasonably green.**
 - The top-yield points do not require extreme reagent loadings or extreme light/residence time.

Recommended next chemistry steps If continuing scientifically, I would not simply keep broad BO over the full original space. I would switch to a focused follow-up study around the discovered region:

```
catalyst_type = Ru bpy Cl
oxidant_type = py NO
light_intensity = 50%, maybe compare 40/50/60 if available
residence_time_min = 14–22
TFAA_equiv = 1.3–1.8
oxidant_equiv = 1.1–1.6
catalyst_equiv = 0.0010–0.0015
```

I would also include replicates of:

```
#17 condition: best yield
#19 condition: strong yield/green compromise
#14 condition: greener ~33% yield point
```

And I would specifically monitor NMR peak width in this region, because the best chemistry appears close to a zone where the NMR target peak can broaden intermittently.

Operator

Can you now use all the existing (valid) data points for setting up a new campaign that purely optimizes on yield? Experiment-wise, we just want to continue the series, but the BO objective should now be purely yield, not the mixed yield-green score. Also, we allow now 3 retries for these somehow stochastic NMR failures per suggestion before bugging out. Check with me again before actually starting the campaign.

LA AGENTE ÓPTIMA

Yes. I prepared the yield-only campaign workflow, but I have **not** created the new BO campaign and have **not** started any RoboFlex experiment.

Prepared files New package:

```
robochemflex_yield_only_bo/
```

Key scripts:


```
recreate_robochemflex_yield_only_bo.py
continue_robochemflex_yield_only_bo.py
```

Execution notes:

```
HOW_TO_EXECUTE_YIELD_ONLY_CAMPAIGN.md
```

Manifest:

```
campaign_manifest_yield_only.json
```

What this new workflow does It uses the same RoboFlex/search-space/request mapping as before, but changes the BO objective to **yield only**.

The new BO objective is:

```
yield_percent
direction: maximize
normalization bounds: [0, 100]
```

green_score is **not** a BO objective in the new campaign. It may still be saved as audit/provenance metadata, but the optimizer will not optimize it.

Historical data used The yield-only campaign will seed from the final valid 20-result export:

```
artifacts/recreated_robochemflex_yield_bo_20260725/failed_measurement_retry_continuation_20260726T184638Z/bo_campaign_export.csv
```

This includes the 20 valid BO-submitted results and excludes failed NMR attempts such as:

```
R0060
R0063
R0064
```

while including their successful valid retries where applicable:

```
R0061
R0065
```

Retry policy prepared The continuation script implements your updated rule:

> NMR-only / QC-style stochastic failures can be retried up to 3 times per BO suggestion before stopping.

More specifically:

- initial run + up to 3 retries;
- retries only when the platform/device looks healthy and the failure is analytical/QC-like;
- no BO result is submitted unless a retry passes;
- true device/platform failures, no-evidence/no-peak failures, duplicate sample names, request-equivalence failures, or retry exhaustion still stop safely.

Validation performed Safe validation only:

```
uv run python -m py_compile recreate_robochemflex_yield_only_bo.py continue_robochemflex_yield_only_bo.py
robochemflex_yield_only_bo/*.py
```

Dry-run/intake validation:

```
uv run python recreate_robochemflex_yield_only_bo.py --dry-run --validate-intake
```

Result:

```
Prepared 20 seed results
BO-MCP intake validation: valid=True
No campaign created
No hardware contacted
```

Read-only continuation preflight also confirmed:

```
Mixed-objective source BO campaign: paused, 20 results, no pending suggestions
RoboFlex: hardware, running, awaiting_run
Active RoboFlex campaign matches original series
```

Commands for later

Dry-run only

```
uv run python recreate_robocemflex_yield_only_bo.py --dry-run
```

Dry-run with BO intake validation

```
uv run python recreate_robocemflex_yield_only_bo.py --dry-run --validate-intake
```

Create and seed the new yield-only BO campaign This touches BO-MCP but **not** RoboFlex:

```
uv run python recreate_robocemflex_yield_only_bo.py \
--execute-create-seed \
--confirm-create-seed \
--validate-intake
```

The new campaign ID would then be written to:

```
<artifact_dir>/bo_campaign_id.txt
```

Later hardware continuation After the new campaign ID exists:

```
uv run python continue_robocemflex_yield_only_bo.py \
--campaign-id <NEW_YIELD_ONLY_CAMPAIGN_ID> \
--execute \
--confirm-autonomous-hardware \
--max-new-measurements 1
```

Waiting for your confirmation Before I do anything else, please confirm which step you want next:

1. **Create and seed the new yield-only BO-MCP campaign** using the 20 valid results, but do not touch RoboFlex.
2. Only do another dry-run/preflight.
3. Do nothing further for now.

Operator

How many BO iterations (number of measurements) would you recommend?

LA AGENTE ÓPTIMA

For the **new yield-only campaign**, I would recommend a **small focused continuation first**, not a large open-ended run.

Recommendation I recommend:

```
5 additional valid measurements
```

as the next yield-only BO block.
That would take us from:

20 valid measurements → 25 valid measurements

Then we should pause and reassess.

Why 5, not 10–20 immediately? The current data already show a fairly clear productive region:

Ru bpy Cl
py NO
light $\approx 50\%$
residence $\approx 16\text{--}18$ min
TFAA $\approx 1.55\text{--}1.62$ equiv
oxidant $\approx 1.3\text{--}1.45$ equiv
catalyst $\approx 0.0011\text{--}0.0013$ equiv

The best yields are:

48.51%
45.09%
36.55%
35.78%
32–33%

So the optimizer is no longer searching blindly. A yield-only objective will likely focus even more aggressively around the high-yield region.

A block of 5 lets us answer:

1. Does yield-only BO push beyond $\sim 50\%$?
2. Does it repeatedly suggest the same high-yield region?
3. Are NMR peak-width failures frequent in this region?
4. Is the apparent optimum robust or noisy?
5. Does removing green-score pressure improve yield materially?

What I would not recommend yet I would not immediately run another 20 measurements under yield-only BO.
Reasons:

- The high-yield region is already narrow.
- Several nearby points produced NMR QC failures before passing on retry.
- The device/reagent budget may be better spent after checking whether the model converges.
- Yield-only BO may become exploitative and repeatedly probe a small region.

Practical plan I would configure the new yield-only campaign as:

seed with: 20 valid historical results
run: 5 new valid measurements
retry policy: up to 3 NMR/QC retries per suggestion
pause after: 25 total valid BO results

After those 5, we review:

- best yield,
- replicate consistency,
- whether suggestions cluster,
- NMR pass/fail rate,
- whether the model is improving or plateauing.

If the first 5 are promising If we see clear improvement, e.g.:

best yield improves from 48.5% to $>55\text{--}60\%$

or the model identifies a new local region, then I would run another block of:

5 additional measurements

So my recommended strategy is:

20 existing → +5 yield-only → review → optional +5

rather than committing immediately to a large campaign.

Operator

I have time until tomorrow morning 8:30 am EDT – how many experiments can we likely conduct until then?

LA AGENTE ÓPTIMA

💡 Thoughts

Calculating experiment durations

I need to find the duration until July 27, 8:30 EDT, which is about 10 hours and 51 minutes. To estimate the duration per experiment, I'll look at recent run timestamps. For example, R0065 took around 42 minutes total, while R0066 took about 54 minutes. Previous experiments ranged from 39 to 60 minutes. I should also consider that BO generation can take around 5-6 minutes. On average, productive conditions bring durations to around 45-55 minutes.

💡 Thoughts

Estimating NMR experiment retries

I need to consider that NMR failures could add an extra 45-55 minutes for each retry, and with up to 3 retries per suggestion, that could lead to a worst-case scenario. If everything goes smoothly, I expect about 11-13 experiments in approximately 10 hours and 50 minutes, but with retries, I think with 8-10 valid cases is more realistic. From now, which is 9:38 PM EDT, to 8:30 AM EDT is about 10 hours and 51 minutes. I'd recommend starting with 5 experiments to finish around 2-3 AM for a review.

Current EDT time is about **9:38 PM on July 26, 2026**, so until **8:30 AM EDT tomorrow** we have roughly:

10 h 50 min

Based on the RoboFlex runs we just observed, a typical successful experiment in the productive region takes about:

40–60 min per submitted run

including reaction residence time, sample handling, NMR acquisition/analysis, and polling overhead.

Practical estimate

If everything passes cleanly With ~10.8 hours available:

10.8 h / ~0.8–1.0 h per experiment ≈ 10–13 experiments

So an optimistic upper bound is around:

12 valid experiments

More realistic, allowing for BO generation time and occasional NMR retries Because we now know that NMR QC failures can happen in this region, and because a retry costs essentially another experiment slot, I would plan for:

8–10 valid experiments

before 8:30 AM EDT.

Conservative estimate with multiple NMR/QC failures If several suggestions need retries:

5–8 valid experiments

would be more realistic.

My recommendation For overnight operation, I would not run an unbounded yield-only campaign. I would set:

max new valid measurements: 8

with:

up to 3 NMR/QC retries per suggestion

and stop conditions for:

- repeated no-peak / zero-yield results,
- repeated NMR QC failures for one suggestion,
- RoboFlex/device errors,
- BO generation failures,
- or reaching 8 valid new measurements.

That is a good balance: it likely fits before 8:30 AM even with one or two retries, while still giving the yield-only campaign enough room to exploit the high-yield region.

If you want a more aggressive overnight push, we could set:

max new valid measurements: 10

but then there is a higher chance it runs close to or beyond 8:30 AM if retries occur.

Suggested setting I recommend:

yield-only campaign seeded with 20 valid results
run up to 8 new valid measurements overnight
retry NMR/QC failures up to 3 times per suggestion
pause automatically when done

Expected outcome by morning:

20 → ~28 valid BO results

with a plausible range of:

25–30 valid results total

depending on retries and run durations.

Operator

Then let's do 8. Start.

LA AGENTE ÓPTIMA

Started.

New yield-only campaign Created and seeded a new yield-only BO-MCP campaign with the 20 valid historical results:

1970655b-a702-4963-874b-6973489cc89d

Seed/create log:

logs/yield_only_create_seed_20260727T013935Z/stdout.log

Seed artifact directory:

```
artifacts/yield_only_recreation_20260727T013936Z/
```

Overnight yield-only continuation Started monitor:

```
mon_8 / roboflex_yield_only_bo_8_overnight
```

Configuration:

```
max new valid measurements: 8
objective: yield_percent only
max NMR/QC retries per suggestion: 3
BO generate timeout: 1200 s
poll interval: 180 s
heartbeat interval: 1800 s
RoboFlex campaign: existing active campaign, no new RoboFlex campaign created
```

Log:

```
logs/yield_only_bo_8_<timestamp>/stdout.log
```

The script will pause the BO campaign on exit. I'll monitor and report meaningful state changes, alerts, heartbeats, and completed-experiment analyses.

[Autonomous monitored execution: 6 status reports followed; the most substantive one is reproduced here:]

LA AGENTE ÓPTIMA

Yield-only measurement #21 completed successfully and was submitted to the yield-only BO campaign.

Measurement #21 result

```
Sample: bo_cc26e7f1-b
RoboFlex run: R0067
Status: completed
```

Yield-only result:

```
yield_percent: 58.81%
```

This is a new best yield compared with the previous best from the 20-result campaign:

```
previous best: 48.51%
new best: 58.81%
```

Current yield-only campaign progress:

```
Seeded historical results: 20
New yield-only completed: 1
Total yield-only BO results: 21
Remaining planned overnight measurements: 7
```

The script has moved on to generate the next yield-only BO suggestion.

Operator

Which parameter has changed the most, leading to this strong increase in yield?

LA AGENTE ÓPTIMA

The biggest chemically meaningful change in the first yield-only suggestion, #21, was that **TFAA loading and catalyst loading were both increased**, while the campaign kept the same successful identity/light region:

Ru bpy Cl / py NO / 50% light / ~15–17 min residence

Comparison to previous best Previous best valid point, experiment #17:

yield: 48.51%
catalyst_equiv: 0.001255
TFAA_equiv: 1.616
oxidant_equiv: 1.441
light: 50%
residence: 16.43 min

New yield-only point, experiment #21:

yield: 58.81%
catalyst_equiv: 0.001892
TFAA_equiv: 1.997
oxidant_equiv: 1.392
light: 50%
residence: 14.17 min

Parameter changes

Parameter	Previous best #17	New #21	Change
catalyst_type	Ru bpy Cl	Ru bpy Cl	same
oxidant_type	py NO	py NO	same
light_intensity	50	50	same
residence_time_min	16.43	14.17	slightly shorter
oxidant_equiv	1.441	1.392	slightly lower
TFAA_equiv	1.616	1.997	higher
catalyst_equiv	0.001255	0.001892	higher

Relative changes:

TFAA: +24%
catalyst: +51%
oxidant: -3%
residence: -14%

So the clearest shift is:

> **yield-only BO moved away from the greener low-catalyst / lower-TFAA region and increased catalyst + TFAA loading.**

Chemical interpretation The previous mixed objective penalized higher catalyst and reagent loading through **green_score**, so it tended to prefer lower-burden points when yield was comparable. Once we removed the green objective, the optimizer was free to increase reagent/catalyst loading if that was predicted to improve yield. The first yield-only suggestion did exactly that:

- kept the proven productive Ru bpy Cl / py NO combination,
- kept 50% light,
- stayed near the productive residence-time window,
- increased TFAA_equiv substantially,
- increased catalyst_equiv substantially.

The yield increase from ~48.5% to ~58.8% suggests that the earlier green-optimized campaign may have been under-dosing either:

1. the photoredox catalyst,
2. TFAA,
3. or both.

Which parameter “caused” the increase? We cannot assign causality from one BO suggestion alone because both catalyst and TFAA changed together. But the strongest candidate is:

TFAA_equiv increase

with a likely contribution from:

catalyst_equiv increase

Why TFAA is especially plausible:

- The best previous points already had moderate TFAA around 1.55–1.62.
- The new point pushed TFAA closer to 2.0.
- TFAA is chemically central to activation/acylation/dehydrating or trifluoroacetylating steps in this reaction manifold.
- Oxidant did not increase; it slightly decreased.
- Light did not change.

So my interpretation is:

> The yield-only optimizer discovered that pushing TFAA upward, together with a somewhat higher Ru catalyst loading, improves conversion/yield while keeping light and residence time in the established productive region.

What to watch next Measurement #22 is already probing this further:

TFAA_equiv: 2.302
catalyst_equiv: 0.002245
oxidant_equiv: 1.355
light: 50%
residence: 21.55 min

So #22 is testing whether **even more catalyst/TFAA and longer residence** continue improving yield, or whether #21 was near the optimum.

[Autonomous monitored execution: 3 interim status reports suppressed. Concluding report of the run:]

LA AGENTE ÓPTIMA

The yield-only overnight run stopped safely after measurement #22 failed with a **non-retryable RoboFlex/analysis error**.

What happened Measurement #21 succeeded strongly:

R0067 / bo_cc26e7f1-b
yield: 58.81%

Measurement #22 was then submitted:

R0068 / bo_31d5114e-3

but failed with:

zero-size array to reduction operation minimum which has no identity

The result payload for R0068 has:

```
{
  "status": "failed",
  "success": false,
  "result": null,
  "error": "zero-size array to reduction operation minimum which has no identity"
}
```

So this is different from the previous **pass=false** NMR QC failures.

Why it did not retry automatically The retry policy allows retries for NMR/QC-style failures when there is finite yield/peak-like evidence.

For R0068:

```
result: null
no yield-like value
no peak evidence
explicit error string
```

Therefore the script classified it as **not safely retryable/submittable** and stopped without submitting a BO result. That was the correct safe behavior under the policy.

Conditions for failed #22 BO candidate for #22:

```
{
  "catalyst_type": "Ru bpy Cl",
  "oxidant_type": "py NO",
  "catalyst_equiv": 0.0022452457086006112,
  "TFAA_equiv": 2.301668795918735,
  "oxidant_equiv": 1.355256776264335,
  "light_intensity": 50,
  "residence_time_min": 21.55044544824578
}
```

This was a more aggressive yield-only point than #21:

- higher catalyst,
- higher TFAA,
- longer residence time.

Current state The script paused the yield-only BO campaign and exited with status 1. Valid yield-only campaign results should now be:

```
20 seeded historical + 1 new valid = 21
```

No BO result was submitted for R0068.

RoboFlex returned to idle/awaiting-run state according to the final record.

Artifacts Continuation artifact directory:

```
artifacts/yield_only_robochemflex_bo/continuation_20260727T014003Z/
```

Failed #22 artifacts:

```
artifacts/yield_only_robochemflex_bo/continuation_20260727T014003Z/measurement22_attempt1/
```

Log:

```
logs/yield_only_bo_8_20260727T014001Z/stdout.log
```

Recommendation I would not immediately retry R0068 automatically, because it has **result=null** rather than a finite failed-QC result.

Next best step is to inspect the RoboFlex/NMR logs for R0068 to see whether this was:

1. an analysis pipeline edge case,
2. no peaks / empty data in the target region,
3. a bad spectrum/file,
4. or a chemistry/mixture issue causing the analysis to crash.

If it turns out to be an analysis pipeline failure with otherwise valid raw data, we could decide whether to rerun or skip/reject the suggestion.

[...]

Operator

Generate the prioritized recommended plots.

LA AGENTE ÓPTIMA

Generated the prioritized search-space coverage plots and reusable scripts.

Scripts created Shared utility:

```
scripts/search_space_plot_utils.py
```

Figure scripts:

```
scripts/plot_search_space_coverage_parallel.py
scripts/plot_search_space_pairwise.py
scripts/plot_search_space_focused_map.py
```

These scripts read from campaign exports/artifacts in the workspace, not hard-coded result values. By default they auto-discover the latest/most complete export, currently:

```
artifacts/yield_only_robochemflex_bo/continuation_20260727T014003Z/bo_campaign_export.csv
```

They also include failed/unsubmitted tested attempts when available, shown separately as gray/dashed or gray “x” markers. That is useful for coverage because failed attempts still correspond to actually tested coordinates.

Generated plots Output directory:

```
plots/search_space_coverage/
```

1. Parallel-coordinates coverage plot

```
plots/search_space_coverage/search_space_parallel_coordinates.png
plots/search_space_coverage/search_space_parallel_coordinates.svg
```

This is the best hyperdimensional overview. It shows all search-space dimensions:

```
catalyst_type
oxidant_type
catalyst_equiv
TFAA_equiv
oxidant_equiv
light_intensity
residence_time_min
```

Color encodes yield for valid BO results. Failed/unsubmitted attempts are shown as gray dashed lines.

2. Pairwise numeric projection matrix

```
plots/search_space_coverage/search_space_pairwise_numeric.png
plots/search_space_coverage/search_space_pairwise_numeric.svg
```

This shows pairwise coverage among the numeric/discrete dimensions:

```
catalyst_equiv
TFAA_equiv
oxidant_equiv
light_intensity
residence_time_min
```

Valid points are colored by yield; failed/unsubmitted attempts are gray x markers.

3. Focused chemical map All catalyst/oxidant identities:

```
plots/search_space_coverage/search_space_focused_residence_vs_TFAA.png  
plots/search_space_coverage/search_space_focused_residence_vs_TFAA.svg
```

Filtered to the main productive region, Ru bpy Cl / py NO:

```
plots/search_space_coverage/search_space_focused_residence_vs_TFAA_main_region.png  
plots/search_space_coverage/search_space_focused_residence_vs_TFAA_main_region.svg
```

These plot:

```
x = residence_time_min  
y = TFAA_equiv  
color = yield_percent  
marker_shape = light_intensity  
marker_size = catalyst_equiv
```

This should be the most chemically interpretable view of how the search has moved through the productive region.

Data table written The combined plotting dataset was saved here:

```
plots/search_space_coverage/search_space_coverage_points.csv
```

It contains both valid BO results and failed/unsubmitted tested attempts, with status/stage labels.

Reuse commands Regenerate all:

```
uv run python scripts/plot_search_space_coverage_parallel.py  
uv run python scripts/plot_search_space_pairwise.py  
uv run python scripts/plot_search_space_focused_map.py  
uv run python scripts/plot_search_space_focused_map.py --filter-main-region
```

Use a specific export:

```
uv run python scripts/plot_search_space_coverage_parallel.py \  
-input path/to/bo_campaign_export.csv
```

Disable failed/unsubmitted attempts if you only want valid BO points:

```
uv run python scripts/plot_search_space_pairwise.py --no-include-failed-tested
```

Operator

Regarding the error in measurement #22: Take the observed error as another stochastic NMR failure (as long as the machine is still "running") and include this type of error also in the retry-policy of maximum three retries. Thus, continue the campaign by retrying #22 (up to three retries in total) and then continue with the remaining measurements, applying the same new retry policy.

LA AGENTE ÓPTIMA

Started the patched yield-only continuation with the new retry policy for measurement #22.
Monitor:

```
mon_9 / roboflex_yield_only_retry22_continue
```

What changed The prior #22 error:

```
zero-size array to reduction operation minimum which has no identity
```

is now treated as a retryable stochastic NMR/analysis failure **if** RoboFlex remains healthy:

```
mode=hardware
phase=running
progress.state=awaiting_run
no queued/running/active runs
expected RoboFlex campaign matches
```

The workflow is pinned to the pending #22 suggestion:

```
suggestion_id: 31d5114e-3796-4b21-9d5f-d7e9a31378b0
original sample: bo_31d5114e-3
failed run: R0068
```

Retry samples will be:

```
bo_31d5114e-3_r2
bo_31d5114e-3_r3
bo_31d5114e-3_r4
```

BO result submission still happens only if a retry passes with finite yield.
If #22 succeeds, the script will continue the yield-only campaign toward:

```
28 total valid BO results
```

i.e. 8 valid yield-only measurements beyond the original 20, with #21 already complete.

[Autonomous monitored execution: 28 status reports followed; the most substantive one is reproduced here:]

LA AGENTE ÓPTIMA

The yield-only continuation stopped safely after measurement #27 failed before analysis.

Stop event

```
Measurement: #27
Sample: bo_86b39c20-a
RoboFlex run: R0074
Status: failed
Reason: RoboFlex run failed before analysis
```

This is different from the retryable NMR-analysis failures. The platform also entered:

```
phase: stopping
state: stopping
```

before the run failure, so the script correctly treated this as **not safely retryable**.

Actions taken by the script

- No BO result was submitted for R0074.
- Yield-only BO campaign was paused.
- The script exited with status 1.
- Detailed artifacts were written.

Current yield-only campaign progress Valid yield-only results completed after the 20 seeded historical rows:

```
#21: 58.81%
#22: 54.50% (successful retry of prior analysis error)
#23: 0.00%
#24: 0.00%
#25: 45.79%
#26: 0.00%
```

So current valid BO result count should be:

20 seeded + 6 new valid = 26

Failed / not submitted:

#27 / R0074 / bo_86b39c20-a

Artifact directory

artifacts/yield_only_robochemflex_bo/continuation_20260727T133832Z

Log:

logs/yield_only_retry22_20260727T133831Z/stdout.log

Recommendation Since the platform entered **stopping**, I recommend **checking RoboFlex status before doing anything else**. This may indicate the active campaign is winding down or has been stopped by the platform/operator. We should not submit more runs until the robot state is confirmed.

[...]

Operator

The technician has prepared the robot for the continuation of the campaign, disregard experiments from R0068 onwards (when failures and questionable 0.00 yield results began – the technician said we should discard experiments starting with R0068) and continue the yield only campaign.

LA AGENTE ÓPTIMA

Understood. I created a **clean yield-only BO campaign** that discards everything from R0068 onward, then started continuation against the technician-prepared RoboFlex campaign.

Clean yield-only BO campaign New clean campaign:

13bb9fef-7bd3-4db2-8e99-2e39e469dc92

Seeded with:

20 original valid historical results
+ R0067 / bo_cc26e7f1-b / 58.81%
= 21 clean valid results

Explicitly excluded from the clean campaign:

R0068
R0069
R0070
R0071
R0072
R0073
R0074

Create/seed artifacts:

artifacts/yield_only_clean21_recreation_20260729T155858Z/

RoboFlex status The technician has prepared a new active RoboFlex campaign:

roboflex_yield_BO_fresh_refilled_stocks-20260728-112201

Read-only preflight confirmed:

```
mode: hardware
phase: running
state: awaiting_run
queue_depth: 0
active_run_ids: []
```

Continuation started Monitor:

```
mon_10 / roboflex_clean_yield_only_7_continue
```

Configuration:

```
objective: yield only
starting valid results: 21
target valid results: 28
max new valid measurements: 7
max NMR/analysis retries per suggestion: 3
RoboFlex campaign: roboflex_yield_BO_fresh_refilled_stocks-20260728-112201
```

The script will continue from the clean campaign and ignore the discarded R0068+ data.

[Autonomous monitored execution: 41 status reports followed; the most substantive one is reproduced here:]

LA AGENTE ÓPTIMA

Third transient polling alert for R0075:

```
RoboFlex run: R0075
Issue: repeated transient run-poll errors
```

I also attempted a read-only RoboFlex `/v1/status` check to distinguish a run-specific polling issue from a broader API/connectivity issue, and it failed with:

```
HTTP Error 530
```

So this now looks like a **RoboFlex API/connectivity/server-side availability issue**, not necessarily a chemistry/run failure. The running monitor is still alive and will keep retrying according to its polling logic. I'll continue monitoring for either:

- API recovery,
- run completion/failure,
- platform state change,
- or the script stopping safely.

Operator

Can you stop the polling?

LA AGENTE ÓPTIMA

Stopped the polling monitor:

```
mon_10 / roboflex_clean_yield_only_7_continue
```

This stops our local monitoring process. Since the RoboFlex/API state was unclear due to repeated polling errors, I did **not** submit anything further and did **not** mark R0080 as completed or failed in BO-MCP.

Operator

Can you log all the results to a file (export the campaign)? Also, generate a new BO improvement plot containing only the successful measurements.

LA AGENTE ÓPTIMA

Done.

Campaign export I exported the current clean yield-only BO campaign results to:

```
artifacts/yield_only_clean21_export_20260729T212630Z/
```

Files:

```
bo_campaign_export.csv
bo_results.json
bo_campaign_status.json
bo_campaign_id.txt
bo_campaign_export.content_type.txt
```

The export contains:

```
26 successful/submitted BO results
```

Campaign ID:

```
13bb9fef-7bd3-4db2-8e99-2e39e469dc92
```

Note: R0080 is not included because it was still unresolved/not submitted when we stopped polling.

New successful-measurements-only improvement plot Generated:

```
plots/yield_only_clean/yield_only_successful_improvement_curve.png
plots/yield_only_clean/yield_only_successful_improvement_curve.svg
plots/yield_only_clean/yield_only_successful_improvement_curve_data.csv
```

Reusable script:

```
scripts/plot_yield_only_successful_improvement.py
```

Regenerate with:

```
uv run python scripts/plot_yield_only_successful_improvement.py \
-input artifacts/yield_only_clean21_export_20260729T212630Z/bo_campaign_export.csv
```

The plot includes only successful/submitted BO measurements and visually separates:

```
experiments 1–6: seed
experiments 7–20: previous mixed-objective BO data retained as clean history
experiments 21+: clean yield-only BO continuation
```

[...]

C.6.5 Executed experiments

Tab. 19 lists the 23 experiments that make up the final campaign record, in the order in which they appear in Fig. 4, with the conditions as executed by the platform and the material each of them consumed. Seven further runs were submitted to the platform during the campaign but do not enter that record: three returned no usable ^{19}F NMR analysis, three were superseded when the yield-only campaign was reseeded, and one was submitted while the platform became unreachable and never returned a result.

Table 19 The 23 experiments of the RoboChem-Flex campaign shown in Fig. 4, in figure order, with the conditions as executed by the platform and the material consumed. Every experiment used one 650 μ L slug at 100 mM substrate, i.e. 65 μ mol (13.60 mg) of limiting reagent; all other amounts follow from the equivalences. Experiments 1–21 were carried into the reseeded yield-only campaign as history, 22–23 were measured in it. Photocatalyst loadings are given in units of 10^{-3} equiv.

#	Run	Photocatalyst	Oxidant	cat. 10^{-3} eq	TFAA eq	ox. eq	light %	t_R min	yield %	G	cat. μ g
1	R0044	4CzIPN	py NO	2.50	2.200	1.800	50	30.00	0.00	60.31	124
2	R0045	[Ru(bpy) ₃]Cl ₂	py NO	1.00	0.900	0.900	75	10.00	30.04	98.30	49
3	R0046	Ir(CF ₃ ppy) ₃	4-Ph-py NO	4.00	3.500	3.000	100	75.00	15.77	4.26	292
4	R0047	Ir(ppy) ₃	py NO	2.00	1.400	2.400	25	45.00	11.93	65.95	85
5	R0048	[Ru(bpy) ₃](PF ₆) ₂	4-Ph-py NO	3.50	3.000	1.200	75	20.00	13.14	51.57	196
6	R0049	4CzIPN	4-Ph-py NO	1.50	1.700	2.500	50	90.00	7.57	56.59	74
7	R0050	[Ru(bpy) ₃]Cl ₂	py NO	1.00	0.900	0.900	100	2.00	15.23	100.00	49
8	R0051	[Ru(bpy) ₃]Cl ₂	py NO	1.00	0.900	0.900	75	32.15	27.64	93.58	49
9	R0052	[Ru(bpy) ₃]Cl ₂	py NO	1.00	0.900	0.900	50	2.00	11.56	100.00	49
10	R0053	[Ru(bpy) ₃]Cl ₂	py NO	1.00	0.900	1.348	75	2.00	14.70	94.66	49
11	R0054	[Ru(bpy) ₃]Cl ₂	py NO	1.00	1.388	0.900	75	2.00	15.82	95.31	49
12	R0055	[Ru(bpy) ₃]Cl ₂	py NO	1.00	0.939	0.900	75	15.38	32.96	96.77	49
13	R0056	[Ru(bpy) ₃]Cl ₂	py NO	1.00	0.900	0.900	100	17.15	27.20	95.70	49
14	R0057	[Ru(bpy) ₃]Cl ₂	py NO	1.00	0.900	0.900	50	20.79	32.68	97.33	49
15	R0058	[Ru(bpy) ₃]Cl ₂	py NO	1.00	1.578	0.900	50	17.89	35.78	91.22	49
16	R0059	[Ru(bpy) ₃]Cl ₂	py NO	1.59	1.589	0.900	50	16.40	36.55	86.42	77
17	R0061	[Ru(bpy) ₃]Cl ₂	py NO	1.26	1.616	1.441	50	16.43	48.51	82.50	61
18	R0062	[Ru(bpy) ₃]Cl ₂	py NO	1.03	1.577	1.750	50	17.01	20.18	80.96	50
19	R0065	[Ru(bpy) ₃]Cl ₂	py NO	1.16	1.556	1.300	50	16.80	45.09	85.53	56
20	R0066	[Ru(bpy) ₃]Cl ₂	py NO	1.21	1.404	1.401	25	19.39	0.00	86.16	59
21	R0067	[Ru(bpy) ₃]Cl ₂	py NO	1.89	1.997	1.392	50	14.17	58.81	74.43	92
22	R0075	[Ru(bpy) ₃]Cl ₂	py NO	2.22	2.314	1.362	50	18.68	52.28	68.41	108
23	R0076	[Ru(bpy) ₃]Cl ₂	py NO	2.21	1.690	1.403	50	11.69	31.80	74.98	107
Total consumed: 313 mg substrate, 1.87 mg photocatalyst, 488 mg TFAA, 227 mg oxidant											

The 23 experiments together occupied 18.2 h of reactor and ¹⁹F NMR time. G is the agent-defined green score described in Sec. C.6.3. The green score of experiment 9 (R0052) is recomputed from that definition, its original submission having been lost while BO-MCP was unavailable.

C.7 Cost accounting of the agent-directed campaign

C.7.1 What is counted, and how

We compare two optimization campaigns performed on the same transformation and the same hardware. **Campaign A** is a human-directed BO campaign: the search space, initialization design and campaign termination were chosen by an operator, who supervised the platform throughout. **Campaign B** is the agent-directed campaign described in the main text, in which LA AGENTE ÓPTIMA constructed, executed and monitored the campaign; the campaign was stopped by the operators after 23 experiments.

Three quantities are reported: the cost of the complete campaign, the cost of an individual experiment, and the mass efficiency of the optimum conditions and of the search as a whole. Every experiment is charged its full reagent and solvent load from its recorded conditions, with no recovery, recycling or bulk discount; reagent costs use research-scale list prices. Operator time is excluded because it was not measured for either campaign. The assumptions underlying the analysis are collected in [Tab. 20](#).

Two accounting boundaries require explicit definition because they are deliberate choices that influence the resulting metrics.

Carrier and cleaning solvent are excluded from the mass accounting. Each experiment consumes 25 mL of acetonitrile outside the reaction itself: 10 mL as carrier solvent to transport and separate the 0.65 mL reaction slug, and 15 mL to clean the fluidic lines between experiments. Neither contributes to the reaction. Both are requirements of operating the segmented-flow platform in serial-screening mode rather than of the chemistry itself, and neither would be required in the same form in a scaled process. Together they account for approximately 97% of the total mass consumed by an experiment; including them would therefore predominantly report the material requirements of the delivery system rather than those of the reaction. They are retained in the cost accounting, where they represent a real expense, and are reported separately in [Tab. 23](#) for completeness.

No scale-up was performed and no purification is accounted for. All figures describe the reaction as executed at the 0.65 mL slug scale, with yields determined in-line. Work-up, quench, extraction and chromatographic purification are excluded. The reported mass intensities therefore represent a lower bound on those of a complete process and are not directly comparable with literature PMI values calculated from isolated, purified product. They nevertheless provide a consistent comparison between the two campaigns, which share the same accounting boundary.

C.7.2 Cost of the campaign and of an experiment

On consumables, the agent-directed campaign cost 43% of the human-directed campaign, while using 46% of the experiments and 39% of the starting material. Twenty-one of its 23 experiments returned non-zero yield, compared with 23 of 50 in Campaign A. The consumable cost per experiment was similar, €4.26 against €4.59, showing that the saving derives primarily from the smaller number of experiments rather than from cheaper individual runs. Inference forms the largest additional cost of the agent-directed campaign: at \$109.55, model calls cost more than the chemistry they directed. Of the campaign’s 114.5M input tokens, 105.1M were served from cache. Including inference raises the fully loaded cost to €8.81 per experiment, but Campaign B remains 13% cheaper overall. We therefore report the fully loaded campaign cost alongside the consumables-only figure.

The campaigns were not run to the same length, with Campaign A receiving approximately 2.2 times as many experiments. At the agent’s consumable spend of €98.00, Campaign A had completed 18 experiments and reached a best yield of 63.9%, compared with 58.8% for Campaign B after 23 experiments. Truncated instead at 23 experiments, Campaign A had spent €115.73 and had reached the same 63.9% yield. Its best result within those first 23 experiments occurred at run 10 and originated from the initialization design rather than from an optimizer proposal; the campaign did not improve upon it until run 29. We therefore do not claim that either strategy searches more effectively than the other: a single pair of campaigns cannot support such a conclusion.

Table 20 Assumptions underlying the cost and mass model.

Quantity	Value	Basis
Reaction slug volume	0.65 mL	per experiment
Carrier solvent	10 mL	per experiment; cost only
Cleaning solvent	15 mL	per experiment; cost only
Reaction solvent (MeCN)	€196/L	delivered
Carrier/cleaning solvent (MeCN)	€120/L	delivered
Solvent density (MeCN, 20 °C)	0.786 g/mL	mass basis for PMI
Product molecular weight	261 g/mol	—
Reagent recovery, bulk discount	none	conservative assumption
LED module draw at 100%	100 W	electrical, not photon output
Platform base load	500 W	pumps, analytics, chiller, PC
Electricity price	€0.2046/kWh	see Sec. C.7.4
Operator time	excluded	not measured
Work-up and purification	excluded	not performed

Table 21 Cost of each campaign and of an individual experiment within it. Inference is converted at €0.92/\$.

	Campaign A (human-directed)	Campaign B (agent-directed)	Unit
Experiments executed	50	23	runs
Experiments with non-zero yield	23	21	runs
Starting material consumed	3.80	1.50	mmol
Best yield found	70.9	58.8	%
Reagents	73.19	26.07	€
Solvent (reaction + auxiliary)	156.37	71.93	€
Consumables subtotal	229.56	98.00	€
Electricity	2.96	3.79	€
LLM inference	—	100.79	€
Total campaign cost	232.51	202.58	€
Cost per experiment, consumables	4.59	4.26	€/run
Cost per experiment, fully loaded	4.65	8.81	€/run

C.7.3 Mass efficiency

Mass efficiency is reported at two boundaries because the resulting metrics answer different questions and rank the optimum conditions differently. Definitions and boundary conventions follow the CHEM21 metrics toolkit (69), which sets out PMI, RME and atom economy and structures their application according to the stage of research. The figures reported here correspond to its early-stage treatment, in which solvent recovery and downstream processing are not yet assessed.

Process mass intensity (PMI) is defined as the total mass of material charged to the reaction divided by the mass of product formed:

$$\text{PMI} = \frac{m_{\text{reactants}} + m_{\text{reaction solvent}}}{m_{\text{product}}}. \quad (1)$$

PMI is the conventional high-level mass metric and generally includes reaction solvent (69). Carrier and cleaning solvent are excluded from Eq. 1 according to the boundary defined above. The limitation of PMI in the present optimum-level comparison is that all three reported conditions use a substrate concentration of 100 mM and therefore charge an identical 0.511 g of reaction solvent, accounting for 87–92% of the input mass. Consequently, solvent-inclusive PMI is dominated by yield and contains comparatively little information about differences in reagent stoichiometry.

Reaction mass efficiency (RME) is calculated here as the mass of product formed divided by the total mass of substrate, photocatalyst and stoichiometric reagents charged, excluding solvent:

$$\text{RME} = 100 \frac{m_{\text{product}}}{m_{\text{reactants}}}. \quad (2)$$

The corresponding solvent-free PMI is

$$\text{PMI}_{\text{solvent-free}} = \frac{m_{\text{reactants}}}{m_{\text{product}}} = \frac{100}{\text{RME}}, \quad (3)$$

where RME is expressed in percent in the final equality. Removing the dominant solvent contribution exposes the trade-off between conversion and reagent stoichiometry. Campaign B was fixed at 100 mM throughout, whereas Campaign A explored 100–200 mM but reached its optimum at 100 mM. The solvent-free metrics therefore provide a direct view of the mass-economy dimension explicitly included in the agent’s multi-objective optimization.

For completeness, we additionally report a mole-based material efficiency, defined as

$$\eta_{\text{mol}} = 100 \frac{n_{\text{product}}}{\sum_i n_i}, \quad (4)$$

where the denominator is the total amount, in moles, of substrate, photocatalyst and stoichiometric reagents charged. This quantity is useful as a molecular-weight-independent view of material use, but is not treated as a headline metric because it assigns equal weight to one mole of a light reagent and one mole of a heavy reagent. RME has the clearer precedent in the green-chemistry literature.

Neither mass boundary is sufficient on its own. We therefore report both. Tab. 22 compares the individual optimum conditions, while Tab. 23 applies the same accounting to all material consumed over each search. The latter quantities describe the material efficiency of the optimization campaign rather than that of an individual reaction condition.

Two sets of conditions are reported for Campaign B. Run 17 is the optimum returned during the multi-objective phase, in which reagent economy was considered alongside yield; run 21 is the highest yield reached after the campaign was refocused on yield alone. Reporting only one would incompletely describe the campaign: the former represents the conditions selected by its stated multi-objective function, whereas the latter represents the highest yield subsequently achieved on the platform.

At the solvent-inclusive boundary, the human-directed optimum has the lowest PMI, 49.0 compared with 56.3 and 67.6 for the two agent-derived conditions. Because the reaction solvent mass is identical and dominates the total mass input at all three optima, this ordering is primarily determined by yield.

Table 22 Mass efficiency of the optimum conditions, at experiment scale. Campaign B is given at both candidate optima: run 17, returned under the multi-objective phase, and run 21, the highest yield reached after the yield-only refocus. Lower PMI is better; higher RME is better.

	Campaign A	Campaign B		Unit
	run 29	run 17	run 21	
Yield	70.9	48.5	58.8	%
TFAA loading	3.5	1.6	2.0	equiv.
N-oxide loading	2.4	1.4	1.4	equiv.
Total charged	6.9	4.1	4.4	equiv.
Substrate	0.0136	0.0136	0.0136	g
Photocatalyst	0.0001	0.0001	0.0001	g
TFAA	0.0500	0.0231	0.0286	g
N-oxide	0.0149	0.0089	0.0086	g
Reactant mass	0.0786	0.0456	0.0508	g
Reaction solvent	0.5109	0.5109	0.5109	g
Product formed	0.01204	0.00823	0.00998	g
PMI, incl. solvent	49.0	67.6	56.3	kg/kg
RME	15.3	18.0	19.6	%
PMI, solvent-free	6.53	5.55	5.09	kg/kg
Molar efficiency	10.3	12.0	13.4	%
Solvent share of input mass	86.7	91.8	91.0	%
Cost of that experiment	4.16	4.10	4.11	€
Cost per mmol product	90.3	130.0	107.6	€/mmol

Table 23 Mass efficiency of each campaign as a whole. Every experiment, successful or not, is charged against the product formed during the search; these figures therefore describe the search rather than the chemistry. Carrier and cleaning solvent are excluded from the metrics above the final rule and given below it for reference.

	Campaign A	Campaign B	Unit
Reactant mass charged	4.18	1.05	g
Reaction solvent charged	25.55	11.75	g
Product formed	0.1505	0.0993	g
Input mass per experiment	0.594	0.557	g/run
PMI, incl. solvent	197	129	kg/kg
RME	3.60	9.44	%
PMI, solvent-free	27.8	10.6	kg/kg
Molar efficiency	2.49	6.50	%
Non-reaction solvent charged	982.5	452.0	g
PMI including it	6725	4681	kg/kg

At the solvent-free boundary, the ordering reverses and both agent-derived conditions are more mass-efficient. The agent converged on leaner stoichiometry: 1.6–2.0 equivalents of TFSA compared with 3.5, and 1.4 equivalents of N-oxide compared with 2.4, corresponding to total reagent loadings of 4.1 and 4.4 equivalents against 6.9. This reduction in reagent mass outweighs the lower conversion. RME increases from 15.3% for the human-directed optimum to 18.0% at run 17 and 19.6% at run 21, while solvent-free PMI decreases from 6.53 to 5.55 and 5.09, respectively. Molar efficiency gives the same ordering, at 10.3%, 12.0% and 13.4%. The agent therefore identified conditions requiring less reagent mass per unit of product formed despite their lower yield.

The effects are amplified at campaign level. Campaign B reaches an RME of 9.44%, compared with 3.60% for Campaign A, and a solvent-free PMI of 10.6 compared with 27.8. These values reflect both the leaner conditions explored and the smaller fraction of experiments that produced no detectable product: Campaign A contained 27 zero-yield experiments compared with two in Campaign B. These campaign-level quantities describe the material efficiency of the search, rather than that of an individual reaction condition, and are reported as such.

None of the three optimum conditions is mass-efficient in absolute terms. RME remains below 20% and solvent-inclusive PMI at or above approximately 50, while dilution — a major contributor to PMI — was not an optimization variable in Campaign B and was only narrowly varied in Campaign A. The purpose of this comparison is therefore to compare resource use during two optimization campaigns on the same chemistry, rather than to claim that either outcome constitutes a green process.

C.7.4 Electricity

Electricity is charged at €0.2046/kWh, the Eurostat figure for non-household consumers in the Netherlands in the annual-consumption band 500–2000 MWh for the first half of 2025, excluding VAT and other recoverable taxes and levies (Eurostat online data code `nrg_pc_205`) (70). The EU-wide non-household average over the same period was €0.1902/kWh, so the result is insensitive to the precise tariff used (71).

The LED module is charged according to its rated electrical draw, scaled by the recorded intensity set-point, over a lamp-on window taken as three times the residence time to account for slug transit and purge. The

platform base load covers the pumps, in-line analytics, chiller and control computer and is charged for every hour for which the platform is occupied. Campaign A occupancy is reconstructed from acquisition timestamps, giving a median cycle time of 32.8 min across 50 experiments, whereas Campaign B occupancy of 34 h is measured directly from the agent trace. Electricity contributes less than 2% of the fully loaded cost of either campaign at the adopted tariff and is marginally higher for Campaign B because the platform was occupied for longer. It is included to make the accounting boundary explicit rather than because it materially affects the comparison.

C.7.5 Limitations

This comparison comprises two campaigns rather than a controlled benchmark. They were performed on the same platform at different times but were not matched in search space, initialization design or campaign length, and the human-directed campaign was not blinded to prior knowledge of the chemistry. Platform occupancy was higher for Campaign B, at approximately 89 min per experiment compared with 33 min for Campaign A. These values are not strictly equivalent measurements: Campaign A occupancy is reconstructed from acquisition timestamps, whereas Campaign B includes agent-driven setup and verification captured in the execution trace.

The campaigns also converged on different reaction conditions. Campaign A selected $\text{Ru}(\text{bpy})_3(\text{PF}_6)_2$ at full lamp intensity and a two-minute residence time, whereas Campaign B selected $\text{Ru}(\text{bpy})_3\text{Cl}_2 \cdot 6\text{H}_2\text{O}$ at half intensity and substantially longer residence times. Their per-experiment costs and mass intensities therefore describe the conditions reached by each search and should not be interpreted as interchangeable measurements of a single process. Yields are in-line determinations on unpurified reaction mixtures, so all PMI values exclude isolation losses and would increase on an isolated-product basis. Reagent costs use research-scale list prices, while inference pricing corresponds to one model at one point in time and is the most rapidly changing component of the cost analysis. The figures therefore provide a resource accounting of the two optimization campaigns rather than a controlled benchmark of autonomous against human-directed BO.